FIT1058 — Notes

Course Note(CN) + Rosen Discrete Mathematic (Rosen)

Alston

25 August 2026

Contents

1 Sets **4**
- 1.1 Sets and elements . . . 4
- 1.2 Specifying sets . . . 4
- 1.3 Cardinality . . . 5
- 1.4 Sets of numbers . . . 5
- 1.5 Sets of strings . . . 5
- 1.6 Subsets and supersets . . . 5
- 1.7 Minimal, minimum, maximal, maximum . . . 6
- 1.8 Counting all subsets . . . 6
- 1.9 The power set of a set . . . 6
- 1.10 Counting subsets by size using binomial coefficients . . . 7
- 1.11 Complement and set difference . . . 7
- 1.12 Union and intersection . . . 8
- 1.13 Symmetric difference . . . 10
- 1.14 Cartesian product . . . 10
- 1.15 Partitions . . . 11
- 1.16 Cardinality of infinite sets (Rosen §2.5 – extension, beyond CN syllabus) . . . 11

2 Functions **13**
- 2.1 Definitions . . . 13
  - 2.1.1 Domain . . . 13
  - 2.1.2 Codomain & co. . . . 13
  - 2.1.3 Rule . . . 14
- 2.2 Functions in computer science . . . 14
  - 2.2.1 Functions from analysis . . . 14
  - 2.2.2 Functions in mathematics and programming . . . 14
- 2.3 Notation . . . 14
- 2.4 Some special functions . . . 15
- 2.5 Functions with multiple arguments and values . . . 15
- 2.6 Restrictions . . . 16
- 2.7 Injections, surjections, bijections . . . 16
- 2.8 Inverse functions . . . 17
- 2.9 Composition . . . 18
- 2.10 Cryptosystems . . . 19
- 2.11 Loose composition . . . 20

2.12 Counting functions . . . 20
2.13 Binary relations . . . 21
2.14 Properties of binary relations . . . 22
2.15 Combining binary relations . . . 23
2.16 Equivalence relations . . . 25
2.17 Relations . . . 26
2.18 Counting relations . . . 27
2.19 Matrices (Rosen §2.6 – extension, beyond CN syllabus) . . . 28

3 Proofs **30**
3.1 Theorems and proofs . . . 30
3.2 Logical deduction . . . 30
3.3 Proofs of existential and universal statements . . . 31
3.4 Finding proofs . . . 32
3.5 Types of proofs . . . 33
3.6 Proof by symbolic manipulation . . . 34
3.7 Proof by construction . . . 34
3.8 Proof by cases . . . 35
3.9 Proof by contradiction . . . 36
3.10 Proof by mathematical induction . . . 37
3.11 Induction: more examples . . . 38
3.12 Induction: extended example . . . 45
3.13 Mathematical induction and statistical induction . . . 46
3.14 Programs and proofs . . . 46

4 Propositional Logic **48**
4.1 Truth values . . . 48
4.2 Boolean variables . . . 48
4.3 Propositions . . . 48
4.4 Logical operations . . . 48
4.5 Negation . . . 48
4.6 Conjunction . . . 49
4.7 Disjunction . . . 49
4.8 De Morgan's Laws . . . 50
4.9 Implication . . . 51
4.10 Equivalence . . . 51
4.11 Exclusive-or . . . 51
4.12 Tautologies and logical equivalence . . . 52
4.13 History . . . 53
4.14 Distributive Laws . . . 53
4.15 Laws of Boolean algebra . . . 53
4.16 Disjunctive Normal Form . . . 54
4.17 Conjunctive Normal Form . . . 54
4.18 Satisfiability . . . 55
4.19 Representing logical statements . . . 56
4.20 Statements about how many variables are true . . . 56
4.21 Universal sets of operations . . . 56

5 Predicate Logic **58**
5.1 Relations, predicates, and truth-valued functions . . . 58
5.2 Variables and constants . . . 58
5.3 Predicates and variables . . . 58
5.4 Arguments of predicates . . . 59
5.5 Building logical expressions with predicates . . . 60
5.6 Existential quantifier . . . 60
5.7 Restricting existentially quantified variables . . . 61
5.8 Universal quantifier . . . 61
5.9 Restricting universally quantified variables . . . 61
5.10 Multiple quantifiers . . . 62
5.11 Predicate logic expressions . . . 62
5.12 Doing logic with quantifiers . . . 63
5.13 Duality between quantifiers . . . 63
5.14 Some examples . . . 63
5.15 Summary of rules for logic with quantifiers . . . 66
5.16 Rules of inference (Rosen §1.6 – extension, beyond CN syllabus) . . . 66

6 Sequences & Series **68**
6.1 Definitions and notation . . . 68
6.2 Recursive definitions of sequences . . . 68
6.3 Arithmetic sequences . . . 69
6.4 Geometric sequences . . . 69
6.5 Harmonic sequences . . . 69
6.6 From recursive definitions to formulas . . . 69
6.7 The Fibonacci sequence . . . 74
6.7.1 Upper bounds . . . 74
6.7.2 Lower bounds . . . 75
6.7.3 Asymptotic behaviour . . . 75
6.7.4 An exact formula . . . 75
6.8 Limits of infinite sequences . . . 76
6.9 Big-O notation . . . 80
6.10 Sums and summation . . . 82
6.11 Finite series . . . 83
6.12 Finite arithmetic series . . . 83
6.13 Finite geometric series . . . 84
6.14 Infinite geometric series . . . 84
6.15 Harmonic numbers . . . 84

Notation summary **86**

1 Sets

1.1 Sets and elements

Note (Naive set theory). We work within *naive set theory*: a set is a primitive, undefined notion, and any well-specified collection of objects is assumed to form a set. This suffices for ordinary practice, but unrestricted set formation is inconsistent in general – *Russell's paradox* considers $R = \{x : x \notin x\}$, for which $R \in R \leftrightarrow R \notin R$, a contradiction. Axiomatic set theory (e.g. ZFC) resolves this by restricting which collections may form sets.

Definition 1.1 (Set)**.** A set A is an unordered collection of distinct objects, called *elements* (or *members*) of A.

Definition 1.2 (Membership)**.**

$$a \in A \iff a \text{ is an element of } A, \qquad a \notin A \iff \neg(a \in A).$$

Definition 1.3 (Set equality)**.**

$$A = B \iff \forall x\,(x \in A \leftrightarrow x \in B).$$

Definition 1.4 (Empty set)**.** $\emptyset = \{\}$ is the unique set satisfying $\forall x\,(x \notin \emptyset)$.

Definition 1.5 (Singleton, unordered pair)**.** $\{a\}$ has the sole element a; $\{a, b\}$ has elements a, b, with $\{a, b\} = \{b, a\}$.

Definition 1.6 (Standard number sets)**.**

$\mathbb{N} = \{0, 1, 2, 3, \dots\}$	(natural numbers)
$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$	(integers)
$\mathbb{Z}^+ = \{1, 2, 3, \dots\}$	(positive integers)
$\mathbb{Q} = \{p/q : p \in \mathbb{Z},\ q \in \mathbb{Z},\ q \neq 0\}$	(rationals)
$\mathbb{R}$	(reals)
$\mathbb{C}$	(complex numbers)

Note. Rosen takes $0 \in \mathbb{N}$; confirm this matches CN §1.4 before relying on it elsewhere.

Definition 1.7 (Universal set)**.** The *universal set* U is the set containing all objects under discussion in a given context; every set considered is implicitly $A \subseteq U$.

Definition 1.8 (Venn diagram)**.** A *Venn diagram* represents sets as regions enclosed by curves inside a rectangle representing U; overlapping regions represent intersections, non-overlapping regions represent disjointness.

1.2 Specifying sets

Definition 1.9 (Roster method)**.** $A = \{a_1, a_2, \dots, a_n\}$. Order and repetition do not matter: $\{1, 2, 3\} = \{3, 2, 1\} = \{1, 1, 2, 3\}$.

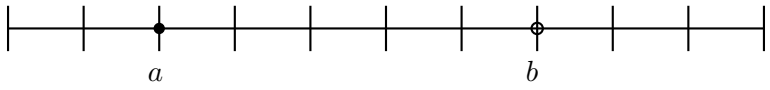
Definition 1.10 (Ellipsis notation)**.** $\{1, 2, 3, \dots\}$, or bounded: $\{1, 2, \dots, 100\}$.

Definition 1.11 (Set-builder notation)**.** $A = \{x \in U : P(x)\}$, equivalently $\{x \mid P(x)\}$, where P is a predicate.

Example 1.12. $\{x \in \mathbb{Z} : x^2 < 9\} = \{-2, -1, 0, 1, 2\}$.

Definition 1.13 (Interval notation, $a, b \in \mathbb{R}$, $a < b$)**.**

$$[a, b] = \{x \in \mathbb{R} : a \leq x \leq b\}, \qquad (a, b) = \{x \in \mathbb{R} : a < x < b\},$$
$$[a, b) = \{x \in \mathbb{R} : a \leq x < b\}, \qquad (a, b] = \{x \in \mathbb{R} : a < x \leq b\}.$$



$[a, b)$: filled dot at a (included), open circle at b (excluded)

1.3 Cardinality

Definition 1.14 (Cardinality)**.** For finite A, $|A|$ is the number of distinct elements of A. $|\emptyset| = 0$.

Definition 1.15 (Finite, infinite set)**.** A is *finite* if $A = \emptyset$ or there exists $n \in \mathbb{Z}^+$ and a bijection $\{1, 2, \ldots, n\} \to A$; then $|A| = n$. If no such n exists, A is *infinite*.

1.4 Sets of numbers

Definition 1.16 (Signed subsets)**.**

$$\mathbb{Z}^+ = \{1, 2, 3, \ldots\}, \quad \mathbb{Z}^- = \{-1, -2, -3, \ldots\}, \quad \mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}, \quad \mathbb{R}^- = \{x \in \mathbb{R} : x < 0\}.$$

Note. $\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R} \subseteq \mathbb{C}$.

1.5 Sets of strings

Definition 1.17 (Alphabet, string, length)**.** An *alphabet* Σ is a finite nonempty set of symbols. A *string* over Σ is a finite sequence of symbols from Σ. $|w|$ is the length of w; ε is the *empty string*, $|\varepsilon| = 0$.

Definition 1.18 (Σ^*, Σ^n)**.**

$$\Sigma^n = \{w : w \text{ over } \Sigma,\ |w| = n\}, \qquad \Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n.$$

1.6 Subsets and supersets

Definition 1.19 (Subset, superset)**.**

$$A \subseteq B \iff \forall x\,(x \in A \to x \in B), \qquad A \supseteq B \iff B \subseteq A.$$

Definition 1.20 (Proper subset)**.**

$$A \subsetneq B \iff (A \subseteq B) \wedge (A \neq B).$$

Theorem 1.21. *For every set A, $\emptyset \subseteq A$.*

Proof. By definition $\emptyset \subseteq A$ iff $\forall x\,(x \in \emptyset \to x \in A)$. Since $x \in \emptyset$ is always false, the conditional is vacuously true for every x, so the universally quantified statement holds. Hence $\emptyset \subseteq A$. ■

Theorem 1.22. *$\subseteq$ is reflexive ($A \subseteq A$) and transitive ($A \subseteq B \wedge B \subseteq C \Rightarrow A \subseteq C$).*

1.7 Minimal, minimum, maximal, maximum

Example 1.23 (Motivating example: cliques)**.** Let B be a set of people, with some pairs acquainted. A *clique* is a subset $A \subseteq B$ such that every two people in A know each other. We want to describe the "biggest" cliques – but "biggest" has two different meanings.

Definition 1.24 (Maximum, maximal, w.r.t. a family $\mathcal{F}$ of sets sharing a property)**.** Let $\mathcal{F}$ be a family of subsets of some set, all sharing a given property (e.g. "is a clique"), ordered by $\subseteq$.

- $M \in \mathcal{F}$ is a *maximum* if $S \subseteq M$ for every $S \in \mathcal{F}$ (largest possible size).
- $M \in \mathcal{F}$ is *maximal* if M is not a proper subset of any $S \in \mathcal{F}$ (cannot be enlarged while keeping the property).

Theorem 1.25. *Every maximum is maximal.*

Proof. If M is a maximum, then $S \subseteq M$ for all $S \in \mathcal{F}$, so no $S \in \mathcal{F}$ can have $M \subsetneq S$ (that would force $M \subsetneq S \subseteq M$, impossible). Hence M is maximal. ■

Note. The converse fails in general: a maximal clique need not be a maximum clique – it just cannot be enlarged, while some other, larger clique may exist elsewhere in B.

Definition 1.26 (Minimum, minimal – dual definitions)**.** Dually, for $\mathcal{F}$ ordered by $\subseteq$:

- $m \in \mathcal{F}$ is a *minimum* if $m \subseteq S$ for every $S \in \mathcal{F}$.
- $m \in \mathcal{F}$ is *minimal* if m is not a proper superset of any $S \in \mathcal{F}$ (removing anything from m destroys the property).

Every minimum is minimal; the converse fails in general.

Note (Why the distinction matters here but not for $\mathbb{R}$). For real numbers, $\leq$ is a *total order*: any two reals are comparable, so "cannot be increased" and "is the largest" coincide, and *maximum*/*maximal* are used interchangeably. But $\subseteq$ is only a *partial order*: two sets A, B can be *incomparable* ($A \not\subseteq B$ and $B \not\subseteq A$). This is exactly why a family can have several maximal elements without any single maximum.

1.8 Counting all subsets

Theorem 1.27. *For finite B with $|B| = n$, the number of subsets of B is 2^n.*

Proof. A subset is determined by choosing, independently for each of the n elements, whether to include it (2 options per element). By the product rule, the number of independent choice-sequences is $\underbrace{2 \times 2 \times \cdots \times 2}_{n} = 2^n$, and each choice-sequence corresponds to exactly one subset. ■

1.9 The power set of a set

Definition 1.28 (Power set)**.** $\mathcal{P}(A) = \{S : S \subseteq A\}$.

Theorem 1.29. $|A| = n \Rightarrow |\mathcal{P}(A)| = 2^n$.

Proof. Immediate from §1.8: $\mathcal{P}(A)$ is exactly the set of subsets of A, and there are 2^n of them. ■

Note. Holds even for $A = \emptyset$: $n = 0$ gives $2^0 = 1$, consistent with $\emptyset$ having exactly one subset, itself. $\mathcal{P}(A)$ is also written 2^A.

Example 1.30. $\emptyset$ has exactly one subset, itself, so $\mathcal{P}(\emptyset) = \{\emptyset\}$. The set $\{\emptyset\}$ has exactly two subsets, $\emptyset$ and $\{\emptyset\}$ itself, so $\mathcal{P}(\{\emptyset\}) = \{\emptyset, \{\emptyset\}\}$.

1.10 Counting subsets by size using binomial coefficients

Definition 1.31 (Binomial coefficient)**.** For $0 \le k \le n$, $\binom{n}{k}$ ("n choose k") is the number of k-element subsets of an n-element set.

Theorem 1.32 (Sum over k)**.**

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

Proof. The binomial coefficients $\binom{n}{0}, \binom{n}{1}, \ldots, \binom{n}{n}$ partition the subsets of an n-set by size, so they sum to the total subset count, 2^n (§1.8). ■

Theorem 1.33 (Symmetry)**.**

$$\binom{n}{k} = \binom{n}{n-k}.$$

Proof. Choosing k elements to include determines which $n-k$ elements are excluded, and vice versa; this is a bijection between k-subsets and $(n-k)$-subsets. ■

Theorem 1.34 (Closed form)**.**

$$\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k!} = \frac{n!}{k!(n-k)!}.$$

Proof. Count *ordered* choices of k distinct elements from n: n options for the first, $n-1$ for the second, …, $n-k+1$ for the k-th, giving $n(n-1)\cdots(n-k+1) = n!/(n-k)!$ ordered sequences. Each k-subset is counted once for every ordering of its k elements, i.e. $k!$ times, so

$$\binom{n}{k} = \frac{1}{k!} \cdot \frac{n!}{(n-k)!} = \frac{n!}{k!(n-k)!}.$$

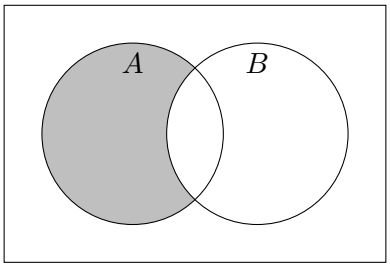
■

Example 1.35. $\binom{n}{0} = \binom{n}{n} = 1$ (only one way to choose nothing, or everything); $\binom{n}{1} = \binom{n}{n-1} = n$.

1.11 Complement and set difference

Definition 1.36 (Set difference, complement w.r.t. U)**.**

$$A - B = \{x : x \in A \wedge x \notin B\}, \qquad \overline{A} = U - A.$$



$A \setminus B$ (shaded)

Theorem 1.37 (De Morgan's laws)**.**

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \qquad \overline{A \cap B} = \overline{A} \cup \overline{B}.$$

Example 1.38. Prove $\overline{A \cap B} = \overline{A} \cup \overline{B}$ using set-builder notation and logical equivalences.

$$\begin{aligned}
\overline{A \cap B} &= \{x : x \notin A \cap B\} \\
&= \{x : \neg(x \in A \cap B)\} \\
&= \{x : \neg(x \in A \wedge x \in B)\} \\
&= \{x : \neg(x \in A) \vee \neg(x \in B)\} && \text{(De Morgan's law, logic)} \\
&= \{x : x \notin A \vee x \notin B\} \\
&= \{x : x \in \overline{A} \vee x \in \overline{B}\} \\
&= \overline{A} \cup \overline{B}.
\end{aligned}$$

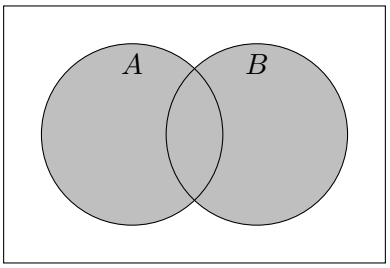
Example 1.39. Let A, B, C be sets. Prove $\overline{A \cup (B \cap C)} = (\overline{C} \cup \overline{B}) \cap \overline{A}$ by chaining previously established identities.

$$\begin{aligned}
\overline{A \cup (B \cap C)} &= \overline{A} \cap \overline{(B \cap C)} && \text{(1st De Morgan's law)} \\
&= \overline{A} \cap (\overline{B} \cup \overline{C}) && \text{(2nd De Morgan's law)} \\
&= (\overline{B} \cup \overline{C}) \cap \overline{A} && \text{(commutativity of } \cap\text{)} \\
&= (\overline{C} \cup \overline{B}) \cap \overline{A}. && \text{(commutativity of } \cup\text{)}
\end{aligned}$$

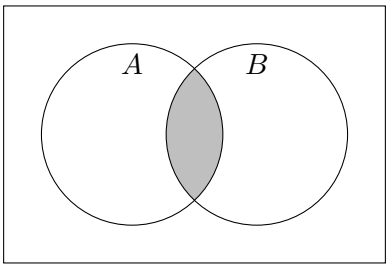
1.12 Union and intersection

Definition 1.40 (Union, intersection)**.**

$$A \cup B = \{x : x \in A \vee x \in B\}, \qquad A \cap B = \{x : x \in A \wedge x \in B\}.$$



$A \cup B$ (shaded)



$A \cap B$ (shaded)

Definition 1.41 (Disjoint)**.** $A \cap B = \emptyset$.

Theorem 1.42 (Inclusion–exclusion)**.** $|A \cup B| = |A| + |B| - |A \cap B|$.

Example 1.43. Prove $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ by the element argument.

($\subseteq$) Let $x \in A \cap (B \cup C)$. Then $x \in A$ and ($x \in B$ or $x \in C$). If $x \in B$ then $x \in A \cap B$; if $x \in C$ then $x \in A \cap C$. Either way $x \in (A \cap B) \cup (A \cap C)$.

($\supseteq$) Let $x \in (A \cap B) \cup (A \cap C)$. Then $x \in A \cap B$ or $x \in A \cap C$; either way $x \in A$ and ($x \in B$ or $x \in C$), so $x \in A \cap (B \cup C)$.

Both directions hold, so the sets are equal.

Note (Methods for proving set identities). (i) *Element argument (double inclusion)*: show LHS $\subseteq$ RHS and RHS $\subseteq$ LHS, as above.

(ii) *Membership table*: tabulate membership (0/1) of an arbitrary x in each named set for every combination, and compare the LHS/RHS columns.

(iii) *Using established identities*: rewrite one side into the other via a chain of already-proven identities (the Set Identities table), analogous to proving logical equivalences.

Definition 1.44 (Family of sets)**.** A *family of sets* indexed by I is a collection $\{A_i\}_{i\in I}$, one set A_i for each $i \in I$; I is the *index set*.

Definition 1.45 (Generalized union and intersection, finite)**.**

$$\bigcup_{i=1}^{n} A_i = A_1 \cup A_2 \cup \cdots \cup A_n = \{x : \exists i \in \{1,\ldots,n\},\ x \in A_i\},$$

$$\bigcap_{i=1}^{n} A_i = A_1 \cap A_2 \cap \cdots \cap A_n = \{x : \forall i \in \{1,\ldots,n\},\ x \in A_i\}.$$

Definition 1.46 (Generalized union and intersection, arbitrary index set)**.** For a family $\{A_i\}_{i\in I}$,

$$\bigcup_{i\in I} A_i = \{x : \exists i \in I,\ x \in A_i\}, \qquad \bigcap_{i\in I} A_i = \{x : \forall i \in I,\ x \in A_i\}.$$

When $I = \mathbb{Z}^+$ one also writes $\bigcup_{i=1}^{\infty} A_i$, $\bigcap_{i=1}^{\infty} A_i$.

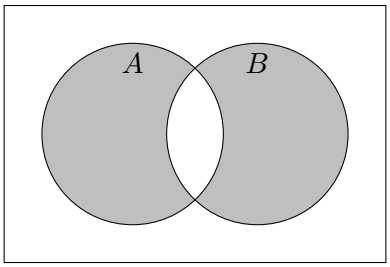
Theorem 1.47 (Set identities)**.** *For sets A, B, C with universal set U:*

$$\begin{aligned}
A \cup \emptyset = A, \quad A \cap U = A && \textit{(Identity laws)}\\
A \cup U = U, \quad A \cap \emptyset = \emptyset && \textit{(Domination laws)}\\
A \cup A = A, \quad A \cap A = A && \textit{(Idempotent laws)}\\
\overline{\overline{A}} = A && \textit{(Complementation law)}\\
A \cup B = B \cup A, \quad A \cap B = B \cap A && \textit{(Commutative laws)}\\
(A \cup B) \cup C = A \cup (B \cup C),\ (A \cap B) \cap C = A \cap (B \cap C) && \textit{(Associative laws)}\\
A \cup (B \cap C) = (A \cup B) \cap (A \cup C) && \textit{(Distributive law)}\\
A \cap (B \cup C) = (A \cap B) \cup (A \cap C) && \textit{(Distributive law)}\\
\overline{A \cup B} = \overline{A} \cap \overline{B}, \quad \overline{A \cap B} = \overline{A} \cup \overline{B} && \textit{(De Morgan's laws)}\\
A \cup (A \cap B) = A, \quad A \cap (A \cup B) = A && \textit{(Absorption laws)}\\
A \cup \overline{A} = U, \quad A \cap \overline{A} = \emptyset && \textit{(Complement laws)}
\end{aligned}$$

Definition 1.48 (Bit-string representation)**.** For finite $U = \{a_1, \ldots, a_n\}$ with a fixed ordering, $A \subseteq U$ corresponds to the bit string $b_1 b_2 \cdots b_n$ where $b_i = 1$ iff $a_i \in A$. Then $A \cup B \leftrightarrow$ bitwise OR, $A \cap B \leftrightarrow$ bitwise AND, $\overline{A} \leftrightarrow$ bitwise NOT.

1.13 Symmetric difference

Definition 1.49 (Symmetric difference)**.** $A\triangle B = \{x : x \in A \text{ or } x \in B \text{ but not both}\}$ (exclusive or).



$A\triangle B$ (shaded)

Theorem 1.50 (Equivalent forms)**.**

$$A\triangle B = (A \setminus B) \cup (B \setminus A) = (A \cap \overline{B}) \cup (\overline{A} \cap B) = (A \cup B) \setminus (A \cap B).$$

Note. $A\triangle A = \emptyset$, and this is the only case where a symmetric difference is empty.

Theorem 1.51. $A = B \iff A\triangle B = \emptyset$.

Proof.

$A = B \iff A \subseteq B$ and $B \subseteq A \iff A\setminus B = \emptyset$ and $B\setminus A = \emptyset \iff (A\setminus B)\cup(B\setminus A) = \emptyset \iff A\triangle B = \emptyset$.

■

Theorem 1.52. $\overline{A\triangle B} = \overline{A}\triangle\overline{B}$.

Proof.

$$\overline{A\triangle B} = \overline{(A \cap \overline{B}) \cup (\overline{A} \cap B)} = \overline{(A \cap \overline{B})} \cap \overline{(\overline{A} \cap B)} = (\overline{A} \cup B) \cap (A \cup \overline{B}).$$

Expanding and simplifying (using distributivity and $A\cap\overline{A} = \emptyset$) gives $(A\cap\overline{B})\cup(\overline{A}\cap B) = A\triangle B$, so the complement identity holds by the same characterization applied to $\overline{A}, \overline{B}$: $\overline{A}\triangle\overline{B} = (\overline{A}\cap B)\cup(A\cap\overline{B}) = A\triangle B = \overline{A\triangle B}$. ■

1.14 Cartesian product

Definition 1.53 (Ordered pair)**.** (a, b) collects a, b in order; $(a, b) = (c, d) \iff a = c \wedge b = d$.

Definition 1.54 (Cartesian product)**.** $A \times B = \{(a, b) : a \in A,\ b \in B\}$.

Theorem 1.55. $|A \times B| = |A| \cdot |B|$ *for finite* A, B.

Proof. There are $|A|$ choices for the first coordinate, and independently $|B|$ choices for the second, so by the product rule there are $|A| \cdot |B|$ pairs. ■

Definition 1.56 (n-tuple, generalized Cartesian product)**.** For sets $A_1, \ldots, A_n$:

$$A_1 \times A_2 \times \cdots \times A_n = \{(a_1, \ldots, a_n) : a_i \in A_i,\ i = 1, \ldots, n\},$$

with equality of n-tuples componentwise: $(a_1, \ldots, a_n) = (b_1, \ldots, b_n) \iff a_i = b_i\ \forall i$. If finite, $|A_1 \times \cdots \times A_n| = |A_1| \cdots |A_n|$.

Definition 1.57 (Power notation)**.** For a set A and $n \in \mathbb{N}$, $A^n = A \times A \times \cdots \times A$ (n factors); $A^0 = \{\emptyset\text{-tuple}\}$ is conventionally treated as a single-element set, $A^1 = A$. If A is finite, $|A^n| = |A|^n$.

Note. When A is an alphabet, A^n is identified with strings of length n (§1.5): e.g. $\{0,1\}^3 = \{000, 001, \ldots, 111\}$. Also $\mathbb{R}^n = \mathbb{R} \times \cdots \times \mathbb{R}$ is the set of coordinates in n-dimensional space.

Theorem 1.58. $|A_1 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$ *for finite* A_i.

1.15 Partitions

Definition 1.59 (Partition)**.** A *partition* of A is a set of pairwise disjoint, nonempty subsets of A (called *parts*, *blocks*, or *cells*) whose union is A.

Note (Equivalent characterization). A collection of nonempty subsets of A is a partition of A iff every element of A belongs to *exactly one* part.

Example 1.60. For $A = \{a, b, c\}$: $\{\{a, c\}, \{b\}\}$ and $\{\{a\}, \{b\}, \{c\}\}$ are partitions; $\{\{a, b\}, \{c\}, \emptyset\}$ fails (empty part), $\{\{a, b\}, \{b, c\}\}$ fails (not disjoint), $\{\{a\}, \{b\}\}$ fails (union $\neq A$, misses c).

Definition 1.61 (Coarsest and finest partitions)**.** For any A: the *coarsest* partition is $\{A\}$ (one part, everything together); the *finest* partition is $\{\{a\} : a \in A\}$ (one singleton part per element). If $|A| = 1$ these coincide; otherwise they are the two extremes, with every other partition of A intermediate between them.

Note (Connection to equivalence relations (Rosen §9.5, previewed here)). Partitions are closely tied to *equivalence relations* (reflexive, symmetric, transitive relations – CN §2.16, Rosen §9.5): every equivalence relation R on A partitions A into *equivalence classes* $[a]_R = \{x \in A : x\,R\,a\}$, and conversely every partition of A arises this way from exactly one equivalence relation (namely, $x\,R\,y \iff x, y$ lie in the same part). We revisit this correspondence formally once binary relations are covered.

1.16 Cardinality of infinite sets (Rosen §2.5 – extension, beyond CN syllabus)

Definition 1.62 (Same cardinality)**.** A, B have the *same cardinality*, $|A| = |B|$, iff there exists a bijection $A \to B$.

Definition 1.63 (Countable)**.** A is *countable* if A is finite, or $|A| = |\mathbb{N}|$ (a bijection $\mathbb{N} \to A$ exists – A can be listed $a_0, a_1, a_2, \ldots$). Otherwise A is *uncountable*.

Theorem 1.64. $\mathbb{Z}$ *is countable.*

Proof. $f : \mathbb{N} \to \mathbb{Z}$, $f(n) = n/2$ if n even, $-(n+1)/2$ if n odd, is a bijection: $0, 1, 2, 3, 4, \cdots \mapsto 0, -1, 1, -2, 2, \ldots$, hitting every integer exactly once. ■

Theorem 1.65 (Countability of $\mathbb{Q}$, Cantor's diagonal enumeration)**.** $\mathbb{Q}$ *is countable.*

Proof sketch. Arrange positive rationals p/q in a grid, row p, column q. Traverse the grid along successive diagonals ($p + q = 2, 3, 4, \ldots$), skipping any p/q already listed in lowest terms. This lists every positive rational exactly once; interleave with negatives and 0 as in the $\mathbb{Z}$ proof above. ■

Theorem 1.66 (Uncountability of $\mathbb{R}$, Cantor's diagonal argument – harder)**.** $\mathbb{R}$ *(even just* $(0, 1)$*) is uncountable.*

Proof. Suppose, for contradiction, $(0,1)$ were countable: list all its elements as $r_1, r_2, r_3, \ldots$, each written as an infinite decimal $r_i = 0.d_{i1}d_{i2}d_{i3}\ldots$.

Construct $s = 0.s_1s_2s_3\ldots$ by setting $s_i = 5$ if $d_{ii} \neq 5$, else $s_i = 6$ (any rule avoiding both d_{ii} and decimal-expansion ambiguity works).

Then $s \in (0,1)$, but $s \neq r_i$ for every i (they differ in digit i), so s is missing from the list – contradicting that the list was exhaustive.

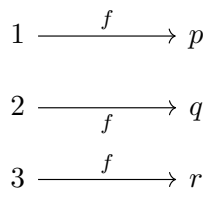
So no such list exists: $(0,1)$ is uncountable. ■

Note (Consequence). Since $\mathbb{Q}$ is countable but $\mathbb{R}$ is not, $|\mathbb{Q}| \neq |\mathbb{R}|$ – there are strictly more reals than rationals, despite both being infinite. This is the first proof most students see that not all infinities are the same size.

2 Functions

2.1 Definitions

Definition 2.1 (Function)**.** A function f consists of: a set called the *domain*; a set called the *codomain*; and a rule assigning, to each x in the domain, a unique value $f(x)$ in the codomain.



A function $f : \{1, 2, 3\} \to \{p, q, r\}$, each argument pointing to its unique value

Definition 2.2 (Rosen's phrasing)**.** Let A, B be nonempty sets. A *function* f from A to B is an assignment of exactly one element of B to each element of A. We write $f(a) = b$ if b is the element assigned to a, and $f : A \to B$.

Note. x is the *argument* (parameter, input); $f(x)$ is the *value* (output).

Definition 2.3 (Equality of functions)**.** $f = g$ iff $\mathrm{dom}(f) = \mathrm{dom}(g)$, their codomains agree, and $f(x) = g(x)$ for every $x \in \mathrm{dom}(f)$.

Example 2.4. Let $\mathrm{CubeSum4}_{\mathbb{Z}}(w, x, y, z) = w^3 + x^3 + y^3 + z^3$ on $\mathrm{dom} = \mathbb{Z}^4$, and $\mathrm{CubeSum4}_{\mathbb{R}}$ the same rule on $\mathrm{dom} = \mathbb{R}^4$. Despite the identical rule, these are *different* functions, since equality of functions requires equal domains.

2.1.1 Domain

Definition 2.5 (Domain)**.** The *domain* of f, written $\mathrm{dom}(f)$, is exactly the set of valid arguments: $f(x)$ must be defined for every $x \in \mathrm{dom}(f)$, and undefined for every $x \notin \mathrm{dom}(f)$.

Note (The domain as a promise). Specifying $\mathrm{dom}(f)$ commits f to working correctly on *every* member of it – no more, no less. A larger domain is a bigger promise (more work to guarantee); a smaller domain that still covers the intended use case is often preferable.

2.1.2 Codomain & co.

Definition 2.6 (Codomain)**.** The *codomain* of f is a set containing every possible value $f(x)$ for $x \in \mathrm{dom}(f)$, but is allowed to be a *loose* superset – it may contain elements never actually attained.

Definition 2.7 (Image)**.** The *image* of f is the exact set of attained values, $\{f(x) : x \in \mathrm{dom}(f)\}$ – a subset of the codomain, not necessarily proper.

Note. CN deliberately avoids the word "range", since usage is inconsistent (sometimes the image, sometimes the codomain). Rosen uses "range" for what CN calls the image.

Note (Why allow looseness at all?). The image is often hard, or even provably impossible, to compute exactly (e.g. a function whose value depends on uncomputable properties of programs), while a simple superset codomain is easy to state. So we specify a codomain we're confident covers every value, rather than insisting on the exact image.

Definition 2.8 (Image of a set, Rosen)**.** For $f : A \to B$ and $S \subseteq A$, the *image of S under f* is

$$f(S) = \{f(s) : s \in S\} \subseteq B.$$

(The image of f itself, above, is the special case $f(A)$.)

Definition 2.9 (Onto / surjective, preview)**.** If the codomain equals the image, f is *onto* (*surjective*), a *surjection*. (Formal treatment in §2.7.)

2.1.3 Rule

Definition 2.10 (Rule)**.** The *rule* of f specifies, for each $x \in \mathrm{dom}(f)$, what $f(x)$ is – it need not specify *how* to compute it (no algorithm required).

Definition 2.11 (Graph of a function)**.** The *graph* of f is $\{(x, f(x)) : x \in \mathrm{dom}(f)\}$, the set of all argument–value pairs.

Example 2.12. A function on the finite domain $\{1, 2, 3, 4\}$ can be given as a finite graph, e.g. $\{(1, a), (2, a), (3, b), (4, c)\}$ (its image is $\{a, b, c\}$, a proper subset of a larger codomain). A function on an infinite domain typically needs a rule rather than a listed graph: the squaring function on $\mathbb{Z}$ has graph $\{(x, x^2) : x \in \mathbb{Z}\}$.

2.2 Functions in computer science

2.2.1 Functions from analysis

Note. In software development, *analysis* produces a precise specification of *what* must be done – often expressible as a mathematical function. *Design* then produces an *algorithm* for computing it (*how*); *implementation* turns that algorithm into code.

2.2.2 Functions in mathematics and programming

Note. Two senses of "function" coexist: the *mathematical* sense (domain, codomain, rule – no algorithm implied), and the *programming* sense (a named block of code that computes something). Unless flagged otherwise ("Python function", "algorithmic function"), we use the mathematical sense throughout.

2.3 Notation

Definition 2.13 (Function declaration)**.**

$$f : \text{domain} \to \text{codomain}, \qquad f(x) = \text{expression in } x.$$

Equivalently, the rule may be given with a *mapping arrow*: $x \mapsto$ expression in x (note: $\mapsto$ for the rule, $\to$ for domain/codomain – do not conflate them).

Example 2.14. $f : \mathbb{R} \to \mathbb{R},\ f(x) = x^2$, equivalently $f : \mathbb{R} \to \mathbb{R},\ x \mapsto x^2$. The codomain need not be the tightest possible: $\mathbb{R}_0^+$ (the image) would also be valid, but a codomain like $\mathbb{R} \cup \{0, -\sqrt{2}, -42\}$, while technically valid, is needlessly cluttered.

2.4 Some special functions

Definition 2.15 (Empty function)**.**

$$\emptyset : \emptyset \to \emptyset$$

Empty domain, empty codomain, vacuous rule.

Definition 2.16 (Identity function)**.** For a set A:

$$i_A : A \to A, \qquad i_A(x) = x.$$

Definition 2.17 (Indicator (characteristic) function)**.** For $A \subseteq D$:

$$\chi_A : D \to \{0,1\}, \qquad \chi_A(x) = \begin{cases} 1, & x \in A; \\ 0, & x \notin A. \end{cases}$$

Depends on D, not just A: different domains give different indicator functions.

Example 2.18.

$$\chi_{A \cup B}(x) = \max\{\chi_A(x), \chi_B(x)\} \quad \text{for all } x.$$

Definition 2.19 (Constant function)**.** For a domain D and object a:

$$c_a : D \to \{a\}, \qquad c_a(x) = a.$$

Definition 2.20 (Floor and ceiling, Rosen)**.** For $x \in \mathbb{R}$:

$$\lfloor x \rfloor = \max\{n \in \mathbb{Z} : n \le x\}, \qquad \lceil x \rceil = \min\{n \in \mathbb{Z} : n \ge x\}.$$

Definition 2.21 (Partial function)**.** A *partial function* f from A to B assigns *at most one* element of B to each element of A; the subset of A where f is actually defined is $\mathrm{dom}(f)$ (possibly a proper subset of A).

Note (Contrast with CN's §2.1.1). CN's definition of a function already *is* what Rosen calls a *total* function: $\mathrm{dom}(f)$ equals the declared domain exactly, with no gaps. Rosen's "partial function from A to B" allows $\mathrm{dom}(f) \subsetneq A$ – a total function on the smaller set $\mathrm{dom}(f) \subseteq A$.

Example 2.22. $f : \mathbb{R} \to \mathbb{R}$, $f(x) = 1/x$ is a partial function on $\mathbb{R}$ (undefined at 0); as a *total* function it is $f : \mathbb{R} \setminus \{0\} \to \mathbb{R}$ – same rule, different declared domain (CN's §2.1.1 view of the domain as an exact promise).

Note (Where partiality actually shows up – harder). Partial functions are essential in computability theory: a program P computing a function of its input may not halt on every input. The function $f_P(x) =$ "output of P on input x, if P halts" is a genuine partial function – $\mathrm{dom}(f_P)$ is exactly the set of inputs on which P halts, and this set can itself be uncomputable (the Halting Problem: no algorithm decides, for arbitrary P, x, whether $x \in \mathrm{dom}(f_P)$).

2.5 Functions with multiple arguments and values

Definition 2.23 (Multiple arguments via Cartesian product)**.** A function of two arguments $f(x, y)$, $x \in X, y \in Y$, value in C, is formally $f : X \times Y \to C$ – a function of *one* argument, the ordered pair (x, y), with $f(x, y)$ shorthand for $f((x, y))$. Extends to any number of arguments via $X_1 \times \cdots \times X_n$.

Note (Notation styles). $f(x, y)$ is *prefix* notation (name before arguments). *Infix* (name between, e.g. $x + y$) and *postfix* (name after) also occur.

Definition 2.24 (Multiple (tuple-valued) outputs)**.** A function returning a pair is $f : X \to Y \times Z$; its value is a single element $(y, z) \in Y \times Z$, viewable as "two outputs." Extends to any number of returned values.

2.6 Restrictions

Definition 2.25 (Restriction)**.** For $f : A \to B$ and $X \subseteq A$, the *restriction* of f to X, written $f|_X$ (or $f \restriction_X$), is

$$f|_X : X \to B, \qquad f|_X(x) = f(x) \text{ for all } x \in X.$$

Note (Why restrict?). A smaller domain means: a smaller stored representation (fewer ordered pairs to list); a smaller "promise" to keep (less testing if implemented); sometimes simpler analysis; and sometimes *stronger properties* unavailable to the original function.

Example 2.26. Let $f : \mathbb{R} \to \mathbb{R},\ f(x) = x^2$. Its restriction $f|_{\mathbb{R}_0^+} : \mathbb{R}_0^+ \to \mathbb{R}$ is strictly increasing throughout its domain (unlike f, which decreases on $\mathbb{R}_0^-$), and every y in its image has a *unique* preimage $\sqrt{y}$ – whereas under f itself, each nonzero y in the image has two preimages, $\pm\sqrt{y}$. (This makes $f|_{\mathbb{R}_0^+}$ invertible, unlike f – see §2.8.)

2.7 Injections, surjections, bijections

Definition 2.27 (Injective / one-to-one)**.** $f : A \to B$ is *injective* (an *injection*, *one-to-one*) if distinct arguments always give distinct values:

$$\forall x_1, x_2 \in A, \quad x_1 \neq x_2 \implies f(x_1) \neq f(x_2).$$

Note (Rosen's contrapositive phrasing). Equivalently (contrapositive):

$$\forall x_1, x_2 \in A, \quad f(x_1) = f(x_2) \implies x_1 = x_2.$$

This is the form most useful in proofs: assume $f(x_1) = f(x_2)$ and derive $x_1 = x_2$.

Note (Injections preserve information). Every y in the image of an injection has a *unique* preimage, so knowing y determines x uniquely. A non-injective function is *lossy* (some inputs become indistinguishable from their output); an injective one is *lossless*.

Definition 2.28 (Surjective / onto)**.** $f : A \to B$ is *surjective* (a *surjection*, *onto*) if

$$\forall y \in B,\ \exists x \in A \text{ such that } f(x) = y,$$

i.e. the image equals the codomain.

Definition 2.29 (Bijective)**.** f is *bijective* (a *bijection*, *one-to-one correspondence*) if it is both injective and surjective.

Note (Permutation). A bijection $f : A \to A$ (domain = codomain) is also called a *permutation* of A – usually reserved for finite A.

Theorem 2.30 (Finite equal-size domain/codomain)**.** *Let $f : A \to B$ with A, B finite and $|A| = |B|$. Then f is injective $\iff$ f is surjective $\iff$ f is bijective.*

Proof by induction on $n = |A| = |B|$. We show: f injective $\implies$ f bijective. (The surjective case is symmetric, swapping the roles of A and B throughout.)

Base case ($n = 0$): $A = B = \emptyset$. The unique function $\emptyset \to \emptyset$ is vacuously injective and vacuously surjective, hence bijective.

Inductive step: Suppose the claim holds for all injective functions between finite sets of size $n - 1$. Let $f : A \to B$ be injective with $|A| = |B| = n \geq 1$.

Pick any $a \in A$ and let $b = f(a)$. Set $A' = A \setminus \{a\}$, $B' = B \setminus \{b\}$, and let $f' = f|_{A'}$ be the restriction of f to A'.

- f' *maps into* B': for $x \in A'$ we have $x \neq a$, so by injectivity of f, $f(x) \neq f(a) = b$; hence $f(x) \in B \setminus \{b\} = B'$.
- f' *is injective*: it is a restriction of the injective function f.
- $|A'| = |B'| = n - 1$.

By the inductive hypothesis, $f' : A' \to B'$ is bijective, in particular surjective onto B'.

Now take any $y \in B$. Either $y = b$, in which case $y = f(a)$; or $y \in B'$, in which case surjectivity of f' gives some $x \in A'$ with $f(x) = f'(x) = y$. Either way, y has a preimage under f. So f is surjective onto B, and being injective by hypothesis, f is bijective.

By induction, the claim holds for all $n \geq 0$. ■

Note. This theorem requires *finite* A, B of equal size – it fails for infinite sets (e.g. $f : \mathbb{N} \to \mathbb{N}$, $f(x) = 2x$ is injective but not surjective, despite $|\mathbb{N}| = |\mathbb{N}|$ in the infinite-cardinality sense).

Note (Proof strategies for injectivity and surjectivity). To prove $f : A \to B$ is **injective**: let $x_1, x_2 \in A$ be arbitrary with $f(x_1) = f(x_2)$, and show $x_1 = x_2$ follows (direct proof of the contrapositive form).

To prove f is **not injective**: exhibit specific $x_1 \neq x_2 \in A$ with $f(x_1) = f(x_2)$ (one counterexample suffices).

To prove f is **surjective**: let $y \in B$ be arbitrary, and *construct* (or show the existence of) some $x \in A$ with $f(x) = y$ – typically by solving $f(x) = y$ for x in terms of y and checking $x \in A$.

To prove f is **not surjective**: exhibit a specific $y \in B$ for which no $x \in A$ satisfies $f(x) = y$ (one counterexample suffices).

2.8 Inverse functions

Note (Why uniqueness can fail). Let $f : A \to B$. For a given $y \in B$, trying to go "backwards" can fail in two distinct ways:

- *Non-uniqueness*: there exist $x_1 \neq x_2 \in A$ with $f(x_1) = f(x_2) = y$.
- *Non-existence*: y lies in the codomain but not the image, so no $x \in A$ satisfies $f(x) = y$.

Both failures are ruled out exactly when f is, respectively, injective and surjective.

Definition 2.31 (Inverse function)**.** If $f : A \to B$ is such that every $y \in B$ has a *unique* $x \in A$ with $f(x) = y$, the *inverse function* $f^{-1} : B \to A$ is defined by

$$f^{-1}(y) = \text{the unique } x \in A \text{ such that } f(x) = y.$$

Equivalently, as a set of ordered pairs (the graph of f, reversed):

$$f^{-1} = \{(y, x) : (x, y) \in f\} = \{(f(x), x) : x \in A\}.$$

Theorem 2.32. *$f : A \to B$ has an inverse function if and only if f is a bijection.*

Proof. ($\Rightarrow$) If f^{-1} exists, every $y \in B$ has *some* preimage (so f is surjective) and *at most one* preimage (so f is injective, since two preimages of the same y would break the "unique x" requirement in the definition of f^{-1}). Hence f is a bijection.

($\Leftarrow$) If f is a bijection, surjectivity guarantees every $y \in B$ has *at least one* preimage, and injectivity guarantees *at most one*; together, exactly one. This unique preimage is exactly what $f^{-1}(y)$ requires, so f^{-1} is well-defined. ■

Note (When invertibility fails). Neither the `Employer` function (not injective: two computers both mapped to Harvard College Observatory) nor the squaring function $\mathbb{R} \to \mathbb{R}$ (not injective: $2^2 = (-2)^2$; also not surjective onto all of $\mathbb{R}$) is invertible.

Theorem 2.33 (f^{-1} undoes f, and vice versa)**.** *If $f : A \to B$ is a bijection, then for all $x \in A$ and all $y \in B$:*

$$f^{-1}(f(x)) = x, \qquad f(f^{-1}(y)) = y.$$

Proof. Let $x \in A$ and $y = f(x)$. By definition, $f^{-1}(y)$ is the unique element of A mapping to y under f – and x is such an element, so $f^{-1}(f(x)) = f^{-1}(y) = x$. The second identity is symmetric: apply the same argument with the roles of f, f^{-1} swapped, using that f^{-1} is itself a bijection (below). ■

Theorem 2.34. *If f is a bijection, so is f^{-1}, and $(f^{-1})^{-1} = f$.*

Proof. $f^{-1} : B \to A$ is injective and surjective by the symmetry of the defining property (unique x for each $y \Leftrightarrow$ unique y for each x, since f itself is a bijection), hence f^{-1} is a bijection and has its own inverse. That inverse must send each x back to the y with $f^{-1}(y) = x$, i.e. $y = f(x)$ – exactly f. So $(f^{-1})^{-1} = f$. ■

Definition 2.35 (Preimage of a set, Rosen)**.** For $f : A \to B$ (*not* necessarily a bijection) and $S \subseteq B$, the *preimage* (or *inverse image*) of S under f is

$$f^{-1}(S) = \{a \in A : f(a) \in S\}.$$

Note (Overloaded notation – read carefully). $f^{-1}(S)$ (preimage of a *set*, defined for *any* function f) and $f^{-1}(y)$ (the *inverse function* applied to a point, defined *only when f is a bijection*) use the same symbol f^{-1} but are different concepts. When f is a bijection, $f^{-1}(\{y\}) = \{f^{-1}(y)\}$ – the set-level and point-level notions agree – but $f^{-1}(S)$ makes sense even when no bijection (or even no inverse function) exists.

Example 2.36. For $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$ (not a bijection, so f^{-1} as a function does not exist), the preimage of $S = [4, 9]$ is

$$f^{-1}([4, 9]) = \{x \in \mathbb{R} : x^2 \in [4, 9]\} = [-3, -2] \cup [2, 3].$$

2.9 Composition

Definition 2.37 (Composition)**.** For $f : A \to B$ and $g : B \to C$, the *composition* $g \circ f : A \to C$ is

$$(g \circ f)(x) = g(f(x)).$$

Note (Order of writing vs. order of application). $g \circ f$ is read right-to-left in application (f first, then g), matching the order inside $g(f(x))$ – but this is the *reverse* of our usual left-to-right reading order. Read carefully.

Note (Compatibility requirement). $g \circ f$ is defined only when $\text{codomain}(f) = \text{domain}(g)$: whatever f guarantees to produce must be exactly what g is guaranteed to handle. If this fails, the composition is simply undefined – not partially defined, not an error to patch around.

Theorem 2.38 (Not commutative, but associative)**.** *In general $g \circ f \neq f \circ g$ (indeed one side may not even be defined). But for $f : A \to B$, $g : B \to C$, $h : C \to D$:*

$$h \circ (g \circ f) = (h \circ g) \circ f,$$

so $h \circ g \circ f$ is unambiguous without parentheses.

Proof. Both sides are functions $A \to D$. For any $x \in A$:

$$\big(h \circ (g \circ f)\big)(x) = h\big((g \circ f)(x)\big) = h(g(f(x))) = (h \circ g)(f(x)) = \big((h \circ g) \circ f\big)(x).$$

Equal on every input, so the functions are equal. ■

Theorem 2.39 (Role of the identity)**.** *For $f : A \to A$ a bijection: $f \circ f^{-1} = i_A$ and $f^{-1} \circ f = i_A$. For any $g : C \to D$: $g \circ i_C = g$ and $i_D \circ g = g$.*

Definition 2.40 (Iterated composition)**.** For $f : A \to A$ and $n \in \mathbb{Z}^+$:

$$f^{(n)} = \underbrace{f \circ f \circ \cdots \circ f}_{n \text{ copies}}.$$

Theorem 2.41 (Composition preserves injectivity/surjectivity, Rosen)**.** *Let $f : A \to B$, $g : B \to C$.*

(i) If f, g are both injective, so is $g \circ f$.

(ii) If f, g are both surjective, so is $g \circ f$.

(iii) If f, g are both bijective, so is $g \circ f$.

Proof. (i) Suppose $(g \circ f)(x_1) = (g \circ f)(x_2)$, i.e. $g(f(x_1)) = g(f(x_2))$. Since g is injective, $f(x_1) = f(x_2)$; since f is injective, $x_1 = x_2$.

(ii) Let $z \in C$. Since g is surjective, $z = g(y)$ for some $y \in B$; since f is surjective, $y = f(x)$ for some $x \in A$. Then $(g \circ f)(x) = g(f(x)) = g(y) = z$.

(iii) Immediate from (i) and (ii). ■

2.10 Cryptosystems

Definition 2.42 (Cryptosystem)**.** A *cryptosystem* consists of a message space M, cyphertext space C, keyspace K, an *encryption function* $e : M \times K \to C$, and a *decryption function* $d : C \times K \to M$, such that for every key $k \in K$, the single-argument functions

$$e_k : M \to C,\ e_k(m) = e(m, k), \qquad d_k : C \to M,\ d_k(c) = d(c, k)$$

satisfy:

(i) e_k, d_k are bijective (onto their images), and

(ii) $d_k = e_k^{-1}$.

Note. Condition (ii) is the essential one: it forces $d_k(e_k(m)) = m$ for all $m \in M$ – decryption with k genuinely undoes encryption with k.

Example 2.43 (Shift cipher, Rosen §4.6)**.** Take $M = C = \mathbb{Z}_{26} = \{0, 1, \ldots, 25\}$ (letters as residues mod 26) and $K = \mathbb{Z}_{26}$. Define

$$e_k(p) = (p + k) \bmod 26, \qquad d_k(c) = (c - k) \bmod 26.$$

Each e_k is a bijection on $\mathbb{Z}_{26}$ with $d_k = e_k^{-1}$, so (M, C, K, e, d) is a cryptosystem. ($k = 3$ is the classical Caesar cipher.)

Definition 2.44 (Composition of cryptosystems)**.** Given $\mathcal{C} = (M, M, K, e, d)$ and $\mathcal{C}' = (M, M, K', e', d')$ (same message space), the *keyed composition* of encryption functions is

$$e' \bullet e : M \times (K \times K') \to M, \qquad (e' \bullet e)(m, (k, k')) = e'(e(m, k), k'),$$

and similarly $d \bullet d'$ for decryption. The composed cryptosystem is $\mathcal{C}' \circ \mathcal{C} = (M, M, K \times K', e' \bullet e, d \bullet d')$. For each key pair (k, k'), the resulting single-argument maps are exactly the ordinary function compositions:

$$e_{(k,k')} = e'_{k'} \circ e_k, \qquad d_{(k,k')} = d_k \circ d'_{k'}.$$

Theorem 2.45. *The composition of two cryptosystems is a cryptosystem.*

Proof. Fix $(k, k') \in K \times K'$. Since $\mathcal{C}, \mathcal{C}'$ are cryptosystems, $e_k, e'_{k'}, d_k, d'_{k'}$ are all bijections. By the composition-preserves-bijectivity theorem (§2.9), $e_{(k,k')} = e'_{k'} \circ e_k$ is a bijection. It remains to check $d_{(k,k')} = (e_{(k,k')})^{-1}$: since $(e'_{k'})^{-1} = d'_{k'}$ and $(e_k)^{-1} = d_k$, and the inverse of a composition reverses order, $(e'_{k'} \circ e_k)^{-1} = e_k^{-1} \circ (e'_{k'})^{-1} = d_k \circ d'_{k'} = d_{(k,k')}$, as required. ∎

Note (Security is not guaranteed by composition). A good cryptosystem needs e_k, d_k easy to compute *with* k, but e_k hard to invert *without* k. Composing two hard-to-invert systems is often, but not always, harder still to break. Counterexample: composing two shift ciphers with keys k, k' gives $e'_{k'}(e_k(p)) = (p + k + k') \bmod 26 = e_{k+k'}(p)$ – just another shift cipher, no stronger than a single shift, since the keyspace effectively collapses.

2.11 Loose composition

Definition 2.46 (Loose composition)**.** For $f : A \to B$ and $g : C \to D$ (no requirement that $B = C$), the *loose composition* $g \mathbin{\hat{\circ}} f$ is

$$g \mathbin{\hat{\circ}} f : \{x \in A : f(x) \in C\} \to D, \qquad (g \mathbin{\hat{\circ}} f)(x) = g(f(x)).$$

Note (Always defined, but harder to use). Unlike ordinary ("tight") composition, loose composition is defined for *any* f, g – if $B \cap C = \emptyset$, it degenerates to the empty function. The cost: determining its domain requires inspecting the *rule* of f (which inputs land in C), not just f's stated codomain – so it is harder to reason about mechanically. This course uses tight composition throughout.

2.12 Counting functions

Theorem 2.47 (Counting all functions)**.** *If $|A| = m$, $|B| = n$, the number of functions $f : A \to B$ is n^m.*

Proof. Each of the m elements of A independently has n possible images in B; by the product rule, $\underbrace{n \times n \times \cdots \times n}_{m} = n^m$. ∎

Theorem 2.48 (Counting injections)**.** *If $|A| = m$, $|B| = n$, the number of injections $f : A \to B$ is*

$$n(n-1)(n-2)\cdots(n-m+1) = \frac{n!}{(n-m)!} \quad (\textit{Rosen: } P(n, m)).$$

Proof. Enumerate $A = \{a_1, \ldots, a_m\}$. a_1 has n choices in B; a_2 must avoid $f(a_1)$, so $n - 1$ choices; …; a_m must avoid the $m - 1$ already-used values, so $n - m + 1$ choices. These choices are made in sequence, so by the product rule the total is $n(n-1)\cdots(n-m+1)$. ∎

Note. If $m > n$, the product includes a factor of 0, correctly giving 0 injections – consistent with the pigeonhole principle.

Theorem 2.49 (Counting bijections / permutations)**.** *If $|A| = |B| = n$, the number of bijections $f : A \to B$ is $n!$.*

Proof. By §2.7, for finite sets of equal size, bijective $\iff$ injective. So this is the injection count with $m = n$: $n(n-1)\cdots(n-n+1) = n!$. ■

Note. In particular, the number of permutations of an n-element set (bijections $A \to A$) is $n!$.

2.13 Binary relations

Definition 2.50 (Binary relation)**.** A *binary relation* from A to B is a subset $R \subseteq A \times B$. We write xRy, $R(x, y)$, or $(x, y) \in R$ interchangeably. A relation *on* A satisfies $R \subseteq A \times A$.

Note (Relations as "relaxed functions")*.* A function $f : A \to B$ is exactly a relation $R \subseteq A \times B$ satisfying: for every $a \in A$, there is a *unique* $b \in B$ with $(a, b) \in R$. Dropping the uniqueness (and even the existence) requirement gives a general binary relation – so every function is a relation, but not conversely.

Definition 2.51 (Domain, range/codomain of a relation)**.** For $R \subseteq A \times B$:

$$\{x \in A : (x, y) \in R \text{ for some } y \in B\} \quad \text{(elements actually related to something)},$$

$$\{y \in B : (x, y) \in R \text{ for some } x \in A\} \quad \text{(elements actually related to)}.$$

Definition 2.52 ($R(x)$, $R^{-1}(y)$ – CN notation)**.** For $x \in A$: $R(x) = \{y \in B : xRy\}$ (a *set*, possibly empty – not a single value, unlike function notation). For $y \in B$: $R^{-1}(y) = \{x \in A : xRy\}$.

Definition 2.53 (Inverse relation)**.** For $R \subseteq A \times B$, the *inverse relation* $R^{-1} \subseteq B \times A$ is

$$yR^{-1}x \iff xRy.$$

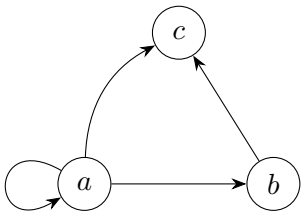
Note. When R happens to be a function, this agrees with the inverse function of §2.8 – *but only when that function is a bijection.* For a general function f, f^{-1} as a *relation* always exists (just reverse the pairs); it is only a *function* in its own right when f is bijective.

Definition 2.54 (Zero-one matrix of a relation)**.** For finite $A = \{a_1, \ldots, a_m\}$, $B = \{b_1, \ldots, b_n\}$ and $R \subseteq A \times B$: the matrix M_R is $m \times n$ with

$$(M_R)_{ij} = 1 \iff (a_i, b_j) \in R, \qquad (M_R)_{ij} = 0 \text{ otherwise.}$$

Note (Properties as matrix conditions, R on A)*.* Reflexive: diagonal all 1s. Symmetric: $M_R = M_R^t$. Antisymmetric: $(M_R)_{ij} = 1$ and $i \neq j \implies (M_R)_{ji} = 0$.

Definition 2.55 (Directed graph of a relation)**.** For R on A: vertices $= A$; a directed edge $a \to b$ exactly when $(a, b) \in R$.



Directed graph of $R = \{(a,a),(a,b),(a,c),(b,c)\}$ – the loop at a shows reflexivity there

Note (Properties as graph conditions). Reflexive: a loop at every vertex. Symmetric: every edge $a \to b$ has a matching edge $b \to a$. Transitive: whenever $a \to b \to c$, also $a \to c$ directly.

Note (Boolean matrix product computes composition – harder, ties to §2.6). If M_R, M_S are zero-one matrices of $R \subseteq A \times B$, $S \subseteq B \times C$, then $M_R \odot M_S = M_{S \circ R}$ (Boolean product, §2.6) – so relation composition (§2.15) reduces to matrix multiplication with $\vee, \wedge$ in place of $+, \times$.

2.14 Properties of binary relations

Definition 2.56 (Reflexive)**.** R on A is *reflexive* iff

$$\forall a \in A, \quad (a,a) \in R.$$

Definition 2.57 (Irreflexive)**.** R on A is *irreflexive* iff

$$\forall a \in A, \quad (a,a) \notin R.$$

Definition 2.58 (Symmetric)**.** R on A is *symmetric* iff

$$\forall x, y \in A, \quad (x,y) \in R \implies (y,x) \in R.$$

Definition 2.59 (Antisymmetric)**.** R on A is *antisymmetric* iff

$$\forall x, y \in A, \quad \big[(x,y) \in R \wedge (y,x) \in R\big] \implies x = y.$$

Definition 2.60 (Asymmetric)**.** R on A is *asymmetric* iff

$$\forall x, y \in A, \quad (x,y) \in R \implies (y,x) \notin R.$$

Definition 2.61 (Transitive)**.** R on A is *transitive* iff

$$\forall x, y, z \in A, \quad \big[(x,y) \in R \wedge (y,z) \in R\big] \implies (x,z) \in R.$$

Example 2.62. On $\mathbb{Z}$: $=$, $\leq$, $\geq$ are reflexive; $<$, $>$ are irreflexive. $=$ is symmetric; $<, \leq, >, \geq$ are not. $=, \leq, \geq$ are antisymmetric. $<$, $>$ are asymmetric. $=, <, \leq, >, \geq$ are all transitive; $\neq$ is not.

Theorem 2.63.

$$R \text{ asymmetric} \implies R \text{ antisymmetric} \wedge R \text{ irreflexive}.$$

The converse is false.

Proof. Suppose R is asymmetric. If $(x,y) \in R$ and $(y,x) \in R$ simultaneously held for $x \neq y$, this would contradict asymmetry directly, so antisymmetry holds. If $(a,a) \in R$ for some a, then taking $x = y = a$ in the asymmetry condition gives $(a,a) \in R \implies (a,a) \notin R$, a contradiction; so R is irreflexive.

For the converse: $\leq$ on $\mathbb{Z}$ is antisymmetric (if $x \leq y$ and $y \leq x$ then $x = y$) but not irreflexive ($a \leq a$ holds for all a), hence not asymmetric. ∎

Example 2.64 (Checking all properties on a concrete relation)**.** Let $A = \{a,b,c,d\}$ and

$$R = \{(a,a),(a,c),(a,d),(b,a),(b,b),(b,c),(b,d),(c,b),(c,c),(d,b),(d,d)\}.$$

- *Reflexive*: yes, since $(a,a),(b,b),(c,c),(d,d) \in R$.
- *Symmetric*: no, since $(a,c) \in R$ but $(c,a) \notin R$.
- *Antisymmetric*: no, since $(b,c),(c,b) \in R$ with $b \neq c$.
- *Transitive*: no, since $(a,c),(c,b) \in R$ but $(a,b) \notin R$.

Theorem 2.65 (Properties under union and intersection)**.** *Let R, S be relations on A.*

$$\begin{aligned} R,S \text{ reflexive} &\implies R \cup S,\ R \cap S \text{ reflexive}, \\ R,S \text{ symmetric} &\implies R \cup S,\ R \cap S \text{ symmetric}, \\ R,S \text{ transitive} &\implies R \cap S \text{ transitive (but not necessarily } R \cup S). \end{aligned}$$

Proof. Reflexive: R, S reflexive $\implies \forall a \in A,\ (a,a) \in R \wedge (a,a) \in S \implies (a,a) \in R \cup S$ and $(a,a) \in R \cap S$.

Symmetric: If $(x,y) \in R \cup S$, then $(x,y) \in R$ or $(x,y) \in S$; symmetry of the relation containing it gives (y,x) in that same relation, hence $(y,x) \in R \cup S$. If $(x,y) \in R \cap S$, then (x,y) is in both, so by symmetry (y,x) is in both, hence $(y,x) \in R \cap S$.

Transitive ($\cap$ only): If $(x,y),(y,z) \in R \cap S$, both pairs lie in R, so transitivity of R gives $(x,z) \in R$; both pairs also lie in S, so $(x,z) \in S$; hence $(x,z) \in R \cap S$. Union fails in general: a chain $x \to y$ witnessed only by R and $y \to z$ witnessed only by S need not put (x,z) in R or in S individually. ■

2.15 Combining binary relations

Definition 2.66 (Union and intersection of relations)**.** For $R, S \subseteq A \times B$:

$$R \cup S = \{(x,y) : xRy \text{ or } xSy\}, \qquad R \cap S = \{(x,y) : xRy \text{ and } xSy\}.$$

Definition 2.67 (Composition of relations)**.** For $R \subseteq A \times B$ and $S \subseteq B \times C$, the *composition* $S \circ R \subseteq A \times C$ is

$$S \circ R = \{(x,z) : \exists y \in B \text{ such that } xRy \text{ and } ySz\}.$$

Note. When R, S happen to be functions, this is exactly function composition (§2.9). Relation composition generalizes it: no uniqueness of the intermediate y is required, and there may be several, or none.

Theorem 2.68 (Composing R with its inverse)**.** *For $R \subseteq A \times B$:*

$$(x,z) \in R^{-1} \circ R \iff \exists y \in B \text{ such that } xRy \text{ and } zRy.$$

Proof.

$$(x,z) \in R^{-1} \circ R \iff \exists y,\ xRy \wedge yR^{-1}z \iff \exists y,\ xRy \wedge zRy$$

using the defining property $yR^{-1}z \iff zRy$. ■

Note. So $R^{-1} \circ R$ relates $x, z \in A$ exactly when they share a common R-image in B; symmetrically, $R \circ R^{-1}$ relates elements of B that share a common preimage in A.

Definition 2.69 (Iterated composition of a relation with itself)**.** For R on A and $n \in \mathbb{Z}^+$:

$$R^{(n)} = \underbrace{R \circ R \circ \cdots \circ R}_{n \text{ copies}}.$$

Example 2.70 ("Six degrees of separation")**.** For the relation knows on the set of all people, knows$^{(2)}$ relates two people with a mutual acquaintance; knows$^{(6)}$ is (under this popular hypothesis) the complete relation on all people.

Definition 2.71 (Transitive closure)**.** The *transitive closure* of R on A is the unique relation R^+ on A such that, for all $x, y, z \in A$:

(i) $xRy \implies xR^+y$;

(ii) $xR^+y \wedge yRz \implies xR^+z$;

(iii) $xRy \wedge yR^+z \implies xR^+z$.

Note. Condition (i) is essential (it seeds the closure); either (ii) or (iii) alone suffices together with (i) – both are not simultaneously required.

Theorem 2.72.

$$R^+ = \bigcup_{i=1}^{\infty} R^{(i)}.$$

R^+ is always transitive.

Proof sketch. Each $R^{(i)}$ is contained in R^+: $R^{(1)} = R \subseteq R^+$ by (i), and if $R^{(i)} \subseteq R^+$, then $(x, z) \in R^{(i+1)}$ means $\exists y$, $(x, y) \in R^{(i)} \subseteq R^+$ and yRz, so $(x, z) \in R^+$ by (ii); induction gives $R^{(i)} \subseteq R^+$ for all i, hence $\bigcup_i R^{(i)} \subseteq R^+$. Conversely, $\bigcup_i R^{(i)}$ satisfies (i)-(iii) itself, and R^+ is the *unique* relation satisfying them, so equality follows. Transitivity of R^+ is then immediate: any finite chain of R^+-steps is absorbed into some $R^{(i)} \subseteq R^+$. ■

Definition 2.73 (Closure, general)**.** The *closure* of R under a property P is the smallest relation $R' \supseteq R$ with property P (if it exists).

Definition 2.74 (Reflexive closure)**.**

$$r(R) = R \cup \Delta_A, \qquad \Delta_A = \{(a, a) : a \in A\} \text{ (the diagonal relation).}$$

Definition 2.75 (Symmetric closure)**.**

$$s(R) = R \cup R^{-1}.$$

Note. Transitive closure R^+ was already defined above: $R^+ = \bigcup_{i=1}^{\infty} R^{(i)}$.

Theorem 2.76 (Warshall's algorithm – harder, computes R^+ efficiently)**.** *Let $A = \{a_1, \ldots, a_n\}$, $M_0 = M_R$. Define M_k from M_{k-1} by:*

$$(M_k)_{ij} = (M_{k-1})_{ij} \ \vee \ \big((M_{k-1})_{ik} \wedge (M_{k-1})_{kj}\big),$$

i.e. allow paths that route through a_k as an intermediate vertex. Then $M_n = M_{R^+}$.

Proof idea. $(M_k)_{ij} = 1$ iff there is a path $a_i \to a_j$ using only intermediate vertices from $\{a_1, \ldots, a_k\}$. Induction on k: base case $k = 0$ is exactly R itself; the update rule captures "either a path avoiding a_k already existed, or one exists routing through a_k." At $k = n$, $(M_n)_{ij} = 1$ iff *any* path $a_i \to a_j$ exists – exactly R^+. ■

Note (Efficiency). Computing R^+ via $\bigcup_i R^{(i)}$ directly can cost $O(n^4)$; Warshall's algorithm computes the same R^+ in $O(n^3)$.

2.16 Equivalence relations

Definition 2.77 (Equivalence relation)**.** R on A is an *equivalence relation* iff R is reflexive, symmetric, and transitive.

Note (Why not antisymmetric too?). Equality is reflexive, symmetric, *and* antisymmetric. But requiring antisymmetry of a general equivalence relation would force $xRy \wedge yRx \implies x = y$ – collapsing "equivalent" into "identical," which defeats the purpose of a looser notion of sameness.

Example 2.78.
- Congruence modulo m: $x \equiv y \pmod{m} \iff m \mid (x - y)$.
- For any $f : A \to B$: $x \sim_f y \iff f(x) = f(y)$ defines an equivalence relation on A.
- The transitive closure of any reflexive, symmetric relation is an equivalence relation.
- For any relation R: $R \circ R^{-1}$ and $R^{-1} \circ R$ are equivalence relations.

Definition 2.79 (Equivalence class)**.** For equivalence relation R on A, an *equivalence class* is a nonempty $X \subseteq A$ such that:

$$\text{(i) } \forall x, y \in X,\ xRy; \qquad \text{(ii) } \nexists x \in X, z \notin X \text{ such that } xRz.$$

Equivalently, X is a *maximal* subset of A on which R holds between every pair.

Note. For $x \in A$, the equivalence class containing x is $[x]_R = \{y \in A : xRy\}$.

Theorem 2.80. *The equivalence classes of an equivalence relation R on A form a partition of A.*

Proof. **(a) Every $x \in A$ lies in some equivalence class.** Let $X = \{y \in A : xRy\} = [x]_R$.

- *X satisfies (i)*: for $y, z \in X$, xRy and xRz; by symmetry yRx, and with xRz and transitivity, yRz.
- *X satisfies (ii)*: suppose $v \in X$, $w \notin X$, vRw. Since $v \in X$: xRv; combined with vRw and transitivity, xRw, so $w \in X$ – contradiction. So no such v, w exist.
- $x \in X$ by reflexivity (xRx).

So X is an equivalence class containing x.

(b) Distinct equivalence classes are disjoint. Suppose X, Y are equivalence classes with $X \cap Y \neq \emptyset$; let $x \in X \cap Y$. We show $X = Y$ by showing $Y \setminus X = \emptyset$ and $X \setminus Y = \emptyset$.

If $y \in Y \setminus X$: since X is an equivalence class and $y \notin X$, $(x, y) \notin R$ (property (ii) for X, using $x \in X$). But $x, y \in Y$, and Y is an equivalence class, so property (i) forces $(x, y) \in R$ – contradiction. So $Y \setminus X = \emptyset$; symmetrically $X \setminus Y = \emptyset$. Hence $X = Y$.

So every element of A lies in exactly one equivalence class – the classes are nonempty (by definition), pairwise disjoint (part (b)), and cover A (part (a)): a partition. ∎

Definition 2.81 (Partial order)**.** R on A is a *partial order* iff reflexive, antisymmetric, and transitive. (A, R) is then a *partially ordered set* (*poset*), often written $(A, \preceq)$.

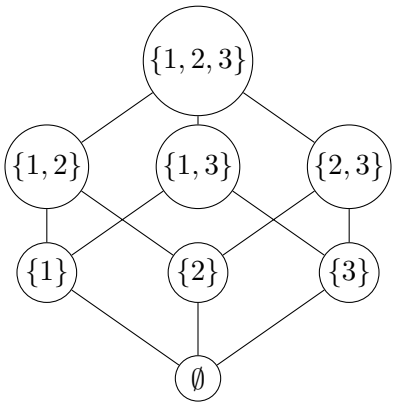
Example 2.82. $(\mathbb{Z}, \leq)$; $(\mathcal{P}(S), \subseteq)$ for any set S (§1.6); $(\mathbb{Z}^+, |)$ where $a \mid b$ means a divides b.

Definition 2.83 (Comparable, total order, chain)**.** $a, b \in A$ are *comparable* if $a \preceq b$ or $b \preceq a$. If *every* pair is comparable, $\preceq$ is a *total* (or *linear*) order and A is a *chain*.

Note. $(\mathbb{Z}, \leq)$ is a total order; $(\mathcal{P}(S), \subseteq)$ is generally *not* (e.g. $\{1\}$ and $\{2\}$ are incomparable in $\mathcal{P}(\{1,2\})$) – exactly why §1.7's maximal/maximum distinction was necessary.

Definition 2.84 (Hasse diagram)**.** A drawing of a finite poset: vertices $= A$, an edge drawn upward from a to b exactly when $a \prec b$ and there is no c with $a \prec c \prec b$ (a *covering* relation) – omitting all edges implied by reflexivity/transitivity.

Example 2.85. For $\mathcal{P}(\{1,2,3\})$ under $\subseteq$: $\emptyset$ at the bottom, the three singletons above it, the three pairs above those, $\{1,2,3\}$ at the top.



Hasse diagram of $(\mathcal{P}(\{1,2,3\}), \subseteq)$

Definition 2.86 (Upper/lower bound, LUB, GLB – harder)**.** For $S \subseteq A$: $u \in A$ is an *upper bound* of S if $s \preceq u$ for all $s \in S$; a *least upper bound* (LUB, or *join* $\vee$) is an upper bound $\preceq$ every other upper bound. Dually for *lower bound* and *greatest lower bound* (GLB, *meet* $\wedge$).

Definition 2.87 (Lattice)**.** A poset in which every pair a, b has both a LUB and a GLB.

Example 2.88. $(\mathcal{P}(S), \subseteq)$ is a lattice: $A \vee B = A \cup B$, $A \wedge B = A \cap B$ (§1.12).

Note. $(\mathbb{Z}^+, |)$ is also a lattice: $\vee = \text{lcm}$, $\wedge = \gcd$.

2.17 Relations

Definition 2.89 (Ternary relation)**.** A *ternary relation* on sets A, B, C is a subset of $A \times B \times C$: a set of triples (x, y, z) with $x \in A,\ y \in B,\ z \in C$.

Definition 2.90 (n-ary relation)**.** An *n-ary relation* on sets $A_1, A_2, \ldots, A_n$ is a subset

$$R \subseteq A_1 \times A_2 \times \cdots \times A_n,$$

i.e. a set of n-tuples $(x_1, \ldots, x_n)$ with $x_i \in A_i$ for each i. Each A_i is the i-th *domain*; n is the *degree*. An n-ary relation is also called an *n-ary predicate*.

Example 2.91. (first name, middle name, surname) triples form a ternary relation. (day, month, year) likewise. Points in $\mathbb{R}^3$ form a ternary relation on $\mathbb{R}$.

Note (Connection to tables and databases, Rosen). An n-ary relation's tuples correspond to the *rows* of an n-column table; each domain A_i (analogous to a function's codomain – allowed to contain more than what actually appears) corresponds to a *column* or *attribute*. This correspondence is the basis of *relational databases*: a database table *is* an n-ary relation, its columns are attributes, and a *key* (or combination of attributes forming a key) is one that uniquely determines each tuple – analogous to how, in a binary relation that happens to be a function, the first coordinate uniquely determines the second.

Definition 2.92 (Database, Rosen §9.2.2)**.** A *database* is (in this model) an n-ary relation, its tuples called *records*, its domains $A_1, \dots, A_n$ called *fields* or *attributes*. A *primary key* is a domain (or combination of domains) whose value(s) uniquely determine each record – i.e. no two distinct tuples agree on it.

Definition 2.93 (Selection operator, Rosen §9.2.3)**.** For an n-ary relation R and a condition C on n-tuples, the *selection operator* s_C maps R to

$$s_C(R) = \{(a_1, \dots, a_n) \in R : C(a_1, \dots, a_n) \text{ holds}\}.$$

(Rows satisfying the condition – SQL's `WHERE`.)

Definition 2.94 (Projection, Rosen §9.2.3)**.** For $1 \le i_1 < i_2 < \cdots < i_m \le n$, the *projection* $P_{i_1,\dots,i_m}$ maps each n-tuple to the m-tuple of its selected components:

$$P_{i_1,\dots,i_m}(a_1, \dots, a_n) = (a_{i_1}, a_{i_2}, \dots, a_{i_m}).$$

(Columns kept, duplicates then removed – SQL's `SELECT`, confusingly named the opposite of the relational-algebra term.)

Note. Projection can reduce the number of tuples: distinct n-tuples that agree on all m retained components collapse to one m-tuple.

Definition 2.95 (Join, Rosen §9.2.3)**.** Let R have degree m, S have degree n, and fix $p \le m, n$. The *join* $J_p(R, S)$ is the relation of degree $m + n - p$ consisting of all $(m + n - p)$-tuples

$$(a_1, \dots, a_{m-p},\ c_1, \dots, c_p,\ b_1, \dots, b_{n-p})$$

such that $(a_1, \dots, a_{m-p}, c_1, \dots, c_p) \in R$ and $(c_1, \dots, c_p, b_1, \dots, b_{n-p}) \in S$ – i.e. tuples of R and S are merged wherever their last p / first p components agree.

Note (Applications, Rosen §9.2.4). n-ary relations underlie *data mining*: a store's transaction database (each transaction an n-ary tuple of purchased items) can be queried with selection/projection/join to find frequent itemsets, association rules, etc.

2.18 Counting relations

Theorem 2.96 (Counting binary relations)**.** *For finite A, B with $|A| = m$, $|B| = n$:*

$$\#\{\textit{binary relations from } A \textit{ to } B\} = 2^{mn}.$$

In particular, with $A = B$, $|A| = m$:

$$\#\{\textit{binary relations on } A\} = 2^{m^2}.$$

Proof. A binary relation from A to B is exactly a subset of $A \times B$, so the count equals $|\mathcal{P}(A \times B)| = 2^{|A \times B|}$ (§1.9). Since $|A \times B| = mn$ (§1.14), this is 2^{mn}. The case $A = B$ substitutes $n = m$. ■

Theorem 2.97 (Counting n-ary relations)**.** *For finite $A_1, \dots, A_n$ with $|A_i| = m_i$:*

$$\#\{n\textit{-ary relations on } A_1, \dots, A_n\} = 2^{m_1 m_2 \cdots m_n}.$$

If all $A_i = A$ with $|A| = m$: 2^{m^n}.

Proof. Identical argument: an n-ary relation is a subset of $A_1 \times \cdots \times A_n$, of size $\prod_i m_i$ (§1.14 generalized), so the count is $2^{\prod_i m_i}$. ■

Example 2.98 (Counting relations with a given property, Rosen-style)**.** Reflexive relations on A with $|A| = m$: the m diagonal pairs (a, a) are forced *in*, leaving $m^2 - m$ off-diagonal pairs free to choose independently, so there are $2^{m^2 - m}$ reflexive relations on A.

2.19 Matrices (Rosen §2.6 – extension, beyond CN syllabus)

Definition 2.99 (Matrix)**.** An $m \times n$ matrix $A = [a_{ij}]$ is a rectangular array with m rows, n columns, entries a_{ij} $(1 \le i \le m,\ 1 \le j \le n)$. A is *square* if $m = n$.

Definition 2.100 (Addition, scalar multiplication)**.** For same-size A, B:

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

For scalar c:

$$(cA)_{ij} = c\, a_{ij}.$$

Addition is defined only when A, B have identical dimensions.

Definition 2.101 (Matrix multiplication)**.** For A $(m \times k)$ and B $(k \times n)$, the product AB is $m \times n$, with entry (i, j) given by

$$(AB)_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ik}b_{kj} = \sum_{l=1}^{k} a_{il}b_{lj}$$

– the dot product of row i of A with column j of B. Defined only when the number of columns of A equals the number of rows of B.

Example 2.102.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Theorem 2.103. *Matrix multiplication is associative:* $(AB)C = A(BC)$ *whenever the products are defined.*

Proof. Let A be $m \times k$, B be $k \times l$, C be $l \times n$. Both $(AB)C$ and $A(BC)$ are $m \times n$.

For entry (i, j) of $(AB)C$: first $(AB)_{ip} = a_{i1}b_{1p} + \cdots + a_{ik}b_{kp}$ for each p, then

$$\big((AB)C\big)_{ij} = \sum_{p=1}^{l} (AB)_{ip}\, c_{pj} = \sum_{p=1}^{l} \left(\sum_{q=1}^{k} a_{iq}b_{qp} \right) c_{pj} = \sum_{q=1}^{k} \sum_{p=1}^{l} a_{iq}b_{qp}c_{pj}.$$

For entry (i, j) of $A(BC)$: first $(BC)_{qj} = b_{q1}c_{1j} + \cdots + b_{ql}c_{lj}$ for each q, then

$$\big(A(BC)\big)_{ij} = \sum_{q=1}^{k} a_{iq}\, (BC)_{qj} = \sum_{q=1}^{k} a_{iq} \left(\sum_{p=1}^{l} b_{qp}c_{pj} \right) = \sum_{q=1}^{k} \sum_{p=1}^{l} a_{iq}b_{qp}c_{pj}.$$

Both expand to the same triple sum $\sum_{q,p} a_{iq}b_{qp}c_{pj}$ (finite sums may be freely reordered/regrouped), so $(AB)C = A(BC)$. ∎

Theorem 2.104. *Matrix multiplication is* not *commutative in general.*

Proof.

$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \qquad \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Different results, so $AB \neq BA$ here. ∎

Definition 2.105 (Identity matrix)**.** I_n is $n \times n$ with 1s on the diagonal, 0s elsewhere: $(I_n)_{ij} = 1$ if $i = j$, else 0.

Theorem 2.106. *For A $(m \times n)$: $I_m A = A$ and $A I_n = A$.*

Definition 2.107 (Transpose)**.** A^t: $(A^t)_{ij} = a_{ji}$ – rows and columns swapped.

Theorem 2.108. $(A + B)^t = A^t + B^t$; $(AB)^t = B^t A^t$ *(order reverses).*

Definition 2.109 (Symmetric matrix)**.** A is *symmetric* if $A = A^t$ (equivalently $a_{ij} = a_{ji}$ for all i, j; requires A square).

Definition 2.110 (Zero-one matrices, meet/join/Boolean product, Rosen – harder, sets up §2.15/9.4)**.** For matrices with entries in $\{0, 1\}$:

$$(A \wedge B)_{ij} = a_{ij} \wedge b_{ij} \quad \text{(meet)}, \qquad (A \vee B)_{ij} = a_{ij} \vee b_{ij} \quad \text{(join)}.$$

Boolean product:

$$(A \odot B)_{ij} = (a_{i1} \wedge b_{1j}) \vee (a_{i2} \wedge b_{2j}) \vee \cdots \vee (a_{ik} \wedge b_{kj}) = \bigvee_{l=1}^{k} (a_{il} \wedge b_{lj}).$$

An entry is 1 iff *some* l has $a_{il} = b_{lj} = 1$ – same shape as ordinary matrix multiplication with $(\vee, \wedge)$ in place of $(+, \times)$.

Example 2.111.

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

$$A \wedge B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \qquad A \vee B = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad A \odot B = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

For $(A \odot B)_{11}$: $(a_{11} \wedge b_{11}) \vee (a_{12} \wedge b_{21}) = (1 \wedge 0) \vee (0 \wedge 1) = 0$. For $(A \odot B)_{21}$: $(1 \wedge 0) \vee (1 \wedge 1) = 1$.

Theorem 2.112 (Connection to relations, Rosen)**.** *Let A be the zero-one matrix of relation $R \subseteq X \times Y$ (§2.13: $a_{ij} = 1 \iff (x_i, y_j) \in R$), B that of $S \subseteq Y \times Z$. Then $A \odot B$ is exactly the matrix of $S \circ R$ (§2.15).*

Proof.

$$(A \odot B)_{ij} = 1 \iff \exists l : \ a_{il} = 1 \wedge b_{lj} = 1 \iff \exists y_l : \ (x_i, y_l) \in R \wedge (y_l, z_j) \in S \iff (x_i, z_j) \in S \circ R,$$

matching the definition of relation composition exactly (§2.15). ∎

Note (Iterating for closures). Writing $A^{[n]} = A \odot A \odot \cdots \odot A$ (n copies) gives the matrix of $R^{(n)}$. For finite A ($n \times n$ zero-one matrix), the join $A \vee A^{[2]} \vee \cdots \vee A^{[n]}$ (only n terms needed, since further terms add nothing new) gives the matrix of the transitive closure R^+ (§2.15) – this is the basis of *Warshall's algorithm*, computed far more efficiently than by literally forming all powers.

3 Proofs

3.1 Theorems and proofs

Definition 3.1 (Theorem)**.** A *theorem* is a mathematical statement that has been proved true.

Definition 3.2 (Lemma, proposition, corollary – Rosen's terminology)**.** A *lemma* is a theorem whose main purpose is as a stepping stone in the proof of a more significant theorem. A *proposition* is a theorem of independent interest, typically less significant or easier to prove than the main theorems around it. A *corollary* is a theorem that follows almost immediately from one just proved.

Definition 3.3 (Proof)**.** A *proof* of a claim is a finite sequence of statements (*steps*), ending in the claim, where each step is one of:

- something already known: a *definition*, an *axiom* (a fundamental assumed property, e.g. $x(y+z) = xy + xz$), or a previously proved theorem;
- an *assumption*, made as a starting point;
- a *logical consequence* of earlier steps.

The final step establishes the claim from what precedes it.

Note (Verifiability). A proof must be readable sequentially – each step justified only by what comes *before* it, never by reading ahead. The end of a proof is marked by ■ (or historically "Q.E.D.", *quod erat demonstrandum*).

Theorem 3.4. *For any sets A, B: if $A \subseteq B$ then $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.*

Proof. Assume $A \subseteq B$. Let $X \in \mathcal{P}(A)$; then $X \subseteq A$ by definition of $\mathcal{P}(A)$. Since $A \subseteq B$, also $X \subseteq B$. Hence $X \in \mathcal{P}(B)$ by definition of $\mathcal{P}(B)$. So $X \in \mathcal{P}(A) \implies X \in \mathcal{P}(B)$, i.e. $\mathcal{P}(A) \subseteq \mathcal{P}(B)$. ■

Note (Annotating each step).

- "Assume $A \subseteq B$" – an *assumption* (the theorem is conditional on it).
- "Let $X \in \mathcal{P}(A)$" – a *definition*, naming an arbitrary element so we can reason about it generically.
- "$X \subseteq A$" – a *logical consequence* of the previous line and the definition of $\mathcal{P}(A)$.
- "$X \subseteq B$" – a consequence of $X \subseteq A$ and the assumption $A \subseteq B$.
- "$X \in \mathcal{P}(B)$" – a consequence of $X \subseteq B$ and the definition of $\mathcal{P}(B)$.
- Final line – restates the whole chain as the subset relation $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.

Every line either restates a definition/axiom, states an assumption, or follows logically from what came strictly before – exactly the requirement above.

3.2 Logical deduction

Note (Modus ponens). The core deduction rule: if P has been established, and $P \implies Q$ has been established, then Q may be deduced. This is used constantly and rarely named explicitly, but it is the essence of every logical deduction step in a proof.

Note (Implication as a subset relation). In general, for statements P, Q, let $\hat{P}, \hat{Q}$ be the sets of situations in which P, resp. Q, holds. Then

$$P \implies Q \quad \text{corresponds to} \quad \hat{P} \subseteq \hat{Q}.$$

This generalizes the set-membership fact from §1.6: $A \subseteq B \iff \forall x\,(x \in A \implies x \in B)$ – the correspondence holds even when P, Q are not phrased as membership statements at all (as the domino example shows).

Note (Vacuous truth). If X is false, $X \implies Y$ is true regardless of Y – corresponding to $\emptyset \subseteq Y$ for any set Y (§1.6, Theorem 1: the empty set is a subset of every set).

Definition 3.5 (Converse)**.** The *converse* of $P \implies Q$ is $Q \implies P$ (equivalently written $P \impliedby Q$). An implication holding does *not* entail its converse holds: implication is not symmetric, as both examples above show ($P \Rightarrow Q$ but not $Q \Rightarrow P$; $X \Rightarrow Y$ but not $Y \Rightarrow X$).

Definition 3.6 (Biconditional)**.** When both $P \implies Q$ and its converse $Q \implies P$ hold, we write $P \iff Q$ ("P if and only if Q"; P, Q are *logically equivalent*: $\hat{P} = \hat{Q}$ as sets of situations).

Definition 3.7 (Contrapositive and inverse, Rosen)**.** For $P \implies Q$: the *contrapositive* is $\neg Q \implies \neg P$; the *inverse* is $\neg P \implies \neg Q$.

Theorem 3.8. *$P \implies Q$ is logically equivalent to its contrapositive $\neg Q \implies \neg P$. It is* not*, in general, equivalent to its converse or its inverse (which are themselves contrapositives of each other).*

Proof. Both $P \Rightarrow Q$ and $\neg Q \Rightarrow \neg P$ are false in exactly one case (P true, Q false) and true otherwise – identical truth tables, hence logically equivalent. The converse $Q \Rightarrow P$ and inverse $\neg P \Rightarrow \neg Q$ each fail in the case P false, Q true (for the converse) or P false, Q true (checking directly) – a different failure case from the original, so neither need agree with $P \Rightarrow Q$ in general; the domino example ($P \Rightarrow Q$ true, $Q \Rightarrow P$ false) is a concrete witness. ■

Note (Common logical fallacies, Rosen). *Affirming the consequent*: from $P \implies Q$ and Q, wrongly concluding P (this is the *converse* error from earlier in this section). *Denying the antecedent*: from $P \implies Q$ and $\neg P$, wrongly concluding $\neg Q$ (this is the *inverse* error). *Begging the question / circular reasoning*: using the claim being proved (or something equivalent to it) as a premise in its own proof.

Example 3.9 (Affirming the consequent, in the wild)**.** "If it rains, the ground is wet. The ground is wet. Therefore it rained." Invalid: the ground could be wet from a sprinkler.

3.3 Proofs of existential and universal statements

Definition 3.10 (Existential statement)**.** An *existential statement* asserts $\exists x \in D,\ P(x)$ for some domain D and predicate P: that *something* with the stated property exists. It may be true or false.

Theorem 3.11. *English has a palindrome.*

Proof. 'rotator' is an English word, and it reads the same forwards and backwards. ■

Note (Proving $\exists x,\ P(x)$). A single *witness* suffices: exhibit one $x \in D$ and verify $P(x)$ holds. No need to search further once found.

Definition 3.12 (Universal statement)**.** A *universal statement* asserts $\forall x \in D,\ P(x)$: *everything* in D has the property.

Theorem 3.13. *Every English word has a vowel or a 'y'.*

Proof sketch. One could, in principle, check every English word individually ('aardvark' has a vowel; 'aardwolf' has a vowel; ...; 'syzygy' has a 'y'; ...) – but D here has tens of thousands of members, making case-by-case verification impractical (and hopeless for an infinite D). ■

Note (Proving $\forall x \in D,\ P(x)$). Checking every case individually only works for small, finite D – and not even then, practically. Instead: let x be an *arbitrary, generic* element of D (no properties assumed beyond $x \in D$), and show $P(x)$ follows from that alone. Since the argument uses nothing specific to the particular x chosen, it applies to *every* element of D simultaneously. This is the technique already used, e.g., in §1.6 ("let $x \in A$, show $x \in B$") and in §3.1's Theorem 13 proof ("let $X \in \mathcal{P}(A)$").

Note (Disproof is the mirror image). To *disprove* a universal statement $\forall x \in D,\ P(x)$ (i.e. prove its negation $\exists x \in D,\ \neg P(x)$), a single *counterexample* suffices – disproof of a universal is existential proof of its negation. Conversely, disproving an existential statement $\exists x \in D,\ P(x)$ requires proving the universal $\forall x \in D,\ \neg P(x)$ – generally the harder direction, since no single case can rule out all possibilities.

Theorem 3.14 (Uniqueness example)**.** *There is exactly one real number x such that $2x + 3 = 7$.*

Proof. **Existence**: $x = 2$ satisfies $2(2) + 3 = 7$.

Uniqueness: suppose x_1, x_2 both satisfy $2x + 3 = 7$. Then $2x_1 + 3 = 2x_2 + 3$, so $2x_1 = 2x_2$, so $x_1 = x_2$.

So exactly one such x exists. ■

Note (The uniqueness pattern). Assume two witnesses x_1, x_2 both satisfy the property; derive $x_1 = x_2$. Combined with a separate existence argument, this proves "exactly one."

Example 3.15 (A plausible-looking false universal claim)**.** Claim: "every prime p satisfies $2^p - 1$ is prime" (a *Mersenne prime* for every prime exponent).

Disproof by counterexample. $p = 11$ is prime, but $2^{11} - 1 = 2047 = 23 \times 89$, not prime. So the universal claim is false – a single counterexample suffices, despite the pattern holding for $p = 2, 3, 5, 7$ ($2^p - 1 = 3, 7, 31, 127$, all prime). ■

Note. This is a genuine cautionary example: checking several small cases can be misleading. $p = 2, 3, 5, 7$ all work, and it's tempting to conjecture the pattern continues – but $p = 11$ already breaks it.

3.4 Finding proofs

Note (No algorithm for finding proofs). There is no systematic method for discovering proofs, for deep theoretical reasons (Gödel 1931, Church 1936, Turing 1936). Finding proofs draws on logical reasoning, familiarity with the objects involved, practice, creativity, and persistence – not a fixed recipe.

Note (Strategy: proving $A \subseteq B$).

1. Name a general member: "Let $x \in A$."
2. Unpack what the definition of A tells you about x.
3. Follow the logical consequences.
4. Aim to conclude x satisfies the definition of B.

Note (Strategy: proving $A = B$ (sets)). Prove $A \subseteq B$ and $A \supseteq B$ separately (§1.6). Each direction uses the subset strategy above.

Note (Strategy: proving $A = B$ (numbers)). If direct symbolic manipulation (algebraic rules) can transform expression A into expression B, use that. Otherwise, prove $A \leq B$ and $A \geq B$ separately – the numerical analogue of the double-inclusion technique.

Note (Without loss of generality (WLOG), Rosen). When a proof is symmetric in two variables (swapping them leaves the claim unchanged), it suffices to prove one case and invoke the symmetry for the other – written "WLOG, assume $x \leq y$" etc. Using WLOG when the situation is *not* actually symmetric is a common error – check the symmetry claim carefully before invoking it.

Note (Forward vs. backward reasoning, Rosen). *Forward*: start from known facts/assumptions, derive consequences, until reaching the goal. *Backward*: start from the goal, ask "what would suffice to prove this?", and work backward to something already known. Backward reasoning is often how a proof is *discovered*; it is then usually rewritten forward for presentation.

3.5 Types of proofs

Note (Proof types).

- *Direct proof / proof by symbolic manipulation* (§3.6): assume P, derive Q via a direct chain of implications or equations.
- *Proof by construction* (§3.7, also called *proof by example*): exhibit a specific object and verify it works – a *constructive* existence proof, producing an explicit witness (contrast: *nonconstructive* existence proofs establish that something exists, e.g. via contradiction or a counting argument, without exhibiting it).
- *Proof by cases* (§3.8): split the domain into exhaustive cases, prove the claim in each.
- *Proof by contradiction* (§3.9): assume the negation of the claim, derive a contradiction (some statement and its negation both true).
- *Proof by contraposition*: to prove $P \implies Q$, instead prove the equivalent $\neg Q \implies \neg P$ (§3.2's contrapositive theorem) – useful when $\neg Q$ gives more to work with than P does.
- *Vacuous / trivial proof*: prove $P \implies Q$ by showing P is always false (vacuous) or Q is always true (trivial).
- *Proof by (mathematical) induction* (§3.10).
- *Uniqueness proofs*: show existence, then show any two witnesses must coincide.

This list is not exhaustive, and real proofs frequently mix several of these – e.g. a proof by cases where one case is handled by contradiction and another by direct construction.

Example 3.16 (Vacuous proof)**.** "For all $x \in \emptyset$, $x^2 < 0$." True vacuously: $\emptyset$ has no members, so the implication "$x \in \emptyset \implies x^2 < 0$" is never tested against an actual counterexample.

Example 3.17 (Trivial proof)**.** "If $n > 0$, then $n^2 + 1 > 0$." The conclusion $n^2 + 1 > 0$ holds for *every* real n regardless of the hypothesis, so the implication is trivially true.

Theorem 3.18 (Contraposition example)**.** *If n^2 is even, then n is even.*

Proof by contraposition. We prove the contrapositive: if n is odd, then n^2 is odd. Suppose $n = 2k+1$ for some $k \in \mathbb{Z}$. Then

$$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which is odd. By §3.2's contrapositive theorem, this proves the original statement. ■

Note. Contraposition was the right tool here because "n is odd" gives a concrete algebraic form $(2k+1)$ to compute with, while "n^2 is even" does not directly hand you a useful form for n itself.

3.6 Proof by symbolic manipulation

Note (The pattern). A chain of equations, each step justified by a known law (axiom or previously proved fact), linking the two sides of the claim.

Theorem 3.19 (Difference of two squares)**.**

$$(x-y)(x+y) = x^2 - y^2.$$

Proof.

$$\begin{aligned}
(x-y)(x+y) &= x^2 + xy - yx - y^2 && \text{(distributive law)} \\
&= x^2 + xy - xy - y^2 && \text{(commutativity of multiplication)} \\
&= x^2 - y^2 && \text{(additive cancellation).}
\end{aligned}$$

■

Note (Direction is arbitrary). The proof above started from the *right*-hand side and worked toward the left – equally valid as starting from the left. Pick whichever direction is more natural for the specific identity.

Note (Examples already seen in this note). This is exactly the technique behind: the De Morgan's-law proof via set-builder notation (§1.11), the chained-identity proof $\overline{A \cup (B \cap C)} = (\overline{C} \cup \overline{B}) \cap \overline{A}$ (§1.11), and the symmetric-difference identities $A \triangle B = \overline{A \triangle B}$ etc. (§1.13) – each is a sequence of equalities, each step citing a specific law.

3.7 Proof by construction

Note (Pattern). Describe a specific object precisely, and show it exists and satisfies the required conditions – a *constructive* proof of an existential statement (§3.3). Theorem 14 ('rotator' is a palindrome) is an example.

Note (Two mistakes to avoid).

- Using construction to attempt a *universal* statement: exhibiting one object with a property never establishes that *every* object has it.
- Mistaking an illustrative example for a proof: an example can clarify a proof's ideas, but is not itself a proof unless the statement being proved is genuinely existential.

Theorem 3.20 (Ramanujan's taxicab number, Rosen)**.** *There is a positive integer expressible as the sum of two positive cubes in two genuinely different ways.*

Proof.

$$1729 = 10^3 + 9^3 = 1000 + 729 = 12^3 + 1^3 = 1728 + 1.$$

Both decompositions use positive integers, and $\{10, 9\} \neq \{12, 1\}$, so these are two different ways. ■

Note. This is the smallest such number – hence its fame as the "taxicab number," from the (possibly apocryphal) anecdote of Hardy visiting Ramanujan in a taxi numbered 1729.

3.8 Proof by cases

Note (Pattern). Identify finitely many cases covering every possibility, and prove the theorem separately in each – each case-proof is a subproof. Cases may overlap (though this can signal redundant effort), but together they must be *exhaustive*: every object the theorem concerns must fall into at least one case.

Note (Typical shape). Often a theorem concerns an *infinite* set, split into a small number of cases, at least one of which still covers infinitely many objects – the reasoning within that case must be general enough to apply to all of them, not just verify a few instances.

Theorem 3.21 (Nonconstructive existence via cases, Rosen)**.** *There exist irrational numbers x, y such that x^y is rational.*

Proof. Consider $\sqrt{2}^{\sqrt{2}}$. Either it is rational or it is irrational – these are the only two (exhaustive) cases.

Case 1: $\sqrt{2}^{\sqrt{2}}$ is rational. Take $x = y = \sqrt{2}$, both irrational, with x^y rational by assumption. Done.

Case 2: $\sqrt{2}^{\sqrt{2}}$ is irrational. Take $x = \sqrt{2}^{\sqrt{2}}$ (irrational, by this case's assumption) and $y = \sqrt{2}$ (irrational). Then

$$x^y = \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^2 = 2,$$

which is rational. Done.

Either way, such x, y exist. ■

Note (Strikingly nonconstructive). This proves the statement true *without ever determining which case actually holds* – we don't know, from this proof alone, whether $\sqrt{2}^{\sqrt{2}}$ itself is rational or irrational. (It is, in fact, irrational – even transcendental, by the Gelfond–Schneider theorem – but that fact is not needed here.) This is the clearest illustration of the constructive/nonconstructive distinction from §3.5.

Definition 3.22 (Chomp)**.** A rectangular grid of cookies; players alternate taking a cookie and every cookie above-and-to-the-right of it (relative to a fixed corner); the top-left cookie is poisoned; whoever takes it loses.

Example 3.23 (Chomp – Rosen §1.8, nonconstructive existence via strategy-stealing)**.** On any rectangular Chomp board with more than one cookie, the *first* player has a winning strategy.

Proof. Consider the first player's opening move: take just the bottom-right cookie.

Case 1: this opening move is part of *some* winning strategy for the first player. Then the first player has a winning strategy – done.

Case 2: this opening move is *not* part of any winning strategy for the first player. Then the second player, facing the resulting position, must have a winning response r. But now imagine the first player had opened with exactly that move r instead (a legal first move, since it was a legal move for the second player from a position that only has more cookies removed than the empty-except-poison start). By the same argument, whatever position r leads to, the first player is now in the position the second player would have been in Case 2 – with a winning strategy available.

Either way, the first player wins. ■

Note (Why this is nonconstructive). The proof establishes a winning strategy *exists* for the first player without ever revealing what it is – indeed, for most board sizes, no explicit winning strategy is known at all. This is the sharpest possible illustration of the constructive/nonconstructive distinction (§3.5): the theorem is proved, but the object it asserts exists (the strategy) remains, in general, unknown.

Note (Strategy-stealing, the technique). The argument never analyzes what the winning strategy actually *does* – it only argues that *if* the second player had a winning response, the first player could "steal" it by playing that same move first. This strategy-stealing pattern appears throughout combinatorial game theory whenever a game has the property that having extra pieces on the board never hurts a player (true for Chomp, since eaten cookies only shrink future options for both sides equally).

3.9 Proof by contradiction

Note (Pattern). Assume the *negation* of the claim; reason from it to a contradiction (some statement and its negation both derived as true); conclude the assumption was false, so the original claim holds.

Theorem 3.24 (Euclid)**.** *There are infinitely many prime numbers.*

Proof. Suppose, for contradiction, that there are only finitely many primes $p_1, p_2, \ldots, p_n$. Let

$$q = p_1 p_2 \cdots p_n + 1.$$

Since $q > p_i$ for every i, q is not itself prime, so q is composite and hence divisible by some prime. But for each p_i, dividing q by p_i leaves remainder 1 (since p_i divides $p_1 \cdots p_n$ exactly), so $p_i \nmid q$. So q is divisible by no prime at all – contradicting that q is composite. Hence the assumption was false: there are infinitely many primes. ■

Note (A useful auxiliary fact used implicitly). The argument above (and many proofs by contradiction over $\mathbb{N}$ or $\mathbb{Z}^+$) relies on: any nonempty set of natural numbers has a smallest element (well-ordering). This lets you say things like "let n be the smallest counterexample" and derive a contradiction from its minimality.

Theorem 3.25. $\sqrt{2}$ *is irrational.*

Proof. Suppose, for contradiction, that $\sqrt{2}$ is rational: $\sqrt{2} = a/b$ for integers a, b with no common factor (in lowest terms). Squaring, $2 = a^2/b^2$, so $a^2 = 2b^2$ – hence a^2 is even, hence a is even (an odd number squares to an odd number), say $a = 2c$. Substituting: $4c^2 = 2b^2$, so $b^2 = 2c^2$ – hence b^2 is even, hence b is even. But then a and b are *both* even, contradicting that a/b was in lowest terms. So the assumption was false: $\sqrt{2}$ is irrational. ■

Note. Alongside Euclid's proof, this is the other canonical proof-by-contradiction example in mathematics – both hinge on deriving a property (divisibility, in each case) that eventually contradicts an assumption baked into the setup (finiteness of the prime list; lowest-terms form of the fraction).

Note (Open problems, Rosen). Not every true mathematical statement has a known proof. An *open problem* is a precisely stated claim believed true (or at least undecided) but not yet proved or disproved.

Example 3.26 (The Collatz conjecture)**.** Define, for $n \in \mathbb{Z}^+$:

$$\text{Collatz}(n) = \begin{cases} n/2, & n \text{ even}, \\ 3n+1, & n \text{ odd}. \end{cases}$$

Conjecture: iterating Collatz from any starting $n \in \mathbb{Z}^+$ eventually reaches 1.

Note. Verified by computer for enormous ranges of starting values, but no proof (or disproof) is known – an active open problem since 1937. Contrast with Euclid's infinitude-of-primes proof (§3.9 above): that claim, once conjectured, was fully settled by a short contradiction argument; Collatz has resisted proof for nearly a century despite its simple statement.

3.10 Proof by mathematical induction

Note (Principle of mathematical induction). To prove $S(n)$ holds for all $n \in \mathbb{N}$ (or all $n \geq n_0$):

- **Base case**: prove $S(1)$ (or $S(n_0)$).
- **Inductive step**: prove, for all k:

$$S(k) \implies S(k+1)$$

  (the assumption $S(k)$ here is the *inductive hypothesis*).

Conclude $S(n)$ holds for all n.

Note (Why this works). $S(1)$ holds (base case). Then:

$$S(1) \implies S(2), \qquad S(2) \implies S(3), \qquad S(3) \implies S(4), \qquad \ldots$$

Induction packages this infinite chain of modus ponens applications into a single finite argument, rather than an informal "and so on forever."

Theorem 3.27 (Generalized De Morgan's law, CN Thm 18)**.** *For all $n \in \mathbb{N}$ and sets $A_1, \ldots, A_n$:*

$$\overline{A_1 \cup A_2 \cup \cdots \cup A_n} = \overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_n}.$$

Proof. Let $S(n)$ be this identity for a given n.

Base case ($n = 1$):

$$\overline{A_1} = \overline{A_1}.$$

Inductive step: assume $S(k)$, i.e.

$$\overline{A_1 \cup \cdots \cup A_k} = \overline{A_1} \cap \cdots \cap \overline{A_k}.$$

Then

$$\overline{A_1 \cup \cdots \cup A_k \cup A_{k+1}} = \overline{(A_1 \cup \cdots \cup A_k) \cup A_{k+1}} = \overline{(A_1 \cup \cdots \cup A_k)} \cap \overline{A_{k+1}}$$

by ordinary De Morgan's law (§1.11). Applying $S(k)$ to the first factor:

$$= (\overline{A_1} \cap \cdots \cap \overline{A_k}) \cap \overline{A_{k+1}} = \overline{A_1} \cap \cdots \cap \overline{A_{k+1}},$$

which is $S(k+1)$.

By induction, $S(n)$ holds for all n. ∎

Theorem 3.28 (Divisibility example)**.** *For all $n \in \mathbb{N}$: $3 \mid (n^3 - n)$.*

Proof. **Base case** ($n = 0$): $n^3 - n = 0$, and $3 \mid 0$.

Inductive step: assume $3 \mid (k^3 - k)$ for some $k \geq 0$, i.e. $k^3 - k = 3m$ for some $m \in \mathbb{Z}$. Consider $n = k + 1$:

$$(k+1)^3 - (k+1) = k^3 + 3k^2 + 3k + 1 - k - 1 = (k^3 - k) + 3k^2 + 3k = 3m + 3k^2 + 3k = 3(m + k^2 + k).$$

This is 3 times an integer, so $3 \mid \big((k+1)^3 - (k+1)\big)$.

By induction, $3 \mid (n^3 - n)$ for all $n \in \mathbb{N}$. ■

Definition 3.29 (Well-ordering property)**.** Every nonempty subset of $\mathbb{N}$ has a least element.

Theorem 3.30 (Well-ordering $\implies$ mathematical induction)**.** *If the well-ordering property holds for $\mathbb{N}$, then the principle of mathematical induction is valid.*

Proof. Suppose $S(1)$ holds and $S(k) \implies S(k+1)$ for all $k \geq 1$. Suppose, for contradiction, $S(n)$ fails for some n. Let

$$F = \{n \in \mathbb{Z}^+ : S(n) \text{ is false}\}.$$

$F \neq \emptyset$ by assumption, so by well-ordering, F has a least element m. Since $S(1)$ holds, $m \neq 1$, so $m \geq 2$ and $m - 1 \geq 1$. Since m is the *least* element of F, $m - 1 \notin F$, i.e. $S(m-1)$ holds. But then the inductive step gives $S(m-1) \implies S(m)$, so $S(m)$ holds – contradicting $m \in F$.

So $F = \emptyset$: $S(n)$ holds for all n. ■

Note. This is why induction and well-ordering are often called *equivalent* principles over $\mathbb{N}$ – each can be used to derive the other, and most texts (Rosen included) simply take one as an axiom and prove the other as a consequence.

Definition 3.31 (Principle of strong induction, formal statement)**.** To prove $S(n)$ for all $n \geq n_0$:

- **Base case**: prove $S(n_0)$.
- **Inductive step**: prove, for all $k \geq n_0$: if $S(n_0), S(n_0+1), \ldots, S(k)$ *all* hold, then $S(k+1)$ holds.

Note (Strong vs. weak induction – equivalent in power). Strong induction's hypothesis is more generous (all of $S(n_0), \ldots, S(k)$, not just $S(k)$), but this doesn't let you prove anything weak induction can't – any strong-induction proof can be mechanically rewritten as a weak-induction proof of the stronger statement $T(n) = S(n_0) \wedge \cdots \wedge S(n)$. Strong induction is a convenience, not a logically stronger tool. See §3.11 for worked examples of both.

3.11 Induction: more examples

Theorem 3.32 (Sum of the first n positive integers, CN Thm 19)**.** *For all $n \geq 1$:*

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Proof. **Base case** ($n = 1$): LHS $= 1$, RHS $= 1 \cdot 2/2 = 1$.

Inductive step: assume $1 + \cdots + k = k(k+1)/2$ for some $k \geq 1$. Then

$$1 + \cdots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = (k+1)\left(\frac{k}{2} + 1\right) = \frac{(k+1)(k+2)}{2},$$

which is the formula for $n = k + 1$.

By induction, the formula holds for all $n \geq 1$. ■

Theorem 3.33 ($2^n > n^2$ for $n \geq 5$, Rosen – inequality with nontrivial base)**.** *For every integer $n \geq 5$: $2^n > n^2$.*

Proof. **Base case** ($n = 5$): $2^5 = 32 > 25 = 5^2$.

Inductive step: let $k \geq 5$, assume $2^k > k^2$. We want $2^{k+1} > (k+1)^2$.

$$2^{k+1} = 2 \cdot 2^k > 2k^2 \qquad \text{(inductive hypothesis)}.$$

It suffices to show $2k^2 \geq (k+1)^2$ for $k \geq 5$:

$$2k^2 - (k+1)^2 = k^2 - 2k - 1 = (k-1)^2 - 2,$$

which is positive whenever $(k-1)^2 > 2$, i.e. certainly for $k \geq 5$. So $2k^2 > (k+1)^2$, hence $2^{k+1} > 2k^2 > (k+1)^2$.

By induction, $2^n > n^2$ for all $n \geq 5$. ■

Note (Why the base case matters here). The inequality is *false* for $n = 1, 2, 3, 4$. Getting the base case right – and matching it to where the inductive step's algebra actually works – is essential; a careless choice of base case can make the whole proof invalid even though the inductive step itself is correctly argued.

Theorem 3.34 (Bernoulli's inequality, Rosen)**.** *For all $x \geq -1$ and all integers $n \geq 0$:*

$$(1+x)^n \geq 1 + nx.$$

Proof. **Base case** ($n = 0$): $(1+x)^0 = 1 = 1 + 0 \cdot x$.

Inductive step: assume $(1+x)^k \geq 1 + kx$ for some $k \geq 0$. Since $x \geq -1$, we have $1 + x \geq 0$, so multiplying both sides of the inductive hypothesis by $(1+x)$ preserves the inequality:

$$(1+x)^{k+1} = (1+x)^k(1+x) \geq (1+kx)(1+x) = 1 + kx + x + kx^2 = 1 + (k+1)x + kx^2.$$

Since $kx^2 \geq 0$:

$$(1+x)^{k+1} \geq 1 + (k+1)x.$$

By induction, the inequality holds for all $n \geq 0$. ■

Note. The condition $x \geq -1$ is exactly what's needed for $1 + x \geq 0$, which is what allows multiplying an inequality through by $(1+x)$ without flipping its direction.

Theorem 3.35 (Harmonic numbers, Rosen)**.** *Let* $H_m = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{m}$*. Then for every* $n \geq 0$*:*

$$H_{2^n} \geq 1 + \frac{n}{2}.$$

Proof. **Base case** ($n = 0$): $H_{2^0} = H_1 = 1 \geq 1 + 0$.

Inductive step: assume $H_{2^k} \geq 1 + \frac{k}{2}$ for some $k \geq 0$. Then

$$H_{2^{k+1}} = H_{2^k} + \frac{1}{2^k + 1} + \frac{1}{2^k + 2} + \cdots + \frac{1}{2^{k+1}}.$$

There are 2^k terms in this added block, each $\geq \frac{1}{2^{k+1}}$, so the block sums to at least $2^k \cdot \frac{1}{2^{k+1}} = \frac{1}{2}$. Hence

$$H_{2^{k+1}} \geq \left(1 + \frac{k}{2}\right) + \frac{1}{2} = 1 + \frac{k+1}{2}.$$

By induction, $H_{2^n} \geq 1 + \frac{n}{2}$ for all $n \geq 0$. ■

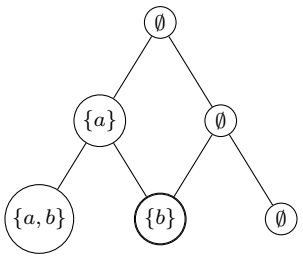
Note. Since $1 + n/2 \to \infty$ as $n \to \infty$, this proves the harmonic series diverges.

Theorem 3.36 (Counting subsets, Rosen – revisits §1.8 with a picture)**.** *A set with* n *elements has* 2^n *subsets.*

Proof. **Base case** ($n = 0$): $\emptyset$ has exactly $1 = 2^0$ subset (itself).

Inductive step: assume a k-element set has 2^k subsets. A $(k+1)$-element set $S = T \cup \{x\}$ (with $|T| = k$, $x \notin T$) has subsets falling into two kinds: those not containing x (exactly the 2^k subsets of T), and those containing x (each is $\{x\} \cup U$ for a subset U of T, again 2^k of them). Total: $2^k + 2^k = 2^{k+1}$.

By induction, an n-element set has 2^n subsets. ■



Building all subsets of $\{a, b\}$ by deciding "include a?" then "include b?" – each level doubles the count

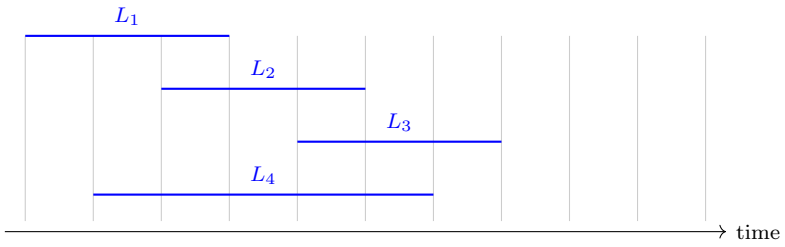
Definition 3.37 (Depth of a set of intervals, Rosen)**.** For lectures given as time intervals, the *depth* is the maximum number that mutually overlap at any single instant.

Theorem 3.38 (Greedy lecture-scheduling algorithm is optimal, Rosen)**.** *Sorting lectures by start time and greedily assigning each to the first available room needs exactly* depth *rooms – optimal, since no schedule can use fewer than* depth *rooms.*

Proof sketch, induction on the number of lectures scheduled. **Base case**: the first lecture needs 1 room; $1 \leq$ depth.

Inductive step: suppose the first k lectures (in start-time order) have been assigned using at most depth rooms. When lecture $k+1$ arrives, if all currently-open rooms are busy at that moment, the number of busy rooms equals the number of lectures overlapping at that instant – at most depth. If exactly depth rooms are busy, opening one more room suffices and keeps the total at depth; if fewer are busy, an existing free room is reused.

By induction, the greedy algorithm never needs more than depth rooms – matching the lower bound, hence optimal. ■



L_1 and L_4 overlap L_2; L_4 overlaps L_3 too – depth $= 2$ at every instant, so 2 rooms suffice

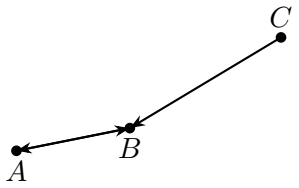
Theorem 3.39 (Odd pie fight – creative induction)**.** *An odd number n of people stand at pairwise distinct distances. Each simultaneously throws a pie at their* nearest *neighbor. Then at least one person is not hit.*

Proof. Induction on odd n (i.e. $n = 2m + 1$, induct on $m \geq 1$).

Base case ($n = 3$): let A, B, C be the three people. Some pair – say A, B – is the *closest* pair overall. Then A, B pie each other; C's pie goes to whichever of A, B is closer to C. Either way, A or B is hit twice and C isn't hit – a survivor exists.

Inductive step: assume the claim for $n = 2k + 1$ people, and consider $n = 2k + 3$. Find the overall closest pair A, B – they pie each other. Remove A, B: the other $2k + 1$ people still throw pies among themselves at mutually distinct distances – a valid instance of the same problem. By the inductive hypothesis, this group has a survivor, who also survives in the full group.

By induction, every odd n has a survivor. ■



Base case $n = 3$: A, B are the closest pair and pie each other; C pies B – A survives

Note. False for even n: e.g. 4 people can be arranged in two mutual-nearest-neighbor pairs, hitting everyone.

Theorem 3.40 (Deficient chessboard tiling, ordinary induction – Rosen §5.1)**.** *For every $n \geq 1$, a $2^n \times 2^n$ chessboard with any* one *square removed can be exactly tiled by L-trominoes.*

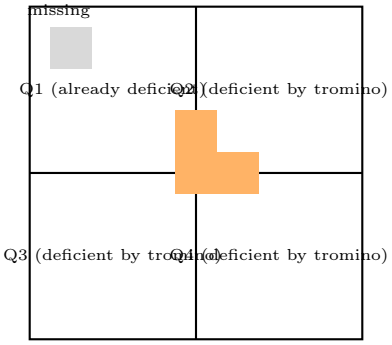
Proof. **Base case** ($n = 1$): a 2×2 board minus one square is exactly one L-tromino.

Inductive step: assume every deficient $2^k \times 2^k$ board can be tiled. Consider a deficient $2^{k+1} \times 2^{k+1}$ board, split into four $2^k \times 2^k$ quadrants.

The missing square lies in exactly one quadrant – deficient, tileable by the inductive hypothesis. Place one L-tromino at the very center, covering the corner cell of *each* of the other three quadrants nearest the center – making each of those quadrants deficient too.

One central tromino plus four tiled quadrants tiles the whole board.

By induction (using only $P(k) \Rightarrow P(k+1)$ – this is *ordinary*, not strong, induction, despite the recursive flavor), the claim holds for all $n \geq 1$. ■



One tromino at the center makes the other three quadrants deficient too

Theorem 3.41 (Postage stamp problem, Rosen – strong induction)**.** *Every integer $n \geq 12$ can be formed using only 4-cent and 5-cent stamps.*

Proof. **Base cases** ($n = 12, 13, 14, 15$):

$$12 = 4+4+4, \qquad 13 = 4+4+5, \qquad 14 = 4+5+5, \qquad 15 = 5+5+5.$$

Inductive step: let $k \geq 15$, and suppose every integer from 12 to k can be formed. Consider $k+1$. Since $k+1 \geq 16$, we have $k+1-4 \geq 12$, so by the strong inductive hypothesis, $k+1-4$ can be formed. Add one more 4-cent stamp to form $k+1$.

By strong induction, every $n \geq 12$ can be formed. ■

Note (Why strong induction, not weak). The step for $k+1$ reaches back to $k+1-4$, which for small k is *not* simply k or $k-1$ – ordinary induction, which only hands you $S(k)$, isn't enough here.

Theorem 3.42 (Existence of prime factorization – classic strong induction example)**.** *Every integer $n \geq 2$ can be written as a product of one or more primes.*

Proof. **Base case** ($n = 2$): 2 itself is prime.

Inductive step: let $k \geq 2$, and suppose every integer from 2 to k has a prime factorization. Consider $n = k+1$.

Case 1: n is prime. Then n itself is its factorization.

Case 2: n is composite, so $n = ab$ for integers $2 \leq a, b \leq n-1 = k$. By the strong inductive hypothesis, both a and b have prime factorizations; concatenating them gives a prime factorization of $n = ab$.

Either way, n has a prime factorization.

By strong induction, every $n \geq 2$ has one. ■

Note. This needs *strong* induction: the factors a, b of composite n can be much smaller than $n-1$, so the hypothesis must cover *all* smaller values.

Theorem 3.43 (Division algorithm – existence part, via well-ordering)**.** *For $a \in \mathbb{Z},\ d \in \mathbb{Z}^+$, there exist $q, r \in \mathbb{Z}$ with $a = dq + r$ and $0 \le r < d$.*

Proof. Consider $S = \{a - dq : q \in \mathbb{Z},\ a - dq \ge 0\}$. $S \subseteq \mathbb{N}$ and $S \neq \emptyset$. By well-ordering, S has a least element $r = a - dq_0$.

We claim $r < d$. If not, $r - d = a - d(q_0 + 1) \in S$, but $r - d < r$, contradicting minimality of r.

So $r < d$, and $q = q_0$ gives $a = dq + r$ with $0 \le r < d$. ■

Example 3.44 (Game-strategy example, Rosen – strong induction, non-numeric)**.** Two players alternately remove 1 or 2 stones from a pile; the player taking the last stone wins. Claim: the first player wins whenever $n \not\equiv 0 \pmod 3$, and loses whenever $n \equiv 0 \pmod 3$.

Proof sketch, strong induction on n. **Base cases**: $n = 1, 2$: first player wins. $n = 3$: whatever the first player takes, the opponent wins – first player loses.

Inductive step: assume the claim for all pile sizes less than n. If $n \equiv 0 \pmod 3$: any move leaves $n-1$ or $n-2$, neither $\equiv 0 \pmod 3$, so the opponent wins – first player loses. If $n \not\equiv 0 \pmod 3$: the first player can always leave a multiple of 3, putting the opponent in a losing position. ■

Definition 3.45 (Simple polygon, diagonal)**.** A *simple polygon* is a polygon whose sides do not cross. A *diagonal* is a line segment connecting two non-adjacent vertices that lies entirely inside the polygon.

Lemma 3.46 (Every simple polygon with ≥ 4 sides has a diagonal)**.**

Proof sketch. Take the leftmost vertex v and its two neighbors u, w. If segment uw lies inside the polygon, it is a diagonal. Otherwise, some vertex lies inside triangle uvw; take the one farthest from line uw – the segment from v to that vertex is a diagonal. ■

Theorem 3.47 (Triangulation of simple polygons – Rosen, strong induction)**.** *Every simple polygon with $n \ge 3$ sides can be triangulated into exactly $n-2$ triangles using $n-3$ non-crossing diagonals.*

Proof. **Base case** ($n = 3$): a triangle is already $1 = 3 - 2$ triangle, using $0 = 3 - 3$ diagonals.

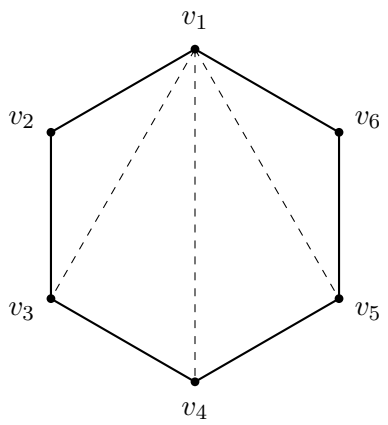
Inductive step: let $k \ge 3$, and suppose every simple polygon with fewer than $k+1$ sides can be triangulated as claimed. Let P have $n = k + 1 \ge 4$ sides.

By the lemma, P has a diagonal d, splitting P into P_1, P_2 sharing edge d, with $n_1 + n_2 = n + 2$.

Since $3 \le n_1, n_2 \le n - 1$, the strong inductive hypothesis applies to both: P_1 triangulates into $n_1 - 2$ triangles, P_2 into $n_2 - 2$.

Together: $(n_1 - 2) + (n_2 - 2) = n - 2$ triangles, using $1 + (n_1 - 3) + (n_2 - 3) = n - 3$ diagonals.

By strong induction, the claim holds for all $n \ge 3$. ■



A hexagon triangulated by 3 diagonals into 4 triangles

Theorem 3.48 (The matchstick game – Rosen §5.2, Example 4, strong induction)**.** *Two piles start with n matches each. Players alternate turns; each turn, remove any positive number of matches from* one *pile. The player who removes the last match wins. Then the* second *player can always force a win, for every $n \geq 1$.*

Proof. Let $P(n)$: "if both piles start with n matches, the second player can force a win."

Base case ($n = 1$): the first player must remove the single match from one pile; the second takes the other and wins.

Inductive step: let $k \geq 1$, suppose $P(1), \ldots, P(k)$ hold. Whatever the first player does – removing r matches from one pile – the second mirrors on the other pile.

- If $r = k + 1$: the second empties the other pile, winning immediately.
- If $r < k + 1$: both piles now have equal size $k + 1 - r \leq k$ – by $P(k + 1 - r)$, the second player can force a win from there.

Either way, the second player wins. By strong induction, $P(n)$ holds for all $n \geq 1$. ■

Theorem 3.49 (Round-robin tournament ranking – classic strong induction example)**.** *In a round-robin tournament with n players, the players can be labeled $p_1, \ldots, p_n$ such that p_i beats p_{i+1} for every i.*

Proof. **Base case** ($n = 1$): trivial.

Inductive step: let $k \geq 1$, suppose the claim holds for at most k players. Consider $k+1$ players. Pick any player x; let B = players who beat x, L = players x beats, so $|B|, |L| \leq k$.

By the inductive hypothesis, B has a chain ordering $b_1, \ldots, b_{|B|}$, and L has $\ell_1, \ldots, \ell_{|L|}$. Concatenate:

$$b_1, \ldots, b_{|B|},\ x,\ \ell_1, \ldots, \ell_{|L|}.$$

The joins $b_{|B|} \to x$ and $x \to \ell_1$ hold by definition of B, L.

By strong induction, the claim holds for all $n \geq 1$. ■

3.12 Induction: extended example

Note (Setup). m job positions A ($|A| = m$), n applicants B ($|B| = n$), a *shortlist relation* $S \subseteq A \times B$.

For $X \subseteq A$, write

$$S(X) = \bigcup_{a \in X} S(a)$$

(§2.13 notation) for the combined shortlist of positions in X.

A valid assignment (every position filled, no applicant doubled) is exactly an *injection* $A \to B$ contained in S.

Theorem 3.50 (Necessity, CN Thm 20)**.** *If S contains an injection with domain A, then $|X| \leq |S(X)|$ for every $X \subseteq A$.*

Proof. An injection assigns distinct applicants to distinct positions, so the positions in X alone already require $|X|$ distinct applicants, all drawn from $S(X)$. ■

Theorem 3.51 (Sufficiency, CN Thm 21 – the converse, by induction on $|A|$)**.** *If $|X| \leq |S(X)|$ for every $X \subseteq A$, then S contains an injection with domain A.*

Proof. Induction on $m = |A|$.

Base case ($m = 1$):

$A = \{a\}$; the condition with $X = \{a\}$ gives $|S(a)| \geq 1$, so pick any $(a, b) \in S$ – this single pair is an injection with domain A.

Inductive step: assume the theorem for all relations with domain-size $\leq m$. Let $|A| = m + 1$ and suppose $|X| \leq |S(X)|$ for all $X \subseteq A$.

Case 1: every nonempty proper $X \subsetneq A$ has *strict* slack,

$$|X| \leq |S(X)| - 1.$$

Pick any $(a, b) \in S$. Restrict S to

$$T = S \cap \big((A \setminus \{a\}) \times (B \setminus \{b\})\big),$$

i.e. drop a and b. For $X \subseteq A \setminus \{a\}$, the strict slack gives $|T(X)| \geq |X|$ even after excluding b.

Since $|A \setminus \{a\}| = m$, the inductive hypothesis applies: T contains an injection g with domain $A \setminus \{a\}$.

Then $g \cup \{(a, b)\}$ is an injection with domain A, contained in S.

Case 2: some nonempty proper $X \subsetneq A$ has *exact* fit,

$$|X| = |S(X)|.$$

Apply the inductive hypothesis to S restricted to $X \times S(X)$ (domain size $|X| \leq m$): get an injection $g : X \to S(X)$.

For the rest, $A \setminus X$: for any $Z \subseteq A \setminus X$, a counting argument using $|X \cup Z| \leq |S(X \cup Z)|$ and $|S(X)| = |X|$ shows

$$|Z| \leq |S(Z) \setminus S(X)|.$$

So the relation $U = S \cap \big((A \setminus X) \times (B \setminus S(X))\big)$ satisfies the same slack condition on domain $A \setminus X$ (size $\leq m$), so by the inductive hypothesis, U contains an injection h with domain $A \setminus X$.

Since g, h have disjoint domains (X, $A \setminus X$) and disjoint codomains ($S(X)$, $B \setminus S(X)$), $g \cup h$ is an injection with domain A, contained in S.

Cases 1 and 2 are exhaustive, completing the inductive step. By induction, the theorem holds for all m. ■

Corollary 3.52 (CN Cor. 22)**.** *$S \subseteq A \times B$ contains an injection with domain $A \iff |X| \leq |S(X)|$ for every $X \subseteq A$.*

Note (Why this matters). This gives easily *verifiable* evidence in both directions: a "yes" is witnessed by the injection itself (checkable directly); a "no" is witnessed by a single offending set X with $|X| > |S(X)|$ – no need to search the whole space of possible assignments. This is a defect-form statement of *Hall's marriage theorem*.

3.13 Mathematical induction and statistical induction

Note. Statistical induction (drawing general conclusions from data, typically under randomness, tolerating some error) is unrelated to *mathematical induction* despite the shared name. Statistical induction is not logical deduction and cannot serve as a proof step; mathematical induction is a rigorous deductive technique. Don't conflate the two.

3.14 Programs and proofs

Note (The analogy, point by point).

- **Statements**: a program is a sequence of statements in a programming language; a proof is a sequence of statements in mathematical language (augmented by precise natural language).
- **Variables**: both use named references to objects; program variables have allocated memory, proof variables do not.
- **Sequential dependence**: a program statement uses only the most recently computed value of a variable, never a later one; a proof statement is justified only by *earlier* statements, never later ones.
- **Branching**: an if-statement picks a branch by a logical condition; proof by cases (§3.8) covers all situations via a finite set of logical conditions.
- **Calling other code**: a program calls other functions/programs; a proof invokes previously proved theorems.
- **Repetition**: a program repeats via loops or recursion; a proof repeats (in effect) via mathematical induction.
- **Bugs**: a program with a bug may crash on some inputs but still work on others; a proof with a genuine gap is *not a proof at all* – it must be repaired, or the claim may simply be false.

Note (Loops $\leftrightarrow$ induction, Rosen §5.5). Proving a loop computes the correct result typically uses a *loop invariant*: a statement I that (i) holds before the loop begins, and (ii) if true before an iteration, remains true after it – exactly the base case and inductive step of mathematical induction, applied to "iteration number." If I also implies the desired postcondition once the loop terminates, this establishes *partial correctness*.

Note (Recursion $\leftrightarrow$ structural induction, Rosen §5.3-5.4). Proving a recursive function correct mirrors proving a recursively-defined structure has some property: verify the function is correct on the base case(s) of the recursion, then verify that, *assuming* correctness on smaller/simpler inputs (the recursive calls), the function is correct on the current input – the direct analogue of an inductive hypothesis.

Note (Programming paradigms). Just as learning multiple programming paradigms (procedural, object-oriented, logic, functional) improves programming skill generally, learning to write rigorous proofs – a distinct "paradigm" of precise reasoning – transfers back to writing better, more carefully-reasoned programs.

4 Propositional Logic

4.1 Truth values

Definition 4.1 (Truth values)**.** Logic (classical, two-valued) is built on two *truth values*: *True* (T) and *False* (F).

Note. Physically, truth values may correspond to presence/absence of electrical current, magnetisation, a switch's on/off state, etc. – the abstraction lets us reason about logic independently of implementation. The *bit* (0 or 1) is a closely related abstraction: $F \leftrightarrow 0$, $T \leftrightarrow 1$.

4.2 Boolean variables

Definition 4.2 (Boolean / propositional variable)**.** A *Boolean variable* (or *propositional variable*) is a variable whose value is T or F.

4.3 Propositions

Definition 4.3 (Proposition, Rosen)**.** A *proposition* is a declarative sentence that is either true or false, but not both.

Example 4.4 (Propositions)**.** $1 + 1 = 2$ (true); "The earth is flat." (false); "It will rain tomorrow." (a proposition – has a definite truth value, even if unknown to us right now).

Example 4.5 (Not propositions)**.** Questions ("Come and work for us!"), nonsensical or non-declarative sentences (Lewis Carroll's "'Twas brillig..."), and self-referential sentences with no consistent truth value ("This statement is false.") are not propositions.

Note (Naming a proposition). A proposition may be named by a Boolean variable, e.g. let X be the proposition $1 + 1 = 2$; then X's truth value is T.

4.4 Logical operations

Note (The basic connectives).

Not	$\neg$
And	$\wedge$
Or	$\vee$
Implies	$\implies$
Equivalence	$\iff$

Logical operations combining propositions are also called *connectives*; in hardware they are implemented as *logic gates*.

4.5 Negation

Definition 4.6 (Negation)**.** For proposition P, $\neg P$ is true iff P is false.

P	$\neg P$
F	T
T	F

Theorem 4.7 (Double negation)**.**

$$\neg\neg P = P.$$

Note (Analogy with set complement). Negation mirrors complementation (§1.11): both give the "completely opposite" outcome, and applying either twice returns the original. If P is the proposition $x \in A$, then $\neg P$ is $x \in \overline{A}$: e.g. $\neg(\sqrt{2} \in \mathbb{Q})$ is the same as $\sqrt{2} \in \overline{\mathbb{Q}}$ (equivalently $\sqrt{2} \notin \mathbb{Q}$).

4.6 Conjunction

Definition 4.8 (Conjunction)**.** For propositions P, Q: $P \wedge Q$ ("P and Q") is true iff *both* P and Q are true.

P	Q	$P \wedge Q$
F	F	F
F	T	F
T	F	F
T	T	T

Note (Analogy with set intersection). For object x and sets A, B:

$$(x \in A) \wedge (x \in B) \iff x \in A \cap B, \qquad A \cap B = \{x : x \in A \wedge x \in B\}$$

(restating §1.12's definition using $\wedge$).

Definition 4.9 (Generalized (n-ary) conjunction, Rosen)**.** For propositions $P_1, \ldots, P_n$:

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n$$

is true iff *every* P_i is true.

Example 4.10 (Translating English "and", Rosen)**.** "You can access the internet from campus only if you are a computer science major or you are not a freshman" involves nested connectives; more simply, "It is below freezing and snowing" translates directly as $P \wedge Q$ for $P =$"it is below freezing", $Q =$"it is snowing". Care is needed when English sentences bury an implicit "and" inside a single clause (e.g. "x is between 2 and 5" is really $(x > 2) \wedge (x < 5)$).

4.7 Disjunction

Definition 4.11 (Disjunction)**.** For propositions P, Q: $P \vee Q$ ("P or Q") is true iff *at least one* of P, Q is true.

P	Q	$P \vee Q$
F	F	F
F	T	T
T	F	T
T	T	T

Note (Inclusive, not exclusive). This is the *inclusive-or*: true even when *both* P, Q are true. Contrast with *exclusive-or* (§4.11), true precisely when *exactly one* holds.

Example 4.12 (English "or" is ambiguous, Rosen)**.** "Students who have taken CS100 or CS200 may take this class" – inclusive: having taken both still qualifies. "Soup or salad comes with this entree" – exclusive in ordinary restaurant use: you get one, not both, even though the sentence uses the same word "or." Mathematical $\vee$ always means *inclusive*-or unless stated otherwise; natural language is not reliable here.

Note (Analogy with set union). For object x and sets A, B:

$$(x \in A) \vee (x \in B) \iff x \in A \cup B, \qquad A \cup B = \{x : x \in A \vee x \in B\}$$

(restating §1.12's definition using $\vee$).

Note (Operator precedence so far, Rosen). Among the connectives introduced up to this point, precedence (highest to lowest) is:

$$\neg \ > \ \wedge \ > \ \vee.$$

So $\neg P \vee Q$ means $(\neg P) \vee Q$, and $P \wedge Q \vee R$ means $(P \wedge Q) \vee R$ – parenthesize explicitly whenever there's any doubt.

4.8 De Morgan's Laws

Theorem 4.13 (De Morgan's Laws for propositions)**.**

$$\neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg(P \wedge Q) = \neg P \vee \neg Q.$$

Proof of the first law, by truth table.

P	Q	$P \vee Q$	$\neg(P \vee Q)$	$\neg P$	$\neg Q$	$\neg P \wedge \neg Q$
F	F	F	T	T	T	T
F	T	T	F	T	F	F
T	F	T	F	F	T	F
T	T	T	F	F	F	F

The columns for $\neg(P \vee Q)$ and $\neg P \wedge \neg Q$ agree in every row, so the two expressions are *logically equivalent* – true or false together, for every assignment of truth values to P, Q. ■

Note (Building the table incrementally). The table is constructed left to right, each column using only earlier columns: start with all combinations of P, Q; build $P \vee Q$; negate it to get $\neg(P \vee Q)$; separately build $\neg P$ and $\neg Q$; conjoin them to get $\neg P \wedge \neg Q$. Comparing the two target columns (highlighted) completes the proof.

Note (Deriving the second law from the first). Rather than repeating the truth-table argument, the second law follows from the first by substitution: replace P, Q with $\neg P, \neg Q$ in the first law and simplify using double negation (§4.5).

Theorem 4.14 (Generalized De Morgan's law, CN Thm 23)**.** *For all n:*

$$\neg(P_1 \vee \cdots \vee P_n) = \neg P_1 \wedge \cdots \wedge \neg P_n.$$

Proof.

$$\begin{aligned} \text{LHS is true} &\iff P_1 \vee \cdots \vee P_n \text{ is false} \\ &\iff P_1, \ldots, P_n \text{ are all false} \\ &\iff \neg P_1, \ldots, \neg P_n \text{ are all true} \\ &\iff \text{RHS is true.} \end{aligned}$$

■

Note. This argument works for *every* n at once – unlike the truth-table method, which would need a separate (and infinite) family of tables, one per n.

Note (Correspondence with the set-theoretic De Morgan's laws). Compare directly with §1.11's Theorem (De Morgan's laws for sets): $\overline{A \cup B} = \overline{A} \cap \overline{B}$ and $\overline{A \cap B} = \overline{A} \cup \overline{B}$ are exactly the propositional laws above, applied to the propositions $x \in A$ and $x \in B$.

Note (Rosen: naming logical equivalence). Two compound propositions are *logically equivalent* iff their truth tables are identical in every row – written $P \equiv Q$ or $P \iff Q$ (a *tautology*, true under every assignment; formalized further in §4.12).

4.9 Implication

Definition 4.15 (Implication)**.** $P \implies Q$ ("if P then Q") is false only when P is true and Q is false.

P	Q	$P \implies Q$
F	F	T
F	T	T
T	F	F
T	T	T

Note (Rosen's phrasings). $P \implies Q$ reads: "if P then Q"; "P only if Q"; "P is sufficient for Q"; "Q is necessary for P"; "Q whenever P".

4.10 Equivalence

Definition 4.16 (Equivalence / biconditional)**.** $P \iff Q$ is true iff P, Q have the same truth value.

P	Q	$P \iff Q$
F	F	T
F	T	F
T	F	F
T	T	T

Theorem 4.17.

$$P \iff Q \quad \equiv \quad (P \implies Q) \wedge (Q \implies P).$$

Note (Rosen's phrasing). $P \iff Q$ reads: "P if and only if Q"; "P is necessary and sufficient for Q".

Note (Analogy with set equality). $x \in A \iff x \in B$ for all x is exactly $A = B$ (§1.6).

4.11 Exclusive-or

Definition 4.18 (Exclusive-or)**.** $P \oplus Q$ is true iff exactly one of P, Q is true.

P	Q	$P \oplus Q$
F	F	F
F	T	T
T	F	T
T	T	F

Theorem 4.19.

$$P \oplus Q = \neg(P \iff Q).$$

Theorem 4.20 (Generalized XOR parity, CN Thm 24)**.** *For all n and propositions $P_1, \ldots, P_n$: $P_1 \oplus \cdots \oplus P_n$ is true iff an* odd *number of the P_i are true.*

Proof. Induction on n.

Base case ($n = 1$): P_1 alone is true iff exactly 1 (odd) of it is true.

Inductive step: assume the claim for k propositions. Then

$$P_1 \oplus \cdots \oplus P_{k+1} = (P_1 \oplus \cdots \oplus P_k) \oplus P_{k+1}.$$

By the inductive hypothesis, the first factor is T iff an odd number of $P_1, \ldots, P_k$ are true, else F. Checking all four combinations of (that factor, P_{k+1}) against $\oplus$'s truth table shows the result is T exactly when the *total* count of true propositions among $P_1, \ldots, P_{k+1}$ is odd.

By induction, the claim holds for all n. ■

Note (Analogy with symmetric difference). $x \in A_1 \triangle \cdots \triangle A_n \iff (x \in A_1) \oplus \cdots \oplus (x \in A_n)$ (§1.13).

Example 4.21 (Knights and knaves, Rosen – a harder puzzle)**.** On an island, every inhabitant is either a *knight* (always tells the truth) or a *knave* (always lies). A says: "B is a knave." B says: "A and I are of opposite types."

Let $a =$ "A is a knight", $b =$ "B is a knight." A's statement, if A is a knight, must be true, and if A is a knave, must be false – i.e. $a \iff \neg b$. B's statement "A, B are opposite types" is $a \oplus b$; by the same reasoning, $b \iff (a \oplus b)$.

From $a \iff \neg b$: exactly one of a, b is true. Substitute into $b \iff (a \oplus b)$: since exactly one of a, b is true, $a \oplus b$ is true, so this forces b true, hence a false. So A is a knave, B is a knight.

4.12 Tautologies and logical equivalence

Definition 4.22 (Tautology)**.** A proposition that is true under every assignment of truth values (every row of its truth table is T).

Definition 4.23 (Logical equivalence)**.** $P \equiv Q$ (written $P = Q$) iff their truth tables are identical, iff $P \iff Q$ is a tautology.

Note (Key equivalences).

$$\neg\neg P = P, \qquad \neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg(P \wedge Q) = \neg P \vee \neg Q,$$
$$P \implies Q = \neg P \vee Q, \qquad P \iff Q = (P \implies Q) \wedge (Q \implies P),$$
$$P \oplus Q = \neg(P \iff Q) = P \iff \neg Q.$$

Theorem 4.24 (Exportation law, Rosen – harder derivation)**.**

$$(P \wedge Q) \implies R \;\equiv\; P \implies (Q \implies R).$$

Proof. Using $X \implies Y = \neg X \vee Y$ repeatedly:

$$\begin{aligned}
(P \wedge Q) \implies R &= \neg(P \wedge Q) \vee R = (\neg P \vee \neg Q) \vee R && \text{(De Morgan's)} \\
&= \neg P \vee (\neg Q \vee R) && \text{(associativity of } \vee) \\
&= \neg P \vee (Q \implies R) \\
&= P \implies (Q \implies R).
\end{aligned}$$

■

4.13 History

4.14 Distributive Laws

Theorem 4.25 (Distributive laws)**.**

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R), \qquad P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R).$$

Theorem 4.26 (Absorption laws, Rosen – derived, not axiomatic)**.**

$$P \vee (P \wedge Q) = P, \qquad P \wedge (P \vee Q) = P.$$

Proof.

$$\begin{aligned} P \vee (P \wedge Q) &= (P \wedge T) \vee (P \wedge Q) && \text{(identity: } P = P \wedge T) \\ &= P \wedge (T \vee Q) && \text{(distributive law)} \\ &= P \wedge T && \text{(domination: } T \vee Q = T) \\ &= P. \end{aligned}$$

The second identity is the dual (swap $\wedge \leftrightarrow \vee$, $T \leftrightarrow F$ throughout). ■

4.15 Laws of Boolean algebra

Note (Full table, dual pairs).

$$\neg\neg P = P$$

$$\neg T = F, \quad \neg F = T$$

$$P \wedge Q = Q \wedge P, \qquad P \vee Q = Q \vee P \quad \text{(commutative)}$$

$$(P \wedge Q) \wedge R = P \wedge (Q \wedge R), \qquad (P \vee Q) \vee R = P \vee (Q \vee R) \quad \text{(associative)}$$

$$P \wedge P = P, \qquad P \vee P = P \quad \text{(idempotent)}$$

$$P \wedge \neg P = F, \qquad P \vee \neg P = T \quad \text{(complement)}$$

$$P \wedge T = P, \qquad P \vee F = P \quad \text{(identity)}$$

$$P \wedge F = F, \qquad P \vee T = T \quad \text{(domination)}$$

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R), \qquad P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R) \quad \text{(distributive)}$$

$$\neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg(P \wedge Q) = \neg P \vee \neg Q \quad \text{(De Morgan's)}$$

Example 4.27 (Algebraic proof, Rosen – harder chain)**.** Show $\neg(P \implies Q) \equiv P \wedge \neg Q$.

$$\begin{aligned} \neg(P \implies Q) &= \neg(\neg P \vee Q) \\ &= \neg\neg P \wedge \neg Q && \text{(De Morgan's)} \\ &= P \wedge \neg Q && \text{(double negation).} \end{aligned}$$

4.16 Disjunctive Normal Form

Definition 4.28 (Literal)**.** A logical variable, unnegated or negated once (X or $\neg X$).

Definition 4.29 (DNF)**.** A disjunction of conjunctions of literals.

Note (Constructing DNF from a truth table). For each row where the expression is T: form the conjunction of literals matching that row (X if $X = T$ in that row, $\neg X$ if $X = F$); disjoin all such conjunctions.

Example 4.30 (Majority function of 3 variables, Rosen-style – harder)**.** $\text{Maj}(X, Y, Z)$ is true iff at least two of X, Y, Z are true.

X	Y	Z	Maj
F	F	F	F
F	F	T	F
F	T	F	F
F	T	T	T
T	F	F	F
T	F	T	T
T	T	F	T
T	T	T	T

DNF (four true rows):

$$\text{Maj}(X, Y, Z) = (\neg X \wedge Y \wedge Z) \vee (X \wedge \neg Y \wedge Z) \vee (X \wedge Y \wedge \neg Z) \vee (X \wedge Y \wedge Z).$$

Note (Cost of DNF). Every expression has an equivalent DNF, but the number of parts can be as large as 2^k for k variables – exponential blow-up. DNF makes *satisfiability* easy to certify (any one part gives a satisfying assignment) but gives no efficient way to certify a *tautology*.

4.17 Conjunctive Normal Form

Definition 4.31 (CNF)**.** A conjunction of *clauses*, each a disjunction of literals.

Note (Duality with DNF, via De Morgan's). If P is in CNF, $\neg P$ – negation of a conjunction of disjunctions – becomes, via repeated De Morgan's and double negation, a disjunction of conjunctions of literals: DNF. So CNF and DNF are dual representations.

Note (Converting truth table $\rightarrow$ CNF). Negate the output column; build DNF for $\neg P$; negate that expression; apply De Morgan's to push the negation inward into CNF form.

Note (3-SAT, Rosen/CS – harder, beyond CN). When every CNF clause has exactly 3 literals, satisfiability is the classical *3-SAT* problem – the canonical NP-complete problem (Cook–Levin theorem). This is why CNF, not DNF, is the standard input format for SAT solvers: checking DNF satisfiability is trivial, but CNF satisfiability is computationally hard in general, which is exactly the structure that makes CNF expressive enough to encode real combinatorial problems.

4.18 Satisfiability

Definition 4.32 (Satisfiable, unsatisfiable)**.** A compound proposition is *satisfiable* if some assignment of truth values to its variables makes it true (a *satisfying assignment*); otherwise it is *unsatisfiable*.

Note (Satisfiability and normal forms). In DNF (§4.16), satisfiability is trivial to check: any one disjunct with no variable appearing both negated and unnegated gives a satisfying assignment directly. In CNF (§4.17), determining satisfiability in general is the SAT problem – NP-complete (§4.17's note on 3-SAT).

Example 4.33 (The n-queens problem as satisfiability)**.** Place n queens on an $n \times n$ chessboard so that no two attack each other (same row, column, or diagonal). Encode as CNF:

Variables $x_{i,j}$ $(1 \le i, j \le n)$: true iff a queen occupies row i, column j.

- *At least one queen per row i*: $x_{i,1} \vee x_{i,2} \vee \cdots \vee x_{i,n}$.
- *At most one queen per row i*: $\neg x_{i,j} \vee \neg x_{i,k}$ for all $j \neq k$.
- *At most one queen per column j*: $\neg x_{i,j} \vee \neg x_{k,j}$ for all $i \neq k$.
- *At most one queen per diagonal*: $\neg x_{i,j} \vee \neg x_{k,l}$ whenever $(i,j) \neq (k,l)$ and $|i-k| = |j-l|$.

The conjunction of all these clauses is satisfiable iff an n-queens solution exists; a satisfying assignment directly gives the queen placements.

Example 4.34 (Sudoku as satisfiability)**.** A 9×9 grid, digits 1–9, each row/column/3×3 box containing every digit exactly once, some cells pre-filled.

Variables $x_{i,j,k}$ $(1 \le i, j, k \le 9)$: true iff cell (i,j) contains digit k.

- *Each cell has at least one digit*: $\bigvee_{k=1}^{9} x_{i,j,k}$, for each cell (i,j).
- *Each cell has at most one digit*: $\neg x_{i,j,k} \vee \neg x_{i,j,l}$ for $k \neq l$.
- *Each row has every digit*: for each row i, digit k: $\bigvee_{j=1}^{9} x_{i,j,k}$.
- *Each column has every digit*: analogous, indices swapped.
- *Each 3×3 box has every digit*: analogous, summing over the 9 cells of each box.
- *Given clues*: for each pre-filled cell $(i,j) = k_0$, the unit clause x_{i,j,k_0}.

A satisfying assignment is exactly a solved Sudoku grid consistent with the clues.

Note. Both encodings turn a combinatorial puzzle into a pure question about propositional satisfiability – this is exactly how modern SAT solvers are used to solve such puzzles in practice, rather than hand-coding a puzzle-specific search algorithm.

4.19 Representing logical statements

Note (Top-down / bottom-up modelling). **Top-down**: identify the top-level conjunction of required conditions. **Bottom-up**: identify the atomic Boolean variables (simplest possible true/false assertions) before worrying about their eventual truth values.

Example 4.35 (CNF from natural-language rules)**.** Variables: one Boolean per guest. Rules:

$$(\text{Harry} \vee \text{Ron} \vee \text{Hermione} \vee \text{Ginny})$$

$$\wedge\ (\neg\text{Hagrid} \vee \text{Norberta})$$

$$\wedge\ (\neg\text{Fred} \vee \text{George}) \wedge (\text{Fred} \vee \neg\text{George})$$

$$\wedge\ (\neg\text{Voldemort} \vee \neg\text{Bellatrix}) \wedge (\neg\text{Voldemort} \vee \neg\text{Dolores}) \wedge (\neg\text{Bellatrix} \vee \neg\text{Dolores}).$$

"At least one of" $\to$ disjunction; "A only if B" $\to$ implication $\to \neg A \vee B$; "none or both" $\to$ biconditional $\to$ two implications; "no more than one of a set" $\to$ conjunction of pairwise "not both" clauses.

4.20 Statements about how many variables are true

Note (At most k of n variables true). Equivalent to: every set of $k+1$ variables has at least one false, i.e. at least one negated literal true. Conjunction over all $\binom{n}{k+1}$ subsets of size $k+1$ of the disjunction of their negations.

Note (At least k of n variables true). Equivalent to: every set of $n-k+1$ variables has at least one true. Conjunction over all $\binom{n}{n-k+1}$ subsets of the disjunction of the (unnegated) literals.

Note (Exactly k). Conjunction of the "at least k" and "at most k" expressions.

Example 4.36. At most 2 of w, x, y, z true:

$$(\neg w \vee \neg x \vee \neg y) \wedge (\neg w \vee \neg x \vee \neg z) \wedge (\neg w \vee \neg y \vee \neg z) \wedge (\neg x \vee \neg y \vee \neg z).$$

4.21 Universal sets of operations

Definition 4.37 (Universal set of operations)**.** X is *universal* if every logical expression is equivalent to one using only operations from X.

Theorem 4.38. $\{\wedge, \vee, \neg\}$ *is universal (immediate from DNF/CNF, §4.16-4.17).*

Theorem 4.39. $\{\wedge, \neg\}$ *and* $\{\vee, \neg\}$ *are each universal.*

Proof. De Morgan's laws express $\vee$ in terms of $\wedge, \neg$ (and vice versa), so either can be dropped while keeping $\neg$. ■

Note ($\{\wedge, \vee\}$ is not universal). Without $\neg$, every expression built from $\wedge, \vee$ is *monotone*: increasing any input from F to T can never make the output go from T to F. Since $\neg P$ itself is not monotone, it cannot be expressed – so $\{\wedge, \vee\}$ fails to be universal.

Definition 4.40 (Sheffer stroke (NAND), Rosen – single-operator universal set)**.**

$$P \mid Q = \neg(P \wedge Q).$$

Theorem 4.41 ({|} alone is universal, Rosen)**.**

$$\neg P = P \mid P, \qquad P \wedge Q = (P \mid Q) \mid (P \mid Q), \qquad P \vee Q = (P \mid P) \mid (Q \mid Q).$$

Proof. Check each directly by truth table, or: $P \mid P = \neg(P \wedge P) = \neg P$; then $(P \mid Q) \mid (P \mid Q) = \neg(P \mid Q) = \neg\neg(P \wedge Q) = P \wedge Q$; and $(P \mid P) \mid (Q \mid Q) = \neg P \mid \neg Q = \neg(\neg P \wedge \neg Q) = P \vee Q$ (De Morgan's). Since $\{\wedge, \vee, \neg\}$ is universal and all three are expressible via $\mid$ alone, {|} is universal. ■

Note. This is why NAND gates alone suffice to build any digital circuit – a genuinely important fact in computer engineering, well beyond what CN's syllabus covers here.

5 Predicate Logic

5.1 Relations, predicates, and truth-valued functions

Definition 5.1 (Predicate)**.** A *predicate* is a relation (§2.13, §2.17), used with intent to do logic with it. A predicate with k arguments is *k-ary*.

Definition 5.2 (Predicate as a truth-valued function, Rosen)**.** A k-ary relation $R \subseteq A_1 \times \cdots \times A_k$ can equivalently be viewed as a function

$$R : A_1 \times \cdots \times A_k \to \{T, F\}, \qquad R(a_1, \ldots, a_k) = T \iff (a_1, \ldots, a_k) \in R.$$

Rosen also calls this a *propositional function*.

Note (Substitution yields a proposition). Giving specific values to every argument of a predicate produces a proposition (definite truth value).

Note (Equality is always available). $=$ is a predicate assumed available for *every* domain – without it, no domain of objects could even be meaningfully discussed.

Definition 5.3 (Property)**.** A unary predicate (*property*) $P(x)$ on domain D corresponds to the set $\{x \in D : P(x)\}$.

5.2 Variables and constants

Definition 5.4 (Variable)**.** A name for an object, without specifying it exactly, used to reason about an unknown or arbitrary member of its *domain*.

Note (Domain). *Domain* also call $domain of discourse$ or $universe of discourse$.

Definition 5.5 (Constant)**.** An object belonging to the domain of some variable – a specific, named member of that domain.

Note (Every variable has a domain). E.g. "let $x \in \mathbb{Z}$" declares x with domain $\mathbb{Z}$; a Boolean variable always has domain $\{T, F\}$.

Note (Math vs. programming variables, Rosen). A programming variable occupies memory and is restricted to finite/representable values; a mathematical variable has no memory association and its domain may be infinite.

5.3 Predicates and variables

Definition 5.6 (Free variable)**.** A variable in a predicate expression not (yet) assigned a value.

Note (Truth value requires all arguments fixed). An expression with ≥ 1 free variable is not a proposition (no truth value). Only when *every* free variable is assigned a value from its domain does the expression become a proposition.

Note (Domain matching is mandatory). Any value or variable placed into an argument of a predicate must belong to that argument's declared domain – mismatched domains make the expression ill-formed, not just false.

Example 5.7. Parent(Alan Turing, X) with X a free variable, domain $\mathbb{P}$ (people): not a proposition. Substituting $X =$"Sara Turing" gives the proposition Parent(Alan Turing, Sara Turing), true.

Note (Prefix vs. infix notation, Rosen). Newly defined predicates typically use prefix notation, $R(a, b)$; predicates for ordering/equality/equivalence conventionally use infix notation, $a < b$, $a = b$ (§2.5).

5.4 Arguments of predicates

Definition 5.8 (Term, recursive)**.** A *term* of domain D is one of:

- a constant from D;
- a variable with domain D;
- a function with codomain D, each of whose arguments is given a term of matching domain.

Note (Recursion bottoms out). Terms are built from constants/variables/functions to arbitrary finite depth (composition, nested function calls), but never infinitely – every term has a finite construction.

Example 5.9. Let a, b be variables with domain $\mathbb{R}$. The expression

$$a^2 + b^2$$

is a term of domain $\mathbb{R}$.

To see this, first consider a^2. The squaring function

$$\text{square} : \mathbb{R} \to \mathbb{R}$$

has codomain $\mathbb{R}$, and its argument a is a term of domain $\mathbb{R}$. Therefore,

$$a^2 = \text{square}(a)$$

is a term of domain $\mathbb{R}$.

Similarly, since b is a term of domain $\mathbb{R}$,

$$b^2 = \text{square}(b)$$

is also a term of domain $\mathbb{R}$.

Finally, consider the real addition function

$$+ : \mathbb{R} \times \mathbb{R} \to \mathbb{R}.$$

Its two arguments must each be terms of domain $\mathbb{R}$. Since both a^2 and b^2 are terms of domain $\mathbb{R}$, they can be given to the two arguments of the addition function. Hence,

$$a^2 + b^2$$

is a term of domain $\mathbb{R}$.

Example 5.10. Let $\mathbb{P}$ be the domain of people, and suppose that

$$\text{FatherOf} : \mathbb{P} \to \mathbb{P}$$

and

$$\text{MotherOf} : \mathbb{P} \to \mathbb{P}.$$

If Y is a variable with domain $\mathbb{P}$, then

$$\text{FatherOf}(Y)$$

is a term of domain $\mathbb{P}$. Consequently,

$$\text{MotherOf}(\text{FatherOf}(Y))$$

is also a term of domain $\mathbb{P}$.

Example 5.11 (Not terms)**.** $2 + 3+$ (malformed), $x{+}{+}$ (undefined operation here), $\log x$ (if log is not among the available functions), z (if z is neither a declared variable nor constant).

Note. Only terms whose domain matches an argument's declared domain may be plugged into that argument – of a function *or* a predicate; the same matching rule applies uniformly at every level of nesting.

5.5 Building logical expressions with predicates

Definition 5.12 (Predicate logic expression, recursive)**.**

- T, F are predicate logic expressions.
- A predicate applied to terms of matching domains is a predicate logic expression.
- If E, F are predicate logic expressions, so are (E), $\neg E$, $E \wedge F$, $E \vee F$, $E \implies F$, $E \iff F$.

Note (Free variables propagate). A predicate logic expression's free variables are the union of the free variables of its terms/sub-expressions – combining expressions never removes a free variable (only quantification, §5.6-5.9, does that).

Example 5.13. $(1 < i) \wedge (i < n)$ has free variables i, n. $\text{Parent}(X, Y) \iff \big(Y = \text{MotherOf}(X)\big) \vee \big(Y = \text{FatherOf}(X)\big)$ has free variables X, Y.

Note (Boolean algebra applies unchanged). Every law from Chapter 4 (De Morgan's, distributive, etc.) applies to predicate logic expressions exactly as to propositions – since fixing values for all free variables reduces any predicate expression to an ordinary proposition, and the laws hold for *every* such assignment.

5.6 Existential quantifier

Definition 5.14 (Existential quantifier)**.** For predicate $P(x)$ on domain D:

$$\exists x\, P(x)$$

is true iff $P(a)$ is true for *at least one* $a \in D$.

Note (Punctuation). $\exists x : P(x)$, $\exists x.\, P(x)$, $\exists x\, P(x)$ are all used; the colon/period is read "such that".

Definition 5.15 (Bound variable)**.** Once quantified, x is *bound*: no longer available to receive a specific value. A fully-quantified expression has no free variables and is a genuine proposition, with a definite truth value.

Note (Domain matters). $\exists W \in \mathbb{N} :\ W < 0$ is false; $\exists W \in \mathbb{Z} :\ W < 0$ is true – same predicate, different domain, different truth value.

Note (Analogy with disjunction – and where it breaks). $\exists x \in D,\ P(x)$ behaves like the disjunction $\bigvee_{a \in D} P(a)$ – true iff at least one disjunct is true. But this is only an analogy: disjunctions must be *finite*, while D may be infinite, so existential quantification genuinely extends what propositional logic alone can express.

Note (Vacuous quantification). If x does not appear in P, $\exists x\, P$ is equivalent to P itself (the quantifier is inert): e.g. $\exists w :\ z < 0$ is just $z < 0$.

Note. Quantifiers apply only to variables, never to constants: $\exists 5$ or $\exists \text{Annie}$ are ill-formed.

Example 5.16 (Rosen – harder: nested quantifiers and negation)**.** Let $P(x, y)$ mean "$x + y = 0$" on domain $\mathbb{R}$. Consider $\exists x\, \forall y\, P(x, y)$ vs. $\forall y\, \exists x\, P(x, y)$.

$\exists x\, \forall y\, P(x, y)$: one fixed x that works for *every* y – false (no single x satisfies $x + y = 0$ for all y).

$\forall y\, \exists x\, P(x, y)$: for each y, some x (possibly depending on y) works – true, take $x = -y$.

This shows quantifier *order* genuinely matters – covered fully in §5.10.

5.7 Restricting existentially quantified variables

Note (Restricting $\exists$ uses $\wedge$). To assert "some X satisfying $R(X)$ also satisfies $P(X)$":

$$\exists X :\ R(X) \wedge P(X).$$

Example 5.17 (Correct vs. a common wrong attempt)**.** "Some computer is human":

$$\text{correct: } \exists X :\ \text{computer}(X) \wedge \text{human}(X), \qquad \text{wrong: } \exists X :\ \text{computer}(X) \implies \text{human}(X).$$

The wrong version says "something is not a computer, or is human" – true of any non-computer, regardless of computers at all, so it fails to capture the intended restriction.

Note (Rosen's restricted-quantifier shorthand). Rosen writes $\exists X \in D\,[\dots]$ or $\exists X < 0\,[\dots]$ directly as shorthand for the conjunction above, folding the restriction into the quantifier notation itself rather than writing it as an explicit $\wedge$.

5.8 Universal quantifier

Definition 5.18 (Universal quantifier)**.** For predicate $P(x)$ on domain D:

$$\forall x\, P(x)$$

is true iff $P(a)$ is true for *every* $a \in D$.

Note. As with $\exists$: the quantified variable becomes bound; an irrelevant quantifier (variable not appearing in the body) can be dropped.

Note (Rosen – vacuous truth generalizes). $\forall x \in \emptyset,\ P(x)$ is vacuously true for *any* P, exactly generalizing $\emptyset \subseteq A$ (§1.6 Theorem 1) to arbitrary predicates, not just set membership.

5.9 Restricting universally quantified variables

Note (Restricting $\forall$ uses $\implies$). To assert "every X satisfying $R(X)$ also satisfies $P(X)$":

$$\forall X :\ R(X) \implies P(X).$$

Example 5.19 (Correct vs. a common wrong attempt)**.** "Every computer is human":

$$\text{correct: } \forall X :\ \text{computer}(X) \implies \text{human}(X), \qquad \text{wrong: } \forall X :\ \text{computer}(X) \wedge \text{human}(X).$$

The wrong version says "everything is both a computer and human" – forces non-computers to also be human computers, which was never intended.

Note (The mirror-image trap, worth memorizing). $\exists$ pairs naturally with $\wedge$; $\forall$ pairs naturally with $\implies$. Swapping these (using $\implies$ with $\exists$, or $\wedge$ with $\forall$) is one of the most common predicate-logic errors – always double check which connective a restricted quantifier needs.

Theorem 5.20 (De Morgan's for quantifiers, Rosen – preview of §5.13)**.**

$$\neg\forall x\, P(x) \ \equiv\ \exists x\, \neg P(x), \qquad \neg\exists x\, P(x) \ \equiv\ \forall x\, \neg P(x).$$

Note. Consequence: negating $\forall X : R(X) \implies P(X)$ gives $\exists X : R(X) \wedge \neg P(X)$ – exactly matching the $\exists/\wedge$ pairing above, since $\neg(R \implies P) \equiv R \wedge \neg P$ (§4.15).

5.10 Multiple quantifiers

Note (Same-type quantifiers commute).

$$\exists X\,\exists Y\,P = \exists Y\,\exists X\,P = \exists(X,Y)\,P, \qquad \forall X\,\forall Y\,P = \forall Y\,\forall X\,P = \forall(X,Y)\,P.$$

Note (Mixed quantifiers do *not* commute).

$$\exists X\,\forall Y\,P, \quad \forall X\,\exists Y\,P, \quad \exists Y\,\forall X\,P, \quad \forall Y\,\exists X\,P$$

are, in general, four genuinely different statements.

Example 5.21 (hasVisited(P,C): person P has visited country C)**.**

$\exists P\,\forall C$: someone has visited every country, $\qquad \forall C\,\exists P$: every country has been visited by someone.

The first is a strong claim about one exceptional traveler; the second is far weaker (each country just needs *some* visitor, possibly a different one each time).

Example 5.22 (Rosen – classic order-matters example)**.** Let $D(x,y)$ mean "x has dated y" on the domain of all people.

$$\exists x\,\forall y\,D(x,y) : \text{someone has dated everyone.}$$

$$\forall y\,\exists x\,D(x,y) : \text{everyone has been dated by someone.}$$

The first implies the second (that one exceptional dater witnesses every y), but not conversely – the second only needs, for each y, *some* x, possibly different for each y.

Theorem 5.23 ($\exists\forall \implies \forall\exists$, general fact, Rosen)**.**

$$\exists x\,\forall y\,P(x,y) \implies \forall y\,\exists x\,P(x,y).$$

The converse implication does not *hold in general.*

Proof. Suppose $\exists x\,\forall y\,P(x,y)$: fix such an x_0, so $P(x_0,y)$ holds for every y. Then for any y, taking $x = x_0$ witnesses $\exists x\,P(x,y)$ – so $\forall y\,\exists x\,P(x,y)$ holds. The dating example above (with D not symmetric enough to reverse) shows the converse can fail. ■

5.11 Predicate logic expressions

Definition 5.24 (Predicate logic expression, complete recursive grammar)**.**

- T, F are predicate logic expressions.
- Terms of matching domains plugged into a predicate give a predicate logic expression.
- If E, F are predicate logic expressions: (E), $\neg E$, $E \wedge F$, $E \vee F$, $E \implies F$, $E \iff F$.
- If E is a predicate logic expression and X is a free variable of E: $\exists X : E$ and $\forall X : E$ are predicate logic expressions, and X is *no longer free* in either.

Note (Free variables shrink under quantification). If V is the free-variable set of E, then $\exists X : E$ and $\forall X : E$ each have free-variable set $V \setminus \{X\}$.

5.12 Doing logic with quantifiers

Note (Instantiation and generalization).

$$\big(\forall X\, P(X)\big) \implies P(\text{obj}), \qquad P(\text{obj}) \implies \big(\exists X\, P(X)\big)$$

for any specific obj in X's domain.

Theorem 5.25 ($\forall$ distributes over $\wedge$; $\exists$ distributes over $\vee$)**.**

$$\forall X\,(P(X) \wedge Q(X)) \;\equiv\; (\forall X\, P(X)) \wedge (\forall X\, Q(X)),$$

$$\exists X\,(P(X) \vee Q(X)) \;\equiv\; (\exists X\, P(X)) \vee (\exists X\, Q(X)).$$

Note ($\forall$ over $\vee$, and $\exists$ over $\wedge$, only give one-way implications, Rosen).

$$(\forall X\, P(X)) \vee (\forall X\, Q(X)) \implies \forall X\,(P(X) \vee Q(X)),$$

$$\exists X\,(P(X) \wedge Q(X)) \implies (\exists X\, P(X)) \wedge (\exists X\, Q(X)),$$

but neither converse holds in general – e.g. every integer is even or odd ($\forall X$, $\text{even}(X) \vee \text{odd}(X)$), yet it is false that every integer is even, or every integer is odd.

Definition 5.26 (Scope of a quantified variable)**.** The sub-expression from the quantifier to the end of its innermost enclosing parentheses (or the end of the whole expression, if unparenthesized).

Example 5.27 (Two disjoint scopes, same variable name)**.** In $(\forall X\, P(X)) \wedge (\forall X\, Q(X))$, the two X's are entirely independent – like distinct local variables in a program that happen to share a name.

5.13 Duality between quantifiers

Theorem 5.28 (De Morgan's laws for quantifiers)**.**

$$\neg\forall X\, P(X) \;\equiv\; \exists X\, \neg P(X), \qquad \neg\exists X\, P(X) \;\equiv\; \forall X\, \neg P(X).$$

Example 5.29 ("Not all stars are visible" = "some star is invisible")**.**

$$\begin{aligned}\neg\forall X\,(\text{star}(X) \implies \text{visible}(X)) &= \exists X\, \neg(\text{star}(X) \implies \text{visible}(X)) \\ &= \exists X\, \neg(\neg\text{star}(X) \vee \text{visible}(X)) \\ &= \exists X\,(\text{star}(X) \wedge \neg\text{visible}(X)).\end{aligned}$$

Note. $\neg\forall Y\, \neg = \exists Y$, and $\neg\exists Y\, \neg = \forall Y$ (apply the laws twice, using double negation).

5.14 Some examples

Example 5.30 ("Infinitely many primes", in predicate logic)**.** With $\text{Prime}(n)$ on domain $\mathbb{N}$ and predicate $\leq$:

$$\forall b\, \exists n\ \text{Prime}(n) \wedge \neg(n \leq b).$$

Read: for every bound b, some prime exceeds it – correctly capturing "infinitely many primes" without needing an infinite conjunction.

Example 5.31 (Swapping the quantifiers gives a different, false, statement)**.**

$$\exists n\, \forall b\ \text{Prime}(n) \wedge \neg(n \leq b).$$

Why this is false – step by step. Distribute $\forall b$ over the conjunction (§5.12):

$$\forall b\ \text{Prime}(n) \wedge \neg(n \leq b)\ \equiv\ (\forall b\, \text{Prime}(n)) \wedge (\forall b\, \neg(n \leq b)).$$

Since $\text{Prime}(n)$ does not involve b, the quantifier is superfluous (§5.9):

$$\forall b\, \text{Prime}(n)\ \equiv\ \text{Prime}(n).$$

Substituting back:

$$\exists n \left(\text{Prime}(n) \wedge \forall b\, \neg(n \leq b)\right)\ \equiv\ \exists n \in \{\text{primes}\}\ \forall b\ n > b.$$

This asserts a single prime exceeding *every* number – false. So the two quantifier orderings genuinely differ in meaning, one true and one false. ■

Example 5.32 (Rosen – translating a nested-quantifier English sentence, harder)**.** "Every real number except 0 has a multiplicative inverse." With domain $= \mathbb{R}$:

$$\forall x \left(x \neq 0\ \implies\ \exists y\, (xy = 1)\right).$$

Note the restricted-$\forall$ pattern from §5.9 ($x \neq 0\ \implies\ \cdots$) nested with a plain $\exists y$ inside – y's existence is allowed to depend on x, which is essential: no single y works for every nonzero x.

Example 5.33 (Limit of a function, nested quantifiers – Rosen, requires calculus)**.** $\lim_{x \to a} f(x) = L$ means: for every real number $\epsilon > 0$ there exists a real number $\delta > 0$ such that $|f(x) - L| < \epsilon$ whenever $0 < |x - a| < \delta$.

In quantifiers, with ϵ, δ ranging over *all* reals and the positivity built into the body:

$$\forall \epsilon\, \exists \delta\, \forall x \left(0 < |x - a| < \delta\ \implies\ |f(x) - L| < \epsilon\right),$$

where the domain for ϵ, δ is understood to already be "positive reals," and for x is all reals.

Equivalently, using *restricted* quantifiers (§5.7, §5.9) to make the positivity explicit rather than implicit in the domain:

$$\forall \epsilon > 0\, \exists \delta > 0\, \forall x \left(0 < |x - a| < \delta\ \implies\ |f(x) - L| < \epsilon\right),$$

now with ϵ, δ ranging over *all* real numbers, and the restriction "> 0" carried by the quantifier notation itself instead of by the choice of domain.

Note (Order is essential here). The quantifier order $\forall \epsilon\, \exists \delta\, \forall x$ matters critically: δ is allowed to depend on ϵ (chosen *after* ϵ is fixed), but must work for *all* x satisfying the closeness condition. Swapping to $\exists \delta\, \forall \epsilon$ would wrongly demand a single δ working for every ϵ simultaneously – a much stronger, generally false, claim.

Example 5.34 ("Limit does not exist," via negation of nested quantifiers – Rosen, requires calculus)**.** $\lim_{x\to a} f(x)$ does not exist means: for every real number L, $\lim_{x\to a} f(x) \neq L$.

Using the restricted-quantifier form above, $\lim_{x\to a} f(x) \neq L$ is the negation of

$$\forall \epsilon > 0\, \exists \delta > 0\, \forall x \Big(0 < |x-a| < \delta \implies |f(x) - L| < \epsilon\Big),$$

i.e.

$$\neg\forall \epsilon > 0\, \exists \delta > 0\, \forall x \Big(0 < |x-a| < \delta \implies |f(x) - L| < \epsilon\Big).$$

Push the negation inward one quantifier at a time, using §5.13's De Morgan's laws for quantifiers at each step:

$$\begin{aligned}
&\neg\forall \epsilon > 0\, \exists \delta > 0\, \forall x\, (0 < |x-a| < \delta \implies |f(x) - L| < \epsilon) \\
\equiv\ &\exists \epsilon > 0\, \neg\exists \delta > 0\, \forall x\, (0 < |x-a| < \delta \implies |f(x) - L| < \epsilon) \\
\equiv\ &\exists \epsilon > 0\, \forall \delta > 0\, \neg\forall x\, (0 < |x-a| < \delta \implies |f(x) - L| < \epsilon) \\
\equiv\ &\exists \epsilon > 0\, \forall \delta > 0\, \exists x\, \neg\big(0 < |x-a| < \delta \implies |f(x) - L| < \epsilon\big) \\
\equiv\ &\exists \epsilon > 0\, \forall \delta > 0\, \exists x \Big(0 < |x-a| < \delta \ \wedge\ |f(x) - L| \geq \epsilon\Big).
\end{aligned}$$

The last step uses $\neg(P \implies Q) \equiv P \wedge \neg Q$ (§4.15), applied with $P = (0 < |x-a| < \delta)$, $Q = (|f(x) - L| < \epsilon)$, and $\neg(|f(x) - L| < \epsilon) \equiv |f(x) - L| \geq \epsilon$.

Since "the limit does not exist" means $\forall L$, $\lim_{x\to a} f(x) \neq L$, prepend $\forall L$:

$$\forall L\, \exists \epsilon > 0\, \forall \delta > 0\, \exists x \Big(0 < |x-a| < \delta \ \wedge\ |f(x) - L| \geq \epsilon\Big).$$

Read aloud: "for every candidate limit value L, there is some $\epsilon > 0$ such that, no matter how small a $\delta > 0$ is chosen, some x within δ of a still has $f(x)$ at least ϵ away from L" – exactly capturing that no L can serve as the limit.

Note. Each of the four inner negation steps flips $\forall \leftrightarrow \exists$ and pushes $\neg$ past exactly one quantifier, mirroring the definitions in §5.13 one at a time; only the final step differs, converting the negated implication via ordinary propositional logic rather than a quantifier law.

Example 5.35 (Negating a triple-nested quantifier statement – Rosen, harder)**.** "There is no woman who has flown on every airline in the world." Let $P(w, f)$: "woman w has taken flight f"; $Q(f, a)$: "flight f is on airline a." "Some woman has flown on every airline" is

$$\exists w\, \forall a\, \exists f\, \big(P(w, f) \wedge Q(f, a)\big),$$

so the negation is $\neg\exists w\, \forall a\, \exists f\, (P(w, f) \wedge Q(f, a))$. Pushing the negation inward, one quantifier at a time:

$$\begin{aligned}
&\neg\exists w\, \forall a\, \exists f\, (P(w, f) \wedge Q(f, a)) \\
\equiv\ &\forall w\, \neg\forall a\, \exists f\, (P(w, f) \wedge Q(f, a)) \\
\equiv\ &\forall w\, \exists a\, \neg\exists f\, (P(w, f) \wedge Q(f, a)) \\
\equiv\ &\forall w\, \exists a\, \forall f\, \neg(P(w, f) \wedge Q(f, a)) \\
\equiv\ &\forall w\, \exists a\, \forall f\, \big(\neg P(w, f) \vee \neg Q(f, a)\big).
\end{aligned}$$

Reading it back: "for every woman, there is some airline such that, for every flight, either she hasn't taken that flight or that flight isn't on that airline."

Note. Three nested quantifiers, three applications of the same rule from §5.13 – the pattern doesn't change with depth, only the bookkeeping does.

5.15 Summary of rules for logic with quantifiers

Note (Full rule list).

$$\exists x \in A\, P(x) \equiv \exists x\, (x \in A \wedge P(x)) \qquad (\S 5.7)$$

$$\forall x \in A\, P(x) \equiv \forall x\, (x \in A \implies P(x)) \qquad (\S 5.9)$$

$$\exists x\, R \equiv R, \quad \forall x\, R \equiv R \qquad (x \text{ not free in } R;\ \S 5.7, 5.9)$$

$$\exists x \exists y\, P(x,y) \equiv \exists y \exists x\, P(x,y) \qquad (\S 5.10)$$

$$\forall x \forall y\, P(x,y) \equiv \forall y \forall x\, P(x,y) \qquad (\S 5.10)$$

$$\forall x\, P(x) \implies P(\text{obj}), \quad P(\text{obj}) \implies \exists x\, P(x) \qquad (\S 5.12)$$

$$\exists x\, (P(x) \vee Q(x)) \equiv (\exists x\, P(x)) \vee (\exists x\, Q(x)) \qquad (\S 5.12)$$

$$\forall x\, (P(x) \wedge Q(x)) \equiv (\forall x\, P(x)) \wedge (\forall x\, Q(x)) \qquad (\S 5.12)$$

$$\neg \exists x\, P(x) \equiv \forall x\, \neg P(x), \quad \neg \forall x\, P(x) \equiv \exists x\, \neg P(x) \qquad (\S 5.13)$$

5.16 Rules of inference (Rosen §1.6 – extension, beyond CN syllabus)

Definition 5.36 (Rule of inference)**.** A *rule of inference* is an argument form: premises above a line, conclusion below, valid because the conjunction of the premises implying the conclusion is a tautology.

Note (Basic rules for propositional logic).

$$\text{Modus ponens: } \frac{P,\ P \implies Q}{\therefore Q} \qquad \text{Modus tollens: } \frac{\neg Q,\ P \implies Q}{\therefore \neg P}$$

$$\text{Hypothetical syllogism: } \frac{P \implies Q,\ Q \implies R}{\therefore P \implies R} \qquad \text{Disjunctive syllogism: } \frac{P \vee Q,\ \neg P}{\therefore Q}$$

$$\text{Addition: } \frac{P}{\therefore P \vee Q} \qquad \text{Simplification: } \frac{P \wedge Q}{\therefore P} \qquad \text{Conjunction: } \frac{P,\ Q}{\therefore P \wedge Q}$$

$$\text{Resolution: } \frac{P \vee Q,\ \neg P \vee R}{\therefore Q \vee R}$$

Theorem 5.37. *Each rule above is valid: the conjunction of its premises implies its conclusion is a tautology.*

Verification for modus tollens. Suppose $\neg Q$ and $P \implies Q$ both hold. If P were true, $P \implies Q$ would force Q true, contradicting $\neg Q$. So P is false, i.e. $\neg P$ holds. (The others verify similarly, by truth table or direct argument – §4.15's laws are exactly what's needed.) ■

Note (Rules of inference for quantified statements).

$$\text{Universal instantiation: } \frac{\forall x\, P(x)}{\therefore P(c)} \ (c \text{ any specific element of the domain})$$

$$\text{Universal generalization: } \frac{P(c) \text{ for arbitrary, generic } c}{\therefore \forall x\, P(x)}$$

$$\text{Existential instantiation: } \frac{\exists x\, P(x)}{\therefore P(c)} \ (c \text{ some element, not otherwise specified})$$

$$\text{Existential generalization: } \frac{P(c) \text{ for a specific } c}{\therefore \exists x\, P(x)}$$

Note (These match CN's own principles). Universal instantiation and existential generalization are exactly the two implications from §5.12; universal generalization is exactly the "let x be arbitrary, generic" technique from §3.3/§3.4 for proving universal statements.

Example 5.38 (Chained formal proof – harder, Rosen-style)**.** Premises: (1) $\forall x\,(\text{Student}(x) \implies \text{Studies}(x))$; (2) $\forall x\,(\text{Studies}(x) \implies \text{Passes}(x))$; (3) $\neg\text{Passes}(\text{Sam})$; (4) $\text{Student}(\text{Sam})$.

Conclusion to derive: a contradiction (showing the premises are jointly unsatisfiable).

(5) $\text{Student}(\text{Sam}) \implies \text{Studies}(\text{Sam})$	(universal instantiation on (1), $c = \text{Sam}$)
(6) $\text{Studies}(\text{Sam}) \implies \text{Passes}(\text{Sam})$	(universal instantiation on (2), $c = \text{Sam}$)
(7) $\text{Student}(\text{Sam}) \implies \text{Passes}(\text{Sam})$	(hypothetical syllogism, (5)+(6))
(8) $\text{Passes}(\text{Sam})$	(modus ponens, (4)+(7))
(9) $\text{Passes}(\text{Sam}) \wedge \neg\text{Passes}(\text{Sam})$	(conjunction, (8)+(3))

Line (9) is a contradiction, so the four premises cannot all hold simultaneously.

6 Sequences & Series

6.1 Definitions and notation

Definition 6.1 (Sequence)**.** A *sequence* is a function whose domain is $\mathbb{N}$ (an *infinite sequence*) or $\{1, \dots, n\}$ for some n (a *finite sequence*, equivalently an ordered n-tuple, §1.14).

Definition 6.2 (Term, n-th term notation)**.** For a sequence f over a set A (i.e. $f : \mathbb{N} \to A$ or similar), $f(n)$ (also written f_n) is the *n-th term*. The *terms* of f are the members of its image (§2.1.2), listed in the order given by the domain.

Definition 6.3 (Length)**.** The *length* of a finite sequence is the size of its domain – its number of terms.

Note (Rosen's phrasing). Rosen defines a sequence identically: a function from a subset of $\mathbb{Z}$ (typically $\{0, 1, 2, \dots\}$ or $\{1, 2, 3, \dots\}$) to a set S; a_n denotes the image of n, called a *term*.

Note (Ways to specify a sequence).

- List all terms explicitly (only practical if short).
- A formula for the n-th term: $(n^2 : n \in \mathbb{N})$ or $(n^2)_{n=1}^{\infty}$, matching set-builder notation (§1.2) but with parentheses since order matters.
- A recurrence relation (§6.2): each term defined using previous term(s).

A sequence is *not* well-defined merely by listing a few terms and trusting a "pattern" – infinitely many different formulas can match any finite initial segment (see §6.2's exercise on this danger, and the counterexample below).

Example 6.4 (A pattern-guessing trap, Rosen – harder)**.** The sequence $1, 2, 4, 8, 16, \dots$ (five terms shown) matches $a_n = 2^{n-1}$ – but it also matches

$$a_n = \binom{n-1}{0} + \binom{n-1}{1} + \binom{n-1}{2} + \binom{n-1}{3} + \binom{n-1}{4},$$

which equals $1, 2, 4, 8, 16$ for $n = 1, \dots, 5$ but gives 31, not 32, at $n = 6$ (since the $\binom{n-1}{5}$ term, absent from this formula, would otherwise contribute). A finite list of terms never uniquely determines a sequence.

Note (Number sequences). Sequences over $\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$ (*number sequences*) are especially important – not only for direct computation, but for measuring properties of computation itself: running time, memory use, input/output size, and so on.

6.2 Recursive definitions of sequences

Definition 6.5 (Recurrence relation)**.** A *recurrence relation* for (a_n) expresses a_n in terms of one or more earlier terms (a_{n-1}, or several), together with enough *base cases* (initial terms given explicitly) to make every term uniquely determined.

Note (General shape of a recursive definition, Rosen §5.3).

- **Base case**: explicitly define finitely many of the simplest objects.
- **Recursive case**: define a general object in terms of simpler ones already in the family.

This is the same two-part shape as recursively-defined sets/functions (§5.3 generalizes it beyond sequences – see the note below).

Example 6.6.

$$f_1 = 1,\ f_n = f_{n-1} + 2 \quad \Rightarrow \quad 1, 3, 5, 7, \ldots \text{ (odd numbers);}$$

$f_1 = 0,\ f_n = f_{n-1}+2 \quad \Rightarrow \quad 0, 2, 4, 6, \ldots$ (even numbers) – same rule, different base case, different sequence;

$$f_1 = 1,\ f_n = nf_{n-1} \quad \Rightarrow \quad 1, 2, 6, 24, 120, \ldots \text{ (factorials, i.e. } n!).$$

Note (Multiple base cases). $f_n = 4f_{n-2}$ (depending on the term *two* back) needs *two* base cases, f_1, f_2, to determine the whole sequence – one base case alone only fixes every other term. E.g. $f_1 = 2, f_2 = 0,\ f_n = 4f_{n-2}$ gives $2, 0, 8, 0, 32, 0, \ldots$

Example 6.7 (Structural induction preview, Rosen §5.3 – harder)**.** Recursively-defined *sets* follow the same base/recursive pattern: e.g. the set S of "balanced parenthesis strings" is defined by (i) $\varepsilon \in S$ (base case); (ii) if $w \in S$, then $(w) \in S$ and, if $u, v \in S$, then $uv \in S$ (recursive case). Proving a property P holds for every $w \in S$ then uses *structural induction*: prove P for the base case, and prove that the recursive case preserves P (assuming it holds for the smaller pieces used to build w) – the direct set-theoretic analogue of induction on sequences.

6.3 Arithmetic sequences

Definition 6.8 (Arithmetic sequence)**.** A sequence with constant difference d between consecutive terms: $f_1 = a,\ f_n = f_{n-1} + d$. The n-th term is $a + (n-1)d$.

Note (Rosen's phrasing). Rosen calls this an *arithmetic progression*: $a_n = a + dn$ (indexing from $n = 0$) or equivalently $a_n = a_{n-1} + d$ – same sequence, differing only in whether the first term is indexed 0 or 1.

6.4 Geometric sequences

Definition 6.9 (Geometric sequence)**.** A sequence with constant ratio r between consecutive terms: $f_1 = a,\ f_n = f_{n-1} \cdot r$. The n-th term is ar^{n-1}.

Note (Rosen's phrasing). Rosen calls this a *geometric progression*: $a_n = ar^n$ (indexing from 0) or $a_n = a_{n-1} \cdot r$.

6.5 Harmonic sequences

Definition 6.10 (Harmonic sequence)**.** (a_n) is *harmonic* iff $(1/a_n)$ is arithmetic.

Note. $(1/n : n \in \mathbb{N})$ is the key example – harmonic since $\mathbb{N}$ itself is arithmetic. See §6.15 for its properties.

6.6 From recursive definitions to formulas

Note (Explore – conjecture – prove).

1. **Explore**: compute the first several terms.
2. **Conjecture**: look for a pattern; propose a closed-form formula.
3. **Prove**: verify the formula for all n, typically by induction.

Example 6.11.

$$f_1 = 1, \quad f_n = 2f_{n-1} + 1.$$

Explore: $f_1 = 1,\ f_2 = 3,\ f_3 = 7,\ f_4 = 15.$
Conjecture: each term is one less than a power of 2: $f_n = 2^n - 1$.
Prove (induction on n):
Base case ($n = 1$): $2^1 - 1 = 1 = f_1$.
Inductive step: assume $f_k = 2^k - 1$ for some $k \geq 1$. Then

$$f_{k+1} = 2f_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 2 + 1 = 2^{k+1} - 1.$$

By induction, $f_n = 2^n - 1$ for all $n \geq 1$.

Definition 6.12 (Linear homogeneous recurrence with constant coefficients, Rosen §8.2 – systematic method)**.**

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}, \qquad c_k \neq 0,$$

with k initial conditions $a_0, \ldots, a_{k-1}$ given. Its *characteristic equation* is

$$x^k - c_1 x^{k-1} - c_2 x^{k-2} - \cdots - c_k = 0.$$

Theorem 6.13 (Distinct roots case, Rosen)**.** *If the characteristic equation has k* distinct *roots $r_1, \ldots, r_k$, then every solution has the form*

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \cdots + \alpha_k r_k^n$$

for constants $\alpha_1, \ldots, \alpha_k$ determined by the initial conditions.

Note (Why this is really linear algebra). The set of all sequences satisfying this recurrence forms a vector space: if (b_n) and (c_n) both satisfy it, so does any combination $\beta b_n + \gamma c_n$ (the recurrence is *linear*). This space has dimension k – a solution is pinned down entirely by its k initial values $a_0, \ldots, a_{k-1}$.

Each geometric sequence r_i^n is itself a solution (plugging it into the recurrence and using $r_i^k = c_1 r_i^{k-1} + \cdots + c_k$ confirms it). When the r_i are distinct, these k solutions are *linearly independent* – this is exactly a Vandermonde-determinant fact from linear algebra: distinct roots $\Rightarrow$ nonzero Vandermonde determinant $\Rightarrow$ independence. k independent vectors in a k-dimensional space form a *basis*, so every solution is a unique linear combination of them – which is exactly the formula above, with $\alpha_1, \ldots, \alpha_k$ solved for using whatever initial conditions are given.

Example 6.14 (Solving a second-order recurrence – harder, worked example)**.**

$$a_n = 5a_{n-1} - 6a_{n-2}, \qquad a_0 = 1,\ a_1 = 4.$$

Characteristic equation: $x^2 - 5x + 6 = 0$, i.e. $(x-2)(x-3) = 0$, roots $r_1 = 2,\ r_2 = 3$.
General solution: $a_n = \alpha_1 2^n + \alpha_2 3^n$.
Solve for α_1, α_2 using initial conditions:

$$a_0 = 1: \ \alpha_1 + \alpha_2 = 1, \qquad a_1 = 4: \ 2\alpha_1 + 3\alpha_2 = 4.$$

From the first equation, $\alpha_1 = 1 - \alpha_2$; substituting: $2(1 - \alpha_2) + 3\alpha_2 = 4 \Rightarrow \alpha_2 = 2,\ \alpha_1 = -1$.
Closed form:

$$a_n = -2^n + 2 \cdot 3^n.$$

Note (Why this beats guess-and-check). The explore-conjecture-prove method above needs a pattern to be *visible* in the first few terms – it can fail entirely on recurrences with less obvious closed forms (this one, $a_n = 5a_{n-1} - 6a_{n-2}$, produces $1, 4, 14, 46, \ldots$, no obvious pattern by inspection). The characteristic-equation method is systematic: it always works, whenever the roots can be found. This is exactly the machinery behind the exact Fibonacci formula in §6.7.4.

Definition 6.15 (Recursively defined set, Rosen §5.3)**.** A set S is defined recursively by:

- **Basis step**: specify an initial collection of elements in S.
- **Recursive step**: rules that construct new elements of S from existing ones.
- (Implicitly) S contains *only* elements obtainable this way.

Example 6.16 (Balanced parenthesis strings)**.** Define B recursively: (i) $\varepsilon \in B$ (empty string); (ii) if $w \in B$, then $(w) \in B$; (iii) if $u, v \in B$, then $uv \in B$.

Definition 6.17 (Recursively defined function, Rosen §5.3)**.** A function f on a recursively defined set S is defined by: giving f's value(s) on the basis elements, and giving a rule for f's value on a constructed element in terms of f's value(s) on the piece(s) it was built from.

Example 6.18. $n!$ on $\mathbb{N}$ (basis: $0! = 1$; recursive: $n! = n \cdot (n-1)!$) fits this exact pattern – a recursively defined function on the recursively defined set $\mathbb{N}$.

Theorem 6.19 (Structural induction, Rosen §5.3)**.** *To prove property P holds for every element of a recursively defined set S:*

- ***Basis step**: prove P holds for every basis element.*
- ***Recursive step**: prove that, if P holds for the element(s) a recursive rule builds from, P holds for the newly constructed element too.*

Example 6.20 (Structural induction on balanced parentheses – harder)**.** Prove: every $w \in B$ has an equal number of '(' and ')'.

Proof. **Basis step**: ε has 0 of each – equal.

Recursive step, rule (ii): suppose $w \in B$ has equal counts, say k each. Then (w) has $k + 1$ of each – still equal.

Recursive step, rule (iii): suppose $u, v \in B$ have equal counts k_u, k_v respectively. Then uv has $k_u + k_v$ of each – still equal.

By structural induction, every $w \in B$ has equal counts of '(' and ')'. ■

Note (Structural induction ≡ ordinary induction, in disguise). Structural induction is not a genuinely new principle – it is ordinary (or strong) induction on the "construction depth" of an element (how many recursive steps were used to build it), applied to the recursively defined set instead of to $\mathbb{N}$ directly.

Example 6.21 (Rabbit population, Rosen §8.1 – the origin of Fibonacci)**.** A pair of rabbits produces one new pair each month starting from their second month of life; no rabbit dies. Let f_n = number of pairs after n months, starting from one newborn pair. Then $f_1 = 1, f_2 = 1$, and for $n \geq 3$: pairs present at month n = pairs already alive at month $n-1$, plus new pairs born (one per pair that was already alive and mature at month $n-2$):

$$f_n = f_{n-1} + f_{n-2},$$

exactly the Fibonacci recurrence (§6.7).

Example 6.22 (Tower of Hanoi, Rosen §8.1)**.** n disks, decreasing size, on peg A; move all to peg C (via peg B), one disk at a time, never placing a larger disk on a smaller one. Let H_n = minimum number of moves.

Note (Recurrence). Move the top $n-1$ disks $A \to B$ (H_{n-1} moves), move the largest disk $A \to C$ (1 move), move the $n-1$ disks $B \to C$ (H_{n-1} moves):

$$H_1 = 1, \qquad H_n = 2H_{n-1} + 1.$$

Theorem 6.23. $H_n = 2^n - 1$.

Proof. Exactly the recurrence and proof of §6.6's worked example (same recurrence shape, $a = 1, b = 1$ shifted), by induction: base case $H_1 = 1 = 2^1 - 1$; inductive step $H_{k+1} = 2H_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$. ■

Definition 6.24 (Linear nonhomogeneous recurrence, Rosen §8.2)**.**

$$a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k} + F(n), \qquad F(n) \not\equiv 0.$$

The associated *homogeneous* recurrence drops $F(n)$.

Theorem 6.25 (General solution shape)**.** *Every solution has the form*

$$a_n = a_n^{(p)} + a_n^{(h)},$$

where $a_n^{(p)}$ is any one *particular solution to the nonhomogeneous recurrence, and $a_n^{(h)}$ is the general solution of the associated homogeneous recurrence.*

Proof. If $a_n^{(p)}$ solves the nonhomogeneous recurrence and (b_n) is *any* solution to it, then $b_n - a_n^{(p)}$ satisfies the *homogeneous* recurrence (subtracting the two nonhomogeneous equations cancels $F(n)$). So $b_n - a_n^{(p)} = a_n^{(h)}$ for some homogeneous solution, i.e. $b_n = a_n^{(p)} + a_n^{(h)}$. ■

Note (Finding a particular solution – trial forms). When $F(n)$ is a polynomial times s^n, try $a_n^{(p)}$ of the same shape (polynomial of the same degree times s^n) – unless s is itself a root of the characteristic equation, in which case multiply by n (or n^m for a root of multiplicity m).

Example 6.26 (Worked nonhomogeneous example – harder)**.**

$$a_n = 3a_{n-1} + 2^n, \qquad a_0 = 1.$$

Homogeneous part: characteristic equation $x - 3 = 0$, so $a_n^{(h)} = \alpha \cdot 3^n$.

Particular solution: try $a_n^{(p)} = A \cdot 2^n$ ($s = 2$ is not a root of $x - 3 = 0$). Substituting:

$$A \cdot 2^n = 3A \cdot 2^{n-1} + 2^n \;\Rightarrow\; 2A = 3A + 2 \;\Rightarrow\; A = -2.$$

So $a_n^{(p)} = -2 \cdot 2^n = -2^{n+1}$.

General solution: $a_n = \alpha \cdot 3^n - 2^{n+1}$.

Match $a_0 = 1$: $\alpha - 2 = 1 \Rightarrow \alpha = 3$.

Closed form: $a_n = 3^{n+1} - 2^{n+1}$.

Theorem 6.27 (Repeated roots case, Rosen – harder)**.** *If characteristic root r has multiplicity m, its contribution to the general solution is*

$$(\alpha_0 + \alpha_1 n + \alpha_2 n^2 + \cdots + \alpha_{m-1} n^{m-1})\, r^n$$

instead of a single term αr^n.

Example 6.28. $a_n = 4a_{n-1} - 4a_{n-2}$: characteristic equation $x^2 - 4x + 4 = (x-2)^2 = 0$, double root $r = 2$. General solution: $a_n = (\alpha_0 + \alpha_1 n)2^n$.

Definition 6.29 (Divide-and-conquer recurrence, Rosen §8.3)**.** An algorithm that splits a size-n problem into a subproblems of size n/b, spending $g(n)$ extra work to combine results, has running time

$$f(n) = a\, f(n/b) + g(n).$$

Example 6.30 (Merge sort)**.** Split into $a = 2$ halves ($b = 2$), merge in $g(n) = n$ (linear) time:

$$f(n) = 2f(n/2) + n.$$

Theorem 6.31 (Master theorem, special case $g(n) = cn^d$, Rosen)**.** *For $f(n) = af(n/b) + cn^d$ ($a \geq 1,\ b > 1,\ c, d \geq 0$ real), when n is a power of b:*

$$f(n) = \Theta\left(\begin{cases} n^d, & a < b^d, \\ n^d \log n, & a = b^d, \\ n^{\log_b a}, & a > b^d. \end{cases}\right)$$

Proof sketch, unrolling the recurrence. Let $n = b^k$. Unroll once: $f(n) = af(n/b) + cn^d$. Unroll again: $f(n) = a^2 f(n/b^2) + ac(n/b)^d + cn^d$. After k unrollings:

$$f(n) = a^k f(1) + cn^d \sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i.$$

The geometric sum $\sum (a/b^d)^i$ behaves differently depending on whether the ratio a/b^d is < 1, $= 1$, or > 1 – giving the three cases (via the finite geometric series formula, §6.13, and $a^k = a^{\log_b n} = n^{\log_b a}$). ■

Example 6.32 (Applying it to merge sort)**.** $a = 2,\ b = 2,\ d = 1$: $a = b^d$ ($2 = 2^1$), so $f(n) = \Theta(n \log n)$.

Definition 6.33 (Generating function, Rosen §8.4)**.** For sequence $(a_n)_{n=0}^{\infty}$, its *generating function* is the formal power series

$$G(x) = \sum_{n=0}^{\infty} a_n x^n.$$

Example 6.34. $(1, 1, 1, \dots)$: $G(x) = \sum x^n = \dfrac{1}{1-x}$ (geometric series, §6.14). $\binom{n}{k}$ for fixed k, varying n: related to $(1-x)^{-(k+1)}$.

Note (Solving recurrences via generating functions – the technique).

1. Multiply the recurrence $a_n = c_1 a_{n-1} + \cdots$ by x^n and sum over n.
2. Recognize each resulting sum as a shifted/scaled copy of $G(x) = \sum a_n x^n$.
3. Solve the resulting algebraic equation for $G(x)$.
4. Expand $G(x)$ back into a power series (e.g. partial fractions) to read off a_n as the coefficient of x^n.

Example 6.35 (Fibonacci via generating functions – harder, alternate route to §6.7.4's formula)**.** $f_0 = 0, f_1 = 1,\ f_n = f_{n-1} + f_{n-2}$ for $n \geq 2$. Let $G(x) = \sum_{n \geq 0} f_n x^n$.

$$G(x) = f_0 + f_1 x + \sum_{n \geq 2} f_n x^n = x + \sum_{n \geq 2} (f_{n-1} + f_{n-2}) x^n = x + x \sum_{n \geq 2} f_{n-1} x^{n-1} + x^2 \sum_{n \geq 2} f_{n-2} x^{n-2}.$$

The two sums are $G(x)$ itself (shifted indices, re-summed from $n = 1$ and $n = 0$ respectively, matching since $f_0 = 0$ doesn't affect the first):

$$G(x) = x + xG(x) + x^2 G(x) \ \Rightarrow\ G(x)(1 - x - x^2) = x \ \Rightarrow\ G(x) = \frac{x}{1 - x - x^2}.$$

Partial-fraction expanding $\frac{x}{1-x-x^2}$ (using the roots of $1 - x - x^2 = 0$, related to the golden ratio) and reading off the coefficient of x^n reproduces exactly the closed form from §6.7.4.

6.7 The Fibonacci sequence

Definition 6.36 (Fibonacci sequence)**.**

$$f_1 = 1, \quad f_2 = 1, \quad f_n = f_{n-1} + f_{n-2}\ (n \geq 3).$$

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

Note (Explore-conjecture-prove, applied here). No formula is obvious from the recurrence alone. The ratio f_{n+1}/f_n appears to settle down (e.g. $f_{10}/f_9 = 55/34 \approx 1.617$), suggesting exponential growth at some rate – explored further below.

6.7.1 Upper bounds

Theorem 6.37. *$f_n \leq 2^n$ for all $n \geq 1$.*

Proof. Strong induction, base cases $n = 1, 2$ ($f_1 = 1 \leq 2,\ f_2 = 1 \leq 4$); inductive step ($k \geq 2$): $f_{k+1} = f_k + f_{k-1} \leq 2^k + 2^{k-1} < 2^{k+1}$. ∎

Note (Tightening the bound). The proof only used $2^k + 2^{k-1} \leq 2^{k+1}$, i.e. (dividing by 2^{k-1}) $2+1 \leq 2^2$. Any r with $r+1 \leq r^2$ would work the same way in place of 2 – so the smallest such r gives the tightest bound of this form.

Theorem 6.38 (Sharpest bound of this form)**.** *Let* $r_1 = \dfrac{1+\sqrt{5}}{2}$ *(the positive root of* $r^2 - r - 1 = 0$*). Then* $f_n \leq r_1^n$ *for all* $n \geq 1$.

Proof. Same induction, using $r_1^k + r_1^{k-1} = r_1^{k+1}$ exactly (since r_1 satisfies $r_1 + 1 = r_1^2$, multiply through by r_1^{k-1}), and base cases $f_1, f_2 = 1 < r_1, r_1^2$. ■

6.7.2 Lower bounds

Theorem 6.39. $f_n \geq r_1^{n-2}$ *for all* $n \geq 1$*, where* r_1 *is as above.*

Proof sketch. Same strategy in reverse: seek the largest $r \leq r_1$ with $r + 1 \geq r^2$; the quadratic $r^2 - r - 1 = 0$'s roots are exactly the boundary, so r_1 itself (the larger root) works with equality in the recurrence step, giving the tightest such lower bound after matching the base cases $n = 1, 2$. ■

6.7.3 Asymptotic behaviour

Note (Squeezing f_n between bounds). Combining both bounds:

$$r_1^{n-2} \leq f_n \leq r_1^n \quad \Longrightarrow \quad r_1^{-2} \leq \frac{f_n}{r_1^n} \leq 1.$$

So f_n / r_1^n is trapped between two constants, independent of n – meaning f_n grows *exactly* like r_1^n up to a bounded factor. r_1 is the *golden ratio*, usually written φ.

6.7.4 An exact formula

Theorem 6.40 (Binet's formula)**.** *For all* $n \geq 1$*:*

$$f_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}, \qquad \varphi = \frac{1+\sqrt{5}}{2}, \quad \psi = \frac{1-\sqrt{5}}{2}.$$

Proof. The characteristic equation of $f_n = f_{n-1} + f_{n-2}$ is $x^2 - x - 1 = 0$, with roots exactly φ, ψ (the two roots identified in §6.7.1–6.7.2). Since these are distinct, the general solution (§6.6's Distinct roots theorem) is

$$f_n = \alpha\varphi^n + \beta\psi^n.$$

Match initial conditions $f_1 = f_2 = 1$:

$$\alpha\varphi + \beta\psi = 1, \qquad \alpha\varphi^2 + \beta\psi^2 = 1.$$

Solving this 2×2 linear system (using $\varphi - \psi = \sqrt{5}$) gives $\alpha = \dfrac{1}{\sqrt{5}}$, $\beta = -\dfrac{1}{\sqrt{5}}$, yielding

$$f_n = \frac{1}{\sqrt{5}}\varphi^n - \frac{1}{\sqrt{5}}\psi^n = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

■

Note (Consistency check with §6.7.3). Since $|\psi| < 1$, $\psi^n \to 0$, so $f_n \approx \varphi^n/\sqrt{5}$ for large n – consistent with the squeeze $r_1^{-2} \le f_n/r_1^n \le 1$ obtained purely by induction, without ever solving the characteristic equation.

Note (A genuinely surprising fact). f_n is always an integer, yet the formula involves $\sqrt{5}$ throughout – the irrational parts exactly cancel in $\varphi^n - \psi^n$ for every n. In fact f_n is the *nearest integer* to $\varphi^n/\sqrt{5}$, since $|\psi^n/\sqrt{5}| < \frac{1}{2}$ for all $n \ge 1$.

6.8 Limits of infinite sequences

Definition 6.41 (Limit of a sequence, ε-N definition)**.** (a_n) has *limit* ℓ, written $\lim_{n\to\infty} a_n = \ell$ (or $a_n \to \ell$), iff

$$\forall \varepsilon > 0 \; \exists N \in \mathbb{N} \; \forall n \ge N \quad |a_n - \ell| < \varepsilon.$$

If such ℓ exists, (a_n) is *convergent*; otherwise *divergent*.

Example 6.42. $\lim_{n\to\infty} \dfrac{1}{n} = 0$: given $\varepsilon > 0$, take $N > 1/\varepsilon$; then $n \ge N \Rightarrow 1/n \le 1/N < \varepsilon$, so $|1/n - 0| < \varepsilon$.

Example 6.43. $\lim_{n\to\infty} \dfrac{n}{n+1} = 1$: given $\varepsilon > 0$,

$$\left| \frac{n}{n+1} - 1 \right| = \frac{1}{n+1} < \varepsilon \iff n > \frac{1}{\varepsilon} - 1,$$

so take N any integer $> \frac{1}{\varepsilon} - 1$.

Theorem 6.44 (Uniqueness of the limit)**.** *If $a_n \to \ell_1$ and $a_n \to \ell_2$, then $\ell_1 = \ell_2$.*

Proof. Suppose $\ell_1 \ne \ell_2$; let $\varepsilon = |\ell_1 - \ell_2|/2 > 0$. By the two limit definitions, there exist N_1, N_2 with $|a_n - \ell_1| < \varepsilon$ for $n \ge N_1$ and $|a_n - \ell_2| < \varepsilon$ for $n \ge N_2$. For $n \ge \max(N_1, N_2)$, the triangle inequality gives

$$|\ell_1 - \ell_2| \le |\ell_1 - a_n| + |a_n - \ell_2| < \varepsilon + \varepsilon = |\ell_1 - \ell_2|,$$

a contradiction. So $\ell_1 = \ell_2$. ∎

Theorem 6.45 (Convergent sequences are bounded)**.** *If (a_n) converges, then (a_n) is bounded: $\exists M > 0$, $|a_n| \le M$ for all n.*

Proof. Let $a_n \to \ell$. Taking $\varepsilon = 1$ in the definition, there is N with $|a_n - \ell| < 1$ for all $n \ge N$, so $|a_n| < |\ell| + 1$ for those n. Let

$$M = \max\big(|a_1|, \dots, |a_{N-1}|, \; |\ell| + 1\big).$$

Then $|a_n| \le M$ for every n. ∎

Theorem 6.46 (Algebra of limits)**.** *If $a_n \to a$ and $b_n \to b$, then:*

$$a_n + b_n \to a + b, \qquad a_n - b_n \to a - b, \qquad a_n b_n \to ab, \qquad \frac{a_n}{b_n} \to \frac{a}{b} \;\; (b \ne 0, \; b_n \ne 0).$$

Proof of the product rule, as a representative example. Since (a_n) converges, it is bounded (previous theorem): $|a_n| \le M$ for all n. Given $\varepsilon > 0$, choose N_1 so that $n \ge N_1 \Rightarrow |a_n - a| < \dfrac{\varepsilon}{2(|b|+1)}$, and N_2 so that $n \ge N_2 \Rightarrow |b_n - b| < \dfrac{\varepsilon}{2M}$. For $n \ge \max(N_1, N_2)$:

$$|a_n b_n - ab| = |a_n b_n - a_n b + a_n b - ab| \le |a_n||b_n - b| + |b||a_n - a| < M \cdot \frac{\varepsilon}{2M} + |b| \cdot \frac{\varepsilon}{2(|b|+1)} < \varepsilon.$$

■

Theorem 6.47 (Squeeze theorem)**.** *If $a_n \le c_n \le b_n$ for all n (eventually), and $a_n \to \ell$, $b_n \to \ell$, then $c_n \to \ell$.*

Proof. Given $\varepsilon > 0$, choose N_1, N_2 with $|a_n - \ell| < \varepsilon$ for $n \ge N_1$ and $|b_n - \ell| < \varepsilon$ for $n \ge N_2$. For $n \ge \max(N_1, N_2)$:

$$\ell - \varepsilon < a_n \le c_n \le b_n < \ell + \varepsilon,$$

so $|c_n - \ell| < \varepsilon$. ■

Example 6.48 (Applying the squeeze theorem – revisits §6.7.3)**.** The bounds $r_1^{-2} \le f_n / r_1^n \le 1$ established for Fibonacci (§6.7.3) do *not* by themselves give $\lim f_n / r_1^n$ – the squeeze theorem needs *both* bounding sequences to converge to the *same* limit, which r_1^{-2} (constant) and 1 (constant) do not (they're different constants) – so this only shows f_n / r_1^n stays bounded, not that it converges. (It does in fact converge, to $1/\sqrt{5}$, by Binet's formula, §6.7.4 – but that requires more than the squeeze theorem alone.)

Theorem 6.49 (Monotone convergence theorem)**.** *Every bounded, monotonic sequence converges.*

Note (Proof idea). Relies on the completeness of $\mathbb{R}$ (every bounded set of reals has a supremum): if (a_n) is increasing and bounded above, $\ell = \sup\{a_n\}$ exists, and one shows $a_n \to \ell$ directly from the definition of supremum. This is genuinely a deeper real-analysis fact (completeness of $\mathbb{R}$), not provable from the algebraic properties of $\mathbb{R}$ alone – it fails, for instance, over $\mathbb{Q}$.

Lemma 6.50 (Cauchy Mean Value Theorem)**.** *For f, g continuous on $[a, x]$, differentiable on (a, x), with $g' \ne 0$ throughout (a, x): there exists $\xi \in (a, x)$ with*

$$\frac{f(x) - f(a)}{g(x) - g(a)} = \frac{f'(\xi)}{g'(\xi)}.$$

This generalizes the ordinary Mean Value Theorem (the case $g(t) = t$).

Theorem 6.51 (L'Hôpital's rule, 0/0 form)**.** *Let f, g be differentiable on (a, b) with $g'(x) \ne 0$ throughout, and suppose $\lim_{x \to a^+} f(x) = \lim_{x \to a^+} g(x) = 0$. If $\lim_{x \to a^+} \dfrac{f'(x)}{g'(x)} = L$ exists (finite or $\pm\infty$), then*

$$\lim_{x \to a^+} \frac{f(x)}{g(x)} = L.$$

Proof. Extend f, g to $[a, b)$ by setting $f(a) = g(a) = 0$ (continuous, by the hypothesis on the one-sided limits). Fix $x \in (a, b)$. By the Cauchy Mean Value Theorem (Lemma above) applied to f, g on $[a, x]$, there exists $\xi \in (a, x)$ with

$$\frac{f(x) - f(a)}{g(x) - g(a)} = \frac{f'(\xi)}{g'(\xi)}, \qquad \text{i.e.} \qquad \frac{f(x)}{g(x)} = \frac{f'(\xi)}{g'(\xi)}$$

(using $f(a) = g(a) = 0$). As $x \to a^+$, also $\xi \to a^+$ (since $a < \xi < x$), so

$$\lim_{x \to a^+} \frac{f(x)}{g(x)} = \lim_{\xi \to a^+} \frac{f'(\xi)}{g'(\xi)} = L.$$

■

Example 6.52 (Applying L'Hôpital's rule to a sequence limit – via the corresponding function limit)**.** For $a_n = \dfrac{\ln n}{n}$: consider $f(x) = \ln x,\ g(x) = x$ on $[1, \infty)$, both $\to \infty$ as $x \to \infty$ (∞/∞ form, same proof idea applies with $x \to \infty$ in place of $x \to a^+$). Then

$$\lim_{x \to \infty} \frac{f'(x)}{g'(x)} = \lim_{x \to \infty} \frac{1/x}{1} = 0,$$

so $\lim_{x \to \infty} \dfrac{\ln x}{x} = 0$, and since a_n is exactly this function evaluated at integer $x = n$, $\lim_{n \to \infty} a_n = 0$ too.

Note (Why sequences need this detour through functions). L'Hôpital's rule is stated for functions of a real variable x (it needs derivatives, which sequences – defined only on integers – don't have). To use it on a sequence limit $\lim_{n \to \infty} a_n$, first find a differentiable function f agreeing with a_n at integer points, compute $\lim_{x \to \infty} f(x)$ by L'Hôpital, then conclude the sequence has the same limit.

Note (The seven indeterminate forms L'Hôpital's rule (and its variants) address).

$$\frac{0}{0}, \quad \frac{\infty}{\infty}, \quad 0 \cdot \infty, \quad \infty - \infty, \quad 0^0, \quad \infty^0, \quad 1^\infty.$$

The last three ($0^0,\ \infty^0,\ 1^\infty$) reduce to the $0/0$ or ∞/∞ form by taking ln of the expression first, applying L'Hôpital to the resulting log-limit, then exponentiating the result.

Theorem 6.53 (L'Hôpital's rule, ∞/∞ form)**.** *Let f, g be differentiable on (a, b) with $g'(x) \neq 0$ throughout, and suppose $\lim_{x \to a^+} f(x) = \lim_{x \to a^+} g(x) = \infty$ (or $-\infty$). If $\lim_{x \to a^+} \dfrac{f'(x)}{g'(x)} = L$ exists, then $\lim_{x \to a^+} \dfrac{f(x)}{g(x)} = L$.*

Note. Same conclusion as the $0/0$ case, but the proof is genuinely different (the extend-by-continuity trick used for $0/0$ doesn't apply when $f, g \to \infty$) – it instead uses the Cauchy Mean Value Theorem on a shrinking interval near a together with a direct ε-argument, rather than a single application at a fixed point.

Note (Variants of the limiting process). The rule (both forms) holds unchanged when $x \to a^+$ is replaced by $x \to a^-$, $x \to a$ (two-sided), $x \to \infty$, or $x \to -\infty$ – only the specific mechanics of the proof (e.g. the substitution $x = 1/t,\ t \to 0^+$ to reduce $x \to \infty$ to $t \to 0^+$) differ.

Example 6.54 ($0 \cdot \infty$ reduced to $0/0$)**.** $\lim_{x\to 0^+} x \ln x$: rewrite as $\lim_{x\to 0^+} \dfrac{\ln x}{1/x}$ (∞/∞ form). L'Hôpital:

$$\lim_{x\to 0^+} \frac{1/x}{-1/x^2} = \lim_{x\to 0^+} (-x) = 0.$$

Example 6.55 (1^∞ reduced via logarithm)**.** $\lim_{x\to\infty} \left(1+\dfrac{1}{x}\right)^x$: let $y = \left(1+\dfrac{1}{x}\right)^x$, so $\ln y = x \ln\left(1+\dfrac{1}{x}\right)$, a $\infty \cdot 0$ form, rewritten as

$$\lim_{x\to\infty} \frac{\ln(1+1/x)}{1/x} \quad (0/0 \text{ form}).$$

L'Hôpital (differentiating with respect to x, using the chain rule on $\ln(1+1/x)$):

$$\lim_{x\to\infty} \frac{\frac{-1/x^2}{1+1/x}}{-1/x^2} = \lim_{x\to\infty} \frac{1}{1+1/x} = 1.$$

So $\ln y \to 1$, hence $y \to e^1 = e$.

Note (A caution, worth stating plainly). L'Hôpital's rule is a *sufficient*, not necessary, condition: $\lim f/g$ can exist even when $\lim f'/g'$ does not (e.g. oscillating derivatives). The rule is also only valid when the hypotheses (differentiability, $g' \neq 0$, and the indeterminate form itself) genuinely hold – applying it to a non-indeterminate ratio, or repeatedly past the point where the form is still $0/0$ or ∞/∞, gives wrong answers.

Definition 6.56 (Subsequence)**.** Given $(a_n)_{n=1}^\infty$ and a strictly increasing sequence of indices $n_1 < n_2 < n_3 < \cdots$, the sequence $(a_{n_k})_{k=1}^\infty$ is a *subsequence* of (a_n).

Theorem 6.57 (Bolzano-Weierstrass theorem)**.** *Every bounded sequence in $\mathbb{R}$ has a convergent subsequence.*

Proof sketch, bisection. Let $(a_n) \subseteq [A, B]$. Bisect $[A, B]$ into two closed halves; at least one half contains a_n for infinitely many n – call it I_1, and pick any $a_{n_1} \in I_1$. Bisect I_1 similarly, get $I_2 \subset I_1$ (half the length) still containing infinitely many terms, pick $a_{n_2} \in I_2$ with $n_2 > n_1$. Repeating gives nested intervals $I_1 \supset I_2 \supset \cdots$ with lengths $\to 0$, and a subsequence $a_{n_k} \in I_k$. By the Nested Interval Property (a consequence of completeness of $\mathbb{R}$), $\bigcap_k I_k$ is a single point ℓ, and $a_{n_k} \to \ell$ since $|a_{n_k} - \ell| \leq \text{length}(I_k) \to 0$. ∎

Definition 6.58 (Limit superior, limit inferior, Rudin)**.** For a bounded sequence (a_n):

$$\limsup_{n\to\infty} a_n = \lim_{N\to\infty} \Big(\sup_{n\geq N} a_n\Big), \qquad \liminf_{n\to\infty} a_n = \lim_{N\to\infty} \Big(\inf_{n\geq N} a_n\Big).$$

Note (Always defined, unlike the limit itself). $\sup_{n\geq N} a_n$ is nonincreasing in N (bounded below, since (a_n) is bounded), so it converges – meaning lim sup always exists for a bounded sequence, even when $\lim a_n$ does not. Similarly for lim inf.

Theorem 6.59. *(a_n) converges to ℓ $\iff$ $\limsup a_n = \liminf a_n = \ell$.*

Example 6.60. $a_n = (-1)^n$: $\limsup a_n = 1$, $\liminf a_n = -1$ – unequal, so (a_n) diverges, but both quantities are still well-defined.

Definition 6.61 (Cauchy sequence)**.** (a_n) is *Cauchy* iff

$$\forall \varepsilon > 0 \ \exists N \in \mathbb{N} \ \forall m, n \geq N \quad |a_n - a_m| < \varepsilon.$$

Theorem 6.62 (Cauchy criterion, Rudin)**.** (a_n) *converges (in* $\mathbb{R}$*)* $\iff$ (a_n) *is Cauchy.*

Proof sketch. ($\Rightarrow$) If $a_n \to \ell$: given $\varepsilon > 0$, take N with $|a_n - \ell| < \varepsilon/2$ for $n \geq N$; then for $m, n \geq N$, $|a_n - a_m| \leq |a_n - \ell| + |\ell - a_m| < \varepsilon$.

($\Leftarrow$) If (a_n) is Cauchy: it is bounded (same argument as convergent $\Rightarrow$ bounded), so by Bolzano-Weierstrass it has a convergent subsequence $a_{n_k} \to \ell$. The full Cauchy condition (terms eventually all close to *each other*) then forces the whole sequence, not just the subsequence, to converge to that same ℓ. ■

Note (Why this matters – proving convergence without knowing the limit). The Cauchy criterion lets you prove (a_n) converges *without ever identifying* ℓ – purely from how close later terms get to each other. This is the standard tool when no closed form for the limit is available, and it is what makes $\mathbb{R}$ *complete* (every Cauchy sequence converges) – a property $\mathbb{Q}$ notably lacks (e.g. decimal truncations of $\sqrt{2}$ are Cauchy in $\mathbb{Q}$ but converge to $\sqrt{2} \notin \mathbb{Q}$).

Theorem 6.63 (L'Hôpital's rule, Rudin's stronger version – allows $L = \pm\infty$)**.** *Let* f, g *be real and differentiable on* (a, b)*,* $g'(x) \neq 0$ *for all* $x \in (a, b)$*, and suppose*

$$\frac{f'(x)}{g'(x)} \to A \quad \text{as } x \to a,$$

where A *may be* $+\infty, -\infty$*, or a real number. If either* $f(x), g(x) \to 0$ *as* $x \to a$*,* or $g(x) \to +\infty$ *as* $x \to a$*, then*

$$\frac{f(x)}{g(x)} \to A \quad \text{as } x \to a.$$

Note (What's genuinely stronger here). Two upgrades beyond the 0/0 version proved earlier: (i) A is allowed to be infinite, not just a finite real number; (ii) the hypothesis on g is relaxed to *just* $g \to \infty$ – f need not tend to 0 or ∞ at all. The proof no longer reduces cleanly to a single Cauchy Mean Value Theorem application (which assumed $f(a) = g(a) = 0$ exactly) – Rudin's argument instead works directly with the defining ε-δ statement of $f'/g' \to A$, choosing points $x_1 < x_2$ near a carefully and bounding $f(x)/g(x)$ between values built from f'/g' at nearby points, without ever assuming f, g vanish at a.

6.9 Big-O notation

Definition 6.64 (Big-O)**.** For (a_n) nonnegative and $f : \mathbb{N} \to \mathbb{R}$:

$$a_n = O(f(n)) \iff \exists c, N \ \forall n \geq N \quad a_n \leq c \cdot f(n).$$

Example 6.65. For $t_n = 100n + 10n^2 + 2n + \log n$: for $n \geq 10$, each of $100n, 10n^2, \log n \leq 2^n$, so

$$t_n \leq 4 \cdot 2^n \quad (n \geq 10), \qquad \text{hence } t_n = O(2^n).$$

Note (Big-O swallows constant factors). $t_n = O(2^n)$, $O(4 \cdot 2^n)$, $O(0.001 \cdot 2^n)$ are all correct – since $2^n = O(0.001 \cdot 2^n)$ too (take $c = 1000$). So constants inside the $O(\cdot)$ are pointless; state $O(2^n)$, not $O(4 \cdot 2^n)$.

Note (Base matters, unlike constant factors). $t_n = O(3^n)$ is also true (looser, since $2^n \leq 3^n$), but $t_n = O(1.9^n)$ is *false* – no constant c makes $c \cdot 1.9^n$ eventually dominate 2^n. Exponential *bases* are not interchangeable, unlike constant multipliers.

Example 6.66 (Dominant-term technique)**.**

$$\tfrac{1}{8}n^3+\tfrac{1}{4}n^2+\tfrac{1}{2}n+1 = O(n^3), \qquad \log n+\log\log n = O(\log n), \qquad (\sqrt{n}+3n+9^{1/n})(17+12\log n) = O(n\log n).$$

Definition 6.67 (Big-Omega, Big-Theta, Rosen §3.2)**.**

$$f(n) = \Omega(g(n)) \iff \exists c, N\ \forall n \geq N\ f(n) \geq cg(n) \quad \text{(lower bound; } g = O(f)\text{)},$$

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)) \quad \text{(matching bounds both ways)}.$$

Note. O alone (CN's focus) only gives an upper bound – e.g. $n = O(n^2)$ is true but loose. Θ pins growth down exactly (up to constants) – $t_n = \Theta(2^n)$, but *not* $t_n = \Theta(3^n)$, since 3^n is not $O(t_n)$.

Definition 6.68 (Time complexity of an algorithm, Rosen §3.3)**.** The *worst-case time complexity* of an algorithm is $f(n) = O(g(n))$ if the number of operations it performs on *any* input of size n is $O(g(n))$.

Example 6.69 (Linear search, Rosen – harder, complete algorithm analysis)**.** Search a list of n items for a target, comparing one at a time until found or the list ends.

- *Worst case*: target absent (or last), n comparisons: $\Theta(n)$.
- *Best case*: target first, 1 comparison: $\Theta(1)$.
- *Average case* (target present, each position equally likely): $\frac{1+2+\cdots+n}{n} = \frac{n+1}{2} = \Theta(n)$ (§6.11's sum formula).

Example 6.70 (Binary search, Rosen)**.** On a sorted list of n items, each comparison halves the remaining search space: $T(n) = T(n/2) + O(1)$, giving $T(n) = O(\log n)$ (§6.6/§8.3's divide-and-conquer machinery, master theorem case $a = 1, b = 2, d = 0$: $a = b^d$).

Note (Why worst-case dominates in practice). CN's emphasis on upper bounds (via O rather than Θ or average-case analysis) matches how complexity is usually reported: a guaranteed ceiling on resource use is often more actionable for budgeting than an exact or average figure, especially when the exact behavior is too complex to pin down precisely (CN's own motivation for introducing O at all).

Definition 6.71 (Polynomial-time algorithm, tractability)**.** An algorithm is *polynomial-time* if its worst-case complexity is $O(n^k)$ for some fixed k – called *tractable*. Complexity like $O(2^n)$ or $O(n!)$ is *intractable*: even modest n makes it infeasible to run.

Example 6.72 (Why the distinction matters – concrete numbers)**.** At 10^9 operations/second: $n = 100$, $O(n^3)$ (10^6 ops) takes a fraction of a second; $O(2^n)$ ($2^{100} \approx 10^{30}$ ops) takes vastly longer than the age of the universe. The gap between polynomial and exponential dwarfs any constant-factor speedup from faster hardware.

Definition 6.73 (Class P, class NP, Rosen – harder, beyond CN)**.** P = decision problems solvable by a polynomial-time algorithm. NP = decision problems whose "yes" answers have a polynomial-time *verifiable* certificate (a solution can be checked quickly, even if it might be hard to *find*).

Note. $P \subseteq NP$: a polynomial-time solver is trivially also a polynomial-time verifier (just re-solve and compare). Whether $P = NP$ – whether every efficiently-*verifiable* problem is also efficiently *solvable* – is the most famous open problem in computer science (a Clay Millennium Prize problem), unresolved despite decades of effort.

Note (Connection to earlier material). The n-queens / Sudoku satisfiability encodings (§4's Satisfiability section) are instances of NP: a proposed solution (a truth assignment) is checked in polynomial time by just evaluating the CNF formula, but no polynomial-time algorithm for *finding* a satisfying assignment is known for general CNF – SAT is, in fact, *NP-complete*: every problem in NP can be reduced to it in polynomial time, so a polynomial-time SAT solver would imply $P = NP$.

6.10 Sums and summation

Note (Notation).

$$\sum_{k=1}^{n} a_k = a_1 + a_2 + \cdots + a_n.$$

Example 6.74 (Double sum, swapping order – Rosen, harder)**.**

$$\sum_{i=1}^{n}\sum_{j=1}^{i} j = \sum_{j=1}^{n}\sum_{i=j}^{n} j = \sum_{j=1}^{n} j(n-j+1).$$

The first equality swaps summation order (the region $1 \le j \le i \le n$ described either "by rows" or "by columns"); the second collapses the inner sum since j is constant over $i = j, \ldots, n$, contributing $n - j + 1$ copies.

Example 6.75 (Telescoping sum – harder)**.**

$$\sum_{k=1}^{n} \frac{1}{k(k+1)} = \sum_{k=1}^{n} \left(\frac{1}{k} - \frac{1}{k+1}\right) = 1 - \frac{1}{n+1},$$

since every interior term cancels: $\left(1 - \frac{1}{2}\right) + \left(\frac{1}{2} - \frac{1}{3}\right) + \cdots + \left(\frac{1}{n} - \frac{1}{n+1}\right)$, leaving only the first and last pieces.

Theorem 6.76 (Abel summation (summation by parts) – beyond Rosen, genuinely advanced)**.** *For sequences* $(a_k), (b_k)$*, let* $A_n = \sum_{k=1}^{n} a_k$*. Then*

$$\sum_{k=1}^{n} a_k b_k = A_n b_n - \sum_{k=1}^{n-1} A_k (b_{k+1} - b_k).$$

Proof. Write $a_k = A_k - A_{k-1}$ (with $A_0 = 0$). Then

$$\sum_{k=1}^{n} a_k b_k = \sum_{k=1}^{n} (A_k - A_{k-1}) b_k = \sum_{k=1}^{n} A_k b_k - \sum_{k=1}^{n} A_{k-1} b_k.$$

Re-index the second sum ($j = k-1$): $\sum_{j=0}^{n-1} A_j b_{j+1} = \sum_{j=1}^{n-1} A_j b_{j+1}$ (since $A_0 = 0$). So

$$\sum_{k=1}^{n} a_k b_k = A_n b_n + \sum_{k=1}^{n-1} A_k (b_k - b_{k+1}) = A_n b_n - \sum_{k=1}^{n-1} A_k (b_{k+1} - b_k).$$

■

Note (The discrete analogue of integration by parts). A_n plays the role of an antiderivative of a_k; $b_{k+1} - b_k$ is a discrete derivative – $\int u\,dv = uv - \int v\,du$ with sums replacing integrals. This underlies proving convergence of series like $\sum \frac{(-1)^k}{k}$ that don't converge absolutely.

6.11 Finite series

Note. A *finite series* is the sum of a finite sequence's terms; a *partial sum* $S_n = \sum_{k=1}^{n} a_k$ of an infinite sequence.

6.12 Finite arithmetic series

Theorem 6.77.

$$\sum_{k=1}^{n} (a + (k-1)d) = n \cdot \frac{2a + (n-1)d}{2}.$$

Example 6.78 (Deriving $\sum k^2$ by telescoping, before proving it – zhangzhusheng-style construction)**.** Start from the identity

$$(k+1)^3 - k^3 = 3k^2 + 3k + 1.$$

Sum both sides for $k = 1, \ldots, n$. The left side telescopes:

$$\sum_{k=1}^{n} \left[(k+1)^3 - k^3\right] = (n+1)^3 - 1^3,$$

since every intermediate term cancels: $(2^3 - 1^3) + (3^3 - 2^3) + \cdots + ((n+1)^3 - n^3)$.

The right side sums termwise:

$$\sum_{k=1}^{n} (3k^2 + 3k + 1) = 3\sum_{k=1}^{n} k^2 + 3\sum_{k=1}^{n} k + n = 3\sum_{k=1}^{n} k^2 + 3 \cdot \frac{n(n+1)}{2} + n.$$

Equating and solving for $\sum k^2$:

$$(n+1)^3 - 1 = 3\sum_{k=1}^{n} k^2 + \frac{3n(n+1)}{2} + n \quad \Longrightarrow \quad \sum_{k=1}^{n} k^2 = \frac{(n+1)^3 - 1 - n - \frac{3n(n+1)}{2}}{3}.$$

Simplifying the numerator (common denominator 2, expand $(n+1)^3 = n^3 + 3n^2 + 3n + 1$):

$(n+1)^3 - 1 - n - \frac{3n(n+1)}{2} = n^3 + 3n^2 + 3n - n - \frac{3n^2+3n}{2} = \frac{2n^3+6n^2+4n-3n^2-3n}{2} = \frac{2n^3+3n^2+n}{2} = \frac{n(n+1)(2n+1)}{2}$.

Dividing by 3:

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}.$$

Note (Why this method matters). Unlike the induction proof below (which needs the formula *guessed* first), this telescoping method *derives* the formula from scratch – the same technique, using $(k+1)^4 - k^4$, derives $\sum k^3$, and in general $(k+1)^{m+1} - k^{m+1}$ derives $\sum k^m$ for any m, always reducing to lower-power sums already known.

6.13 Finite geometric series

Theorem 6.79. *For $r \neq 1$:*

$$\sum_{k=0}^{n} ar^k = a \cdot \frac{r^{n+1} - 1}{r - 1}.$$

Example 6.80 (Sum $\sum k \cdot 2^k$ – via differentiating a generating function, harder)**.** Start from $\sum_{k=0}^{n} x^k = \dfrac{x^{n+1} - 1}{x - 1}$. Differentiate both sides with respect to x:

$$\sum_{k=0}^{n} kx^{k-1} = \frac{(n+1)x^n(x-1) - (x^{n+1} - 1)}{(x-1)^2}.$$

Multiply by x and substitute $x = 2$:

$$\sum_{k=0}^{n} k2^k = 2\big[(n+1)2^n \cdot 1 - (2^{n+1} - 1)\big] = (n-1)2^{n+1} + 2.$$

6.14 Infinite geometric series

Theorem 6.81 (Convergence condition, via §6.8's rigorous limit machinery)**.**

$$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r} \iff |r| < 1.$$

Proof. The partial sums $S_n = a\dfrac{1 - r^{n+1}}{1 - r}$. By §6.8's example ($r^n \to 0$ for $|r| < 1$, established rigorously via ε-N), $S_n \to \dfrac{a}{1-r}$. For $|r| \geq 1$, r^n does not $\to 0$ (it diverges, or stays ≥ 1 in absolute value), so S_n has no finite limit. ■

6.15 Harmonic numbers

Definition 6.82 (Harmonic numbers)**.**

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

Theorem 6.83 (Divergence of the harmonic series)**.** *$H_n \to \infty$ as $n \to \infty$, despite $1/n \to 0$.*

Classical proof, grouping by powers of 2. Group terms:

$$H_{2^k} = 1 + \frac{1}{2} + \left(\frac{1}{3} + \frac{1}{4}\right) + \left(\frac{1}{5} + \cdots + \frac{1}{8}\right) + \cdots + \left(\frac{1}{2^{k-1}+1} + \cdots + \frac{1}{2^k}\right).$$

The block from $\frac{1}{2^{j-1}+1}$ to $\frac{1}{2^j}$ has 2^{j-1} terms, each $\geq \dfrac{1}{2^j}$, so the block sums to $\geq 2^{j-1} \cdot \dfrac{1}{2^j} = \dfrac{1}{2}$. There are k such blocks (for $j = 1, \ldots, k$), so

$$H_{2^k} \geq 1 + \frac{k}{2}.$$

Since $1 + k/2 \to \infty$, $H_{2^k} \to \infty$, and H_n is increasing, so $H_n \to \infty$. ■

Note. This is exactly the same bound proved earlier by induction (§3.11's harmonic-numbers theorem, $H_{2^n} \geq 1 + n/2$) – here derived directly, without induction, by explicit grouping.

Note (Contrast: convergent vs. divergent zero-limit sequences). $(1/n^2)$ and $(1/2^n)$ both $\to 0$ with *finite* sum (the latter by §6.14's geometric series). $(1/n)$ is the borderline case: it decays just slowly enough to have infinite sum – among decaying sequences, $1/n$ marks roughly where convergence of the sum starts to fail (the p-series $\sum 1/n^p$ converges for $p > 1$, diverges for $p \leq 1$).

Note (Euler's constant).

$$\lim_{n\to\infty} (H_n - \ln n) = \gamma = 0.5772\ldots,$$

so $H_n \approx \ln n + \gamma$ for large n. Whether γ is rational is unknown – an open problem.

Notation summary

Sets (Ch. 1)

$\in,\ \notin$	membership
$\emptyset,\ \{\}$	empty set
$\{x : P(x)\}$	set-builder
$[a,b],\ (a,b),\ [a,b),\ (a,b]$	intervals
$\mathbb{N}, \mathbb{Z}, \mathbb{Z}^+, \mathbb{Z}^-, \mathbb{Q}, \mathbb{R}, \mathbb{R}^+, \mathbb{R}^-, \mathbb{C}$	standard number sets
$\lvert A\rvert$	cardinality
$\Sigma,\ \Sigma^n,\ \Sigma^*$	alphabet, strings of length n, all strings
$\subseteq,\ \subsetneq,\ \supseteq$	subset, proper subset, superset
$\mathcal{P}(A)$	power set
$\binom{n}{k}$	binomial coefficient
$A - B$ (or $A \setminus B$), $\overline{A}$	set difference, complement
$\cup,\ \cap$	union, intersection
$\triangle$	symmetric difference
$\times,\ A^n$	Cartesian product, n-fold power

Functions and relations (Ch. 2)

$f : A \to B,\ f(x)$	function declaration, value
$\mathrm{dom}(f)$	domain
i_A	identity function
χ_A	indicator/characteristic function
c_a	constant function
$\lfloor x \rfloor,\ \lceil x \rceil$	floor, ceiling
$f\lvert_X$	restriction
f^{-1}	inverse function (or preimage, context-dependent)
$g \circ f$	composition
$f^{(n)}$	iterated composition
$e_k,\ d_k$	encryption/decryption (cryptosystem)
$R \subseteq A \times B,\ xRy$	binary relation, infix notation
$R(x),\ R^{-1}(y)$	image set, inverse-image set (relation)
$R \circ S$	relation composition
$R^{(n)},\ R^+$	iterated relation, transitive closure
$[x]_R$	equivalence class
$[a_{ij}]$	matrix
$\wedge, \vee$ (matrices)	meet, join (zero-one matrices)
$\odot$	Boolean matrix product
$\preceq$	partial order
$\vee, \wedge$ (posets)	join (LUB), meet (GLB)

Propositional logic (Ch. 4)

$T,\ F$	truth values

$\neg$	negation
$\wedge,\ \vee$	conjunction, disjunction
$\implies,\ \iff$	implication, biconditional
$\oplus$	exclusive-or
$\mid$	Sheffer stroke (NAND)
$\equiv$ (or $=$)	logical equivalence

Predicate logic (Ch. 5)

$P(x)$	predicate applied to a term
$\exists x,\ \exists x \in A$	existential quantifier, restricted
$\forall x,\ \forall x \in A$	universal quantifier, restricted
$\mathrm{dom}(f)$	(as in Ch. 2 – domain of a variable/predicate argument)

Sequences and series (Ch. 6)

$(a_n),\ f_n,\ (n^2 : n \in \mathbb{N})$	sequence, n-th term, formula notation
a, d	arithmetic sequence: first term, common difference
a, r	geometric sequence: first term, common ratio
φ, ψ	golden ratio and its conjugate (Fibonacci/Binet)
$O,\ \Omega,\ \Theta$	big-O, big-Omega, big-Theta
$P,\ NP$	complexity classes
$\sum,\ \prod$	summation, product notation
A_n	partial sum, Abel summation
$\lim_{n\to\infty} a_n,\ \varepsilon, N$	sequence limit, ε-N definition
$\limsup,\ \liminf$	limit superior, limit inferior
H_n	harmonic number
γ	Euler's constant
$G(x)$	generating function

Proof and general notation (Ch. 3, throughout)

■	end of proof
$\mid$ (number theory)	divides ($a \mid b$)
$\Rightarrow$ (proof steps)	logical consequence within a proof